SURVIVAL GUIDE FOR BEGINNER PROGRAMMERS

PARTE 1: FROM CHAOS TO CLARITY

ALICIA ORTEGA MUÑOZ

DERECHOS DE AUTOR / COPYRIGHT

© 2025 Alicia Ortega Muñoz. **All rights reserved.**

The **total or partial reproduction, distribution, transmission or storage** of this work by any means, whether electronic, mechanical, photocopying, recording or other information retrieval systems, **without the prior written permission of the author**, except for the exceptions permitted by law, is prohibited.

First digital edition: 2025
Cover design: Alicia Ortega Muñoz
Style correction / editing: Alicia Ortega Muñoz
Contact for permissions: Aliciaortega1986@gmail.com

This content **is not authorized for use in training artificial intelligence models** without the author's explicit permission.

Likewise, **it is strictly forbidden to obtain economic benefit** through the sale, resale or distribution of this eBook on platforms other than those authorized by the author. **The royalties generated by self-publishing platforms belong exclusively to the author.**

It is also not allowed **to facilitate, share or distribute this work through social networks**, including, but not limited to, **Telegram, WhatsApp, Facebook, Instagram, or any other digital channel**, without the express written consent of the author.

Any infringement will be immediately reported to the distribution channels involved and the competent authorities for copyright infringement.

INDEX

Start of the journey: Motivation, context and accompaniment

Introduction.

Prologue.

About me.

Join my tech communities on LinkedIn and Telegram.

Support me with this project and be part of my community in Ko-fi

Thanks.

Part I: Starting the Path

From Confusion to Clarity: My Personal Process for Finding Direction in the World of Programming.

Phases of learning from beginner to professional: Short and medium-term goals to structure your growth.

Defining your career goal What kind of programmer or developer do you want to be?

Trainer, junior, semi-senior and senior: Roles, salary expectations, responsibilities and realities of the professional path.

Part II: Technical Learning and Good Practices

Building the foundation, from essentials to microservices: How to move from the fundamentals to more complex architectures.

The challenge of microservices and the reality check: Common challenges when specializing and how to face them.

Essential tools for everyday life: Editors, terminal, version control, containers, CI/CD, etc.

Best practices from the start: Clean code, basic testing, documentation and SOLID principles for beginners.

Learning by Building: From Concept to README: How to Organize Personal Projects and Enhance Your Portfolio.

Part III: Personal and Professional Development

Programming is not just about writing code, **soft skills:** Communication, feedback, leadership and teamwork.

Mental health in tech Burnout, anxiety and strategies to maintain balance.

Final Thoughts Autonomy and Self-Discipline Why is autonomy essential in learning and professional development?

Conclusion

Lessons Learned and Survival Guide: Reflections for Beginners: From Chaos to Clarity.

Annex

Free resources for learning with and without certification: Recommended platforms, tools, and links.

Introduction

This book reflects my learning path in developing Java microservices software. Like anyone who starts out in the world of programming, I have encountered challenges, unexpected mistakes and moments of doubt. But I've also discovered solutions, helpful resources, and strategies that I want to share with you today.

My goal with this eBook is simple, to help you move faster, avoid some of the stumbling blocks I experienced and offer you a clear and accessible guide to accompany you in your training process as a developer.

Although I don't have formal work experience in the industry yet, I've built a solid foundation of knowledge through personal projects, self-taught training, and participation in my Telegram technical communities.

Since August 2024, I am the founder and mentor of the Telegram groups **"Programming for beginners"** and **"I want to learn Java",** where I share what I learn and learn by sharing.

Everything you'll read here comes from practice, constant curiosity, and commitment to my own professional growth.

Throughout these pages, I will share with you the approaches and tools that have worked for me, from the logical foundations of programming to the first steps to prepare professionally and build a technical profile that stands out to recruiters.

This is not a technical manual or an advanced academic guide. It's a practical map, focused on the essentials, on what you really need to know to start with confidence and build a firm foundation from which you can continue to grow.

If there's one thing I've learned on this journey, it's that learning is continuous and that every mistake overcome represents real progress. I hope this book will not only help you save time, but also inspire, guide, and motivate you to keep taking firm steps towards your goal of becoming a developer or programmer.

Welcome to this journey.

I am convinced that, with the right tools and a persevering attitude, you too can achieve it.

Prologue

This book is not born from the top, but from the road. Since those days when the code does not work, the curriculum does not seem to open doors and doubts appear more than once.

I write it from my own experience with successes, mistakes, learnings and a clear and honest intention to share what I would have liked to have had when I started.

You won't find magic formulas or absolute truths here. But you will find practical tools, real reflections and sincere words, written with the desire to accompany you.

It is an invitation. To breathe. To lower the pressure. To look at your path with more compassion. Because many times, due to anxiety or haste, we try to run before we are ready. We skip steps, we get frustrated, we compare ourselves.

And the truth is that there is no single correct rhythm. The important thing is to keep moving forward, even if it's slowly. Even if it is with doubts.

A path that starts from scratch

Although I do not yet have formal professional experience in the sector, I have built a solid foundation through personal projects, self-taught training, and participation in technical communities.

I coordinate and mentor group and individual in my **two Telegram tech groups**:

- ***Beginner Programming Group*** (over 670 members)
- ***I want to learn Java*** (more than 370 members)

There I accompany people who, like me, are building their future in tech, in groups and individually.

My intention with this book is simple, to accompany you, help you save time, give you clarity in the midst of noise and remind you that you are not alone.

I know perfectly well how it feels when you have already sent dozens of applications, you have a portfolio and your updated profiles, but still no call arrives.

Here we'll focus on understanding what may be going wrong, how to prepare yourself emotionally to deal with HR *ghosting*, and what you can do to really capture the attention of recruiters and employers.

What you'll find on these pages

A learning path explaining the steps we all must go through from learning programming logic to getting a job. Regardless of your specialty.

A realistic view of how the tech labour market works.

The management of expectations regarding salaries and their corresponding responsibilities.

This is a long-distance career with many learnings at all levels, such as the development of social skills will help you get a job in the tech sector.

The most common mistakes and how to fix them. Because I have also been confused with so much information and I did not have criteria to make my RoadMap at the beginning and I wasted a lot of time.

How to meet those requirements of the offers that job offers ask you for while you specialize in your learning path.

The importance of doing projects to learn and consolidate knowledge through experience making mistakes and successes, solving bugs and finding solutions autonomously, because you need to learn GIT and have your profile on GitHub-GitLab and how to make more complex projects for your portfolio for the CV. It helps you to have a complete tech profile.

Portfolio: More than technology, stories.

One of the things that has helped me understand the industry the most is this:
Your portfolio is not just a list of tools.
It's your most authentic letter of introduction, and it should tell stories. Stories like these:

- What problem did you set out to solve?
- How did you think of the solution?
- What technical decisions did you make?
- What did you learn in the process?

That is what leaves a mark. It's not just about what you know, it's about how you think, how you solve, and how you evolve.

LinkedIn and GitHub: Your professional showcases

Having visibility is fine, but the fundamental thing is to have a solid and well-communicated value proposition. Your profiles shouldn't just show what you've done but reflect who you are as a professional and where you're headed.

LinkedIn isn't just a public resume: It's a living network. It allows you to connect, share, learn and be present in the sector.

GitHub is your open workbook: It talks about how you structure your code, how you document, and how you've evolved over time.

Both platforms should function as an extension of you, reflecting not only your present, but also your professional future.

Mental Health: What Almost No One Tells You

Looking for a job can be a lonely experience.
We send CVs, personalize letters, update profiles... and many times we do not receive a "thank you for applying". That hurts and it's easy to start doubting everything.

That is why I want to remind you of something important. Your well-being can't be put on hold while you wait for something to happen.

Taking care of your mental health is also part of the process:

- Listen to your emotions without judging yourself.
- Maintain a minimal self-care routine.
- Learn to ask for help.
- We must remember that feeling lost does not mean that we are failing.

Personal brand: Your story matters.

You don't need to be a tech influencer to stand out. You just need to tell your story honestly:

- What moves you?
- How do you learn?
- What values guide you as a professional?

Your way of solving, your discoveries, your mistakes overcome... all that speaks about you. And it sets you apart. Much more than you imagine.

My formula so far

I don't have a magic recipe. But if I had to summarize what is helping me, it would be this:

Perseverance + real projects + community + personal honesty

And, above all, don't forget that the process is already transforming you. Every line of code you write, every email you send nervously, every interview you arrive at without certainty... all of that is already part of your growth.

About me

Java Backend Developer specialized in Microservices, mentor in tech communities and independent disseminator.

I am **a Software Developer specialized in Java and microservices architectures**, with a focus on the creation of scalable, efficient and maintainable backend applications. My daily work is focused on development with **Spring Boot**, using **PostgreSQL** and **MySQL databases**, within the **STS Eclipse** development environment, where I integrate and apply clean architecture principles and good software engineering practices.

Since the beginning of my career, I have been committed to solid and continuous training in the technologies that drive digital transformation in the field of backend development.

My interest in software quality is reflected in the constant application of **SOLID principles**, **design patterns**, **automated testing,** and **continuous integration practices**. In addition, I work under **agile methodologies** such as **Scrum**, collaborating proactively in multidisciplinary teams where communication, continuous improvement and value delivery are essential.

At the same time, I maintain a strong vocation for **technical dissemination and collaborative learning**. I share my projects and progress on **GitHub**, where I document my development process and explore new ways to optimize code and architectures.

I strongly believe in the importance of building a technical community that learns together, shares knowledge, and constantly evolves.

Currently, I continue to hone my skills in **Spring Cloud**, **Docker Compose** and **Kubernetes**, with the aim of mastering deployment in distributed environments and the automation of development pipelines.

My professional purpose is to continue growing as a developer specialized in backend and microservices architecture, providing innovative, clean and sustainable solutions that generate real impact.

Participation in technical communities

I believe deeply in the power of **collaborative learning**. That's why I'm **a founder, moderator, and mentor** in two tech communities active on Telegram:

- **Programming for Beginners** (more than 670 members), created in August 2024, aimed at people who are taking their first steps in programming.
- **I want to learn Java** (more than 370 members), where I accompany those who seek to specialize in this language.

For a long time, I offered free group and individual mentoring in these spaces, providing personalized guidance to developers in training, regardless of their technical level or specialization.

My purpose has always been to help them build a solid learning path, focused on achieving personal and professional goals effectively.

With the growth of the project and the evolution of my professional role, I have made the decision to no longer offer free individual mentoring on Telegram. This decision responds to a need to take care of my time, my energy and the balance between my personal, professional and creative life.

I currently channel this support through Ko-fi, where I offer more structured, sustainable and personalized services.

My path in tech is marked by the desire to continue learning, sharing and creating content that provides real value. Every project, every community, and every mentorship is an expression of that commitment.

Thank you for being here. This eBook, like everything I create, is designed to accompany you in your process, with respect, clarity and purpose.

Personal projects: Creating, sharing and accompanying

In addition to my technical work and my participation in communities, I have undertaken a personal project that represents the essence of who I am as a professional and mentor: **creating, sharing and accompanying**.

For some time now, I felt the need to contribute something beyond code. Technology is powerful, but behind each person who learns to program or who seeks to grow professionally, there is also a story, a motivation and, many times, a personal struggle.

For this reason, I decided to build a line of **educational content** designed to accompany those who want to develop in the technological and professional field **with purpose, clarity and emotional balance**.

This content is aimed at both people who are taking their first steps in programming, as well as professionals who are looking to consolidate their career or redefine their course within the tech sector.

My approach combines the technical with the human, because I firmly believe that **professional development is only sustainable if it is built from self-confidence, organization and personal clarity.**

As part of this project, I am building a **collection of self-published eBooks**, in Spanish and English, that will be available on **Amazon KDP**, **Ko-fi**, and other digital platforms.

Each book is part of an evolutionary journey, which goes from the technical fundamentals to the consolidation of a solid and authentic professional identity.

The contents are organized into thematic blocks that reflect my way of understanding the profession and learning:

- **Technical documentation** on tools, best practices and key concepts of modern development.
- **Java Developer Survival Guide:** Logic and Fundamentals of Programming from Scratch.
- **Survival Guide for Beginner Developers:** A Hands-On Introduction to Java Programming.
- **Java Microservices Developer Survival Guide:** Architecture, Spring Boot, and Spring Framework.
- **LinkedIn as a professional tool:** How to stand out, connect and position yourself in the technology sector?
- **Technological personal branding:** Real strategies to define your professional identity in today's job market.

- **Personal and professional development:** Emotional and strategic approach to grow with balance, avoiding burnout and demotivation.
- And many other topics that I will explore in future publications, always with the aim of inspiring, guiding and facilitating the path for those who come after me.

Currently, I am also working on a new eBook entitled:
"How to write, believe in yourself and self-publish with confidence? My experience, to help you take your first steps without fear."

This book is born from my own experience facing the fears, doubts and challenges involved in sharing your knowledge with the world. It is a sincere, close and practical guide, especially designed for those who feel that they have a lot to contribute, but do not know where to start.

Because not everyone is comfortable on **YouTube, Twitch, or personal blogs**, and that's perfectly fine. Everyone has their own way of communicating and connecting with others.

Choosing the medium that best aligns with your way of expressing yourself, whether it is writing, teaching or mentoring, is a strategic decision within your **Personal Branding Plan**. The important thing is not to be everywhere, but to build a digital presence that is coherent, authentic and true to your purpose.

My goal with these projects is not only to share knowledge, but **to accompany people in their learning and growth process**, helping them to trust in their potential, organize their path and turn their goals into concrete achievements.

Because learning to program or develop in technology is not only a matter of code, but also of **mentality, perseverance and self-knowledge**. And my purpose is to be there to guide, inspire and demonstrate that we can all build a meaningful career, step by step, with the right tools and a clear vision.

Training, technical skills and projects

In order to create practical content and projects that really help others, I have been building a solid base of knowledge in **Java, microservices and backend development**, complemented by training in **web development, testing, software architecture and version control**.

My learning combines **certified courses** and hands-on experiences in own projects, applying tools such as **Spring Boot, Git and GitHub**, as well as good software engineering practices and design patterns.

I am currently **expanding my knowledge of Docker, Kubernetes and Apache Kafka**, applying them in practical learning projects.

In addition to writing, I develop and maintain software projects that I share publicly in my GitHub repository:
github.com/AliciaJava

I currently have more than 24 public repositories, which I continue to update and expand as I acquire new knowledge. My projects are geared towards both Java microservices development and Java JDK 21 applications, and reflect my commitment **to continuous learning, personal improvement, and active contribution to the technology community**.

My focus is clear: to keep learning, sharing, and creating **useful, accessible, and honest** content. I hope these projects and resources will be of value to others who, like me, are building their way into the world of software development.

These skills allow me not only **to develop software professionally**, but also **to explain complex concepts in a clear and accessible way**, making learning easier for those who are taking their first steps in programming.

Connect with me

If you want to follow my work, share ideas or just say hello, you can find me here:

- LinkedIn
- GitHub
- Ko-fi

Thank you for being here. This project is yours too.

My role in my technical communities on Telegram

As the founder and active leader of two tech communities on Telegram, I'm committed to building inclusive, dynamic, and enriching spaces that foster the professional development of new talent in the field of programming.

My main mission is to empower developers in training, providing them not only with resources, but also with constant accompaniment and a solid support network, which allows them to move forward with confidence and motivation in their learning.

Communities I lead:

Beginner Programming Group (more than 670 members)

Founded on August 25, 2024, this community is specially designed for those who are taking their first steps in the world of development and programming.

A safe, friendly and accessible environment is promoted here, where new programmers can learn without fear of making mistakes. We encourage collaboration between peers, mutual support and continuous improvement, understanding that mistakes are a fundamental part of the learning process.

I Want to Learn Java (over 370 members)

This community specializes in the Java language, offering a space where both beginners and those who already have some experience can deepen their knowledge.

Through up-to-date resources, constructive discussions, and practical challenges, we work to strengthen each member's technical skills and enhance their ability to solve real problems with this language.

My role in both communities

Close and dedicated mentor

I accompany members in a personalized way, providing both individual and group mentoring. My main focus is to help each person map out learning paths that are realistic, effective, and tailored to their interests, time availability, and level of experience.

I am present to answer questions, offer technical guidance, and provide emotional support, motivating members to overcome challenges that arise along the way.

Disseminator and facilitator of knowledgeI constantly share carefully selected free resources to support self-taught learning, including:

- Documentation in PDF and EPUB formats on programming, software development and key tools of the technological environment.
- Online training platforms in computer science and programming, with and without certification, in Spanish and English, which allow you to learn at your own pace and without the need for a tutor.
- Short and extensive videos of leaders in the tech sector, whose objective is to invite reflection on the real challenges of the learning process and the work environment in technology.
- Links to interactive platforms to carry out practical laboratories in different programming specialties and essential tools for the profile of a developer.
- And many more useful resources that contribute to the comprehensive development of technical and soft skills.

My goal is to encourage technical curiosity, motivate constant exploration, and strengthen a mindset of continuous learning. Through these spaces, I promote the autonomy, collaboration, and personal and professional growth of each member.

Mentoring and autonomy in technical learning

In my Telegram groups, the main objective is **to foster autonomy, trust and responsibility of each member in their learning process**. Therefore, it is not a matter of solving academic tasks or looking for eBook's on demand.

Instead, we share **technical documentation and useful tools**, which serve as a basis for each person to build their own knowledge.

Every obstacle you encounter and manage to overcome is an opportunity to **strengthen your tolerance to frustration, consolidate knowledge and develop your problem-solving skills**.

These skills are essential in the work environment, especially in technology, where you often work under pressure, with limited delivery times and with increasing responsibilities.

Active learning involves **searching, comparing, and verifying sources** on your own. This process, although sometimes taking hours or even days, is highly formative. It develops your judgment, your agility to investigate and your ability to solve real problems.

Delegating this effort to others not only delays your learning, but it can also give an impression of a lack of initiative in professional settings and technical communities.

Many eBook's or resources circulate outdated. That's why I always recommend **going to official sources**, where you'll find tutorials, guides, and up-to-date documentation, whether it's a programming language, software, or framework.

Providing resources in the group does not mean doing the work for you but providing you with tools to learn independently and professionally.

My work as a mentor goes beyond the technical. I also **promote essential complementary skills**, such as self-discipline, critical thinking, problem-solving creativity, and teamwork skills.

*My goal is to accompany you so that you become a **professionally autonomous** person, capable of facing real challenges and solving them on your own.*

Group mentoring and individual follow-up

Group **mentoring** is done directly in the chats of my two Telegram groups. They are designed to **resolve doubts and promote collective learning**. A single question from one member can spark an enriching conversation, from which many other members benefit from shared answers, clarifications, and experiences.

There is also room for those who are more shy or prefer not to interact directly. These people can **take advantage of shared resources** and read the answers to questions raised by others, learning at their own pace without falling behind.

I want to clarify that **private mentoring within Telegram groups is no longer available**. For those who want **individualized attention or personalized follow-up**, it is best to request **a one-off mentoring** or explore the monthly plans according to the need, through my Ko-fi profile:
https://ko-fi.com/aliciaortegamunoz/commissions

In this way, it is possible to receive a **deeper and more structured accompaniment**, while group mentoring continues to offer an open and enriching collaborative learning space for all members.

The most frequently asked questions that arise during these mentorships include topics such as:

- How to draw a **realistic RoadMap** according to the chosen specialty (backend, FrontEnd, FullStack, etc.).
- The importance of **following a logical order in learning** and not skipping fundamental stages.
- Why **you shouldn't start programming directly without first understanding the logic and fundamentals**, as this lays the foundation for solid learning.
- How the "**level jump**" occurs when moving from basic exercises to small, medium, and finally large projects.
- The importance of learning how to use **Git** and hosting your projects on **GitHub or GitLab**, where you can see your *commits* and the evolution of your work.
- How to make a good **README.md**, well-structured and written in Markdown, that explains the development of the project, includes screenshots and relevant links.
- How to select **2- or 3-star projects** for your portfolio, with a brief explanation of the technology stack used. These projects should be visible on your GitHub or GitLab and ready to include in your CV.

- The need to have your professional profiles, such as **LinkedIn, GitHub or GitLab**, visible with functional and up-to-date links.

During these sessions we also address a fundamental aspect, **the emotional management of learning**. It's completely normal to feel insecurities, blockages, confusion, or frustration. These emotions are part of the process.

The important thing is **how you react, how you look for solutions** and, above all, knowing that **you are not alone**, that you have the support of people who have already gone through that same path.

That's why belonging to specialized technical communities is so valuable. Learning with others, sharing doubts, progress and resources not only helps you advance faster, but also strengthens your motivation and your connection with the professional reality of the tech world.

Individual mentoring: Personalized accompaniment for a path with purpose

In addition to my work as a mentor in Telegram communities, I also offer **personalized one-on-one mentoring**, designed for those people who need more direct, clear, and focused guidance on their situation.

These mentoring sessions are not long sessions or academic interventions; they are brief but meaningful meetings, where I dedicate myself to **listening carefully, analyzing the starting point of the learner and drawing up a realistic and motivating action plan**.

Initial diagnosis: understanding to guide

<u>In each mentoring I begin by identifying three key aspects:</u>

Current **needs**: What's holding you back? What stage of learning are you in?

Your **strengths**: What are you good at? In which areas are you most fluent or motivated?

Your **short, medium and long-term goals**: What do you want to achieve? Are you looking for your first job in tech? Or improve your skills to change roles?

With this information, I can help you build a learning plan that **is not only technical, but also strategic and emotionally sustainable**. Because learning programming and preparing for a career in technology isn't just about mastering a language or a tool, it's about staying consistent, focused, and motivated in the midst of difficulties.

Unlocking confusion and transforming information overload into focus

A very common situation I find is that of people completely **overwhelmed by the excess of information**. On the Internet there are thousands of tutorials, courses, books and RoadMap's, but precisely because of that, many feel paralyzed. They don't know where to start, in what order to advance, or what resources are worthwhile.

This is where I usually step in with a concrete guide, **define priorities**. First, we clarify the specialty (e.g., backend with Java), then we structure a step-by-step path, and finally, we learn how to validate whether that path makes sense according to the labor market.

To do this, I always recommend analyzing **real job offers**, seeing what technologies and knowledge are repeated, and using that information to adapt your RoadMap. Because it is not enough to follow a theoretical list: **you must study what brings you closer to your professional goal**.

Emotional accompaniment: When learning hurts

Not all blocks are technical. Many have to do with the mind, emotions, and expectations. I have accompanied people who arrive feeling "slow", "clumsy" or "too old to learn". And it is in these moments where human accompaniment makes the difference.

I've been there too. I remember how frustrating it was to get stuck on a bug for hours without understanding why the code wasn't working. I also remember my impatience for wanting to do advanced projects when I still didn't even understand the control structures well.

I even went through moments of total doubt about whether I should continue learning to program or if this was not for me.

And in the end, I understood something essential, **I was not on the wrong path, but I did lack direction and emotional support.** And that is precisely what I now try to offer in my mentorships.

<u>During individual mentoring I also focus on helping you:</u>

Make sound decisions about which technologies to learn and in what order.

Detect inefficient study habits and change them for more productive ones.

Organize your personal projects and make them visible on platforms such as GitHub or GitLab.

Create a strategic technical portfolio, with 2 or 3 well-documented star projects, with their respective README.md, screenshots, links and descriptions in Markdown.

Connect your skills with your professional profile for LinkedIn and your CV, clearly integrating your technology stack and your evolution.

In each session I also strive to **demystify the learning process**. It's not about being "the fastest" or "the one who makes the fewest mistakes", but about moving at your own pace and staying on track, without losing focus on comparing yourself to others.

Each person has their own rhythm: Conscious and measurable progress

One of the pieces of advice I give the most is, **don't compare yourself with others**. Although it is a natural tendency, comparing yourself will only lead to frustration. Everyone has their context, their available time, their responsibilities, and their way of learning.

For this reason, I propose the use of **Checklists or lists of objectives**, divided into large, medium and small blocks. This allows you to **objectively see your progress**, identify gaps, and recognize what you've already accomplished, even if it's not visible to the naked eye.

These skills complement your profile and are highly valued by recruiters, even beyond your technical proficiency.

These mentorships are not magic, nor an immediate solution. But if you are committed to your process and are willing to learn with humility, perseverance and strategy, **they can make a big difference in your development as a professional in the tech world**.

I am here to guide you, but the path is up to you. And I'm convinced that, with the right tools and a clear plan, you can also achieve it.

On my way of accompanying and how it has evolved

I currently have my profile active on **Ko-fi**, where I offer **weekly individual mentoring** with **personalized follow-up**. In these sessions, we work together to optimize your **professional profiles**, especially LinkedIn and GitHub, so that they reflect your real value and communicate in a clear and attractive way what you can contribute to the technology sector.

In addition, I share **exclusive documentation**, access to **memberships and subscriptions** adapted to different levels of support, and the possibility of **making donations** to support this project and keep it growing.

On my page you will also find my **eBook store**, with technical and professional resources designed to help you move forward with **autonomy, focus and strategy**.

And if you are looking for a more complete process, the **Tech Professional Brand Plan** accompanies you step by step to stand out as a professional, pass interviews and position yourself confidently in the job market.

This project has grown over time, and with it my way of accompanying has also evolved. **I currently no longer offer free individual or group mentoring** in my Telegram communities. It was a **conscious and necessary decision**.

My time, my knowledge and my experience have a value. Every mentorship, guide, or content I create is designed with a clear purpose, a solid structure, and a real commitment to the growth of those who trust my work.

I combine this project with my **personal and professional life**. In parallel, I am developing my own projects for my **portfolio as a developer of microservices in Java**, and I continue to write **technical eBooks** aimed at learning and professional development in technology.

I do all this in my **free time**, with dedication, perseverance and great care. At this stage I must also **send my CVs and personalized cover letters**, in addition to preparing to **obtain the driver's license** and continue with the **selection processes** in which I participate.

I recognize that it is a period of great pressure, so I also **need times of rest and self-care** to maintain balance.

People interested in accessing my mentoring can do so through my **Ko-fi page**, where you will find the corresponding links in my **eBooks**, in my **Telegram groups** or directly from my **LinkedIn profile**.

I've learned that setting **healthy boundaries** is an essential part of growing up. It allows me to offer quality support, sustainable over time and aligned with my personal and professional values.

I know that this decision may come as a surprise to those who knew me in previous stages, but I trust that they will also understand it. Accompanying others requires energy, presence and responsibility, and in order to continue **creating valuable content** and sustaining this project in the long term, I need to take care of my well-being.

Thank you for being here, for valuing this work, and for being part of a community that grows with purpose, authenticity, and a passion for learning.

Join here: https://ko-fi.com/aliciaortegamunoz

Join my tech communities on LinkedIn and Telegram

Do you want to learn, grow and connect with other programming enthusiasts?

Do you want to learn, grow and connect with other programming enthusiasts? My communities are the place for you. Here you will find free resources, personalized guidance and an inclusive environment where you can share knowledge, solve doubts and advance in your learning at your own pace.

Telegram Groups

Coding Group for Beginners

Join Here

This group is designed for those who are starting from scratch in programming. We offer you:

- Free technical and educational resources
- Group and individual counseling
- Courses and links to boost your learning

Our goal is to help you build a strong foundation that prepares you for success in the development world.

I want to learn Java

Join here

This space is focused on those who want to specialize in Java. Here you will find:

- Free technical and educational resources
- Group and individual counseling
- Courses and links to boost your learning

It is the ideal place for both beginners and developers who want to master this powerful language.

Connect on LinkedIn

I am Alicia Ortega Muñoz, passionate about programming, computer science and applied sciences. On my LinkedIn profile, with a community of more than 3,700 followers, I share quality technical content with the aim of disseminating relevant knowledge, generating value and encouraging the exchange of ideas within the technological ecosystem.

What can you find in my network?

- Dissemination of technical and educational content from other professionals in the sector, videos and useful resources for the community.
- Mentoring and networking.

I am open to connecting with professionals, students, entrepreneurs and companies interested in technological innovation. If you are looking to collaborate, exchange knowledge or promote projects, I will be happy to talk to you!

Feel free to contact me or send me a direct message.

Let's connect and empower learning and professional development in technology together!

Professional Profile: Alicia Ortega Muñoz on LinkedIn

Support me with this project and be part of my community in Ko-fi

This eBook is just one part of a much broader work that I develop as a **mentor**, **creator of technical content** and **disseminator in the tech world**. If what you have read so far has been useful to you, has inspired you or has accompanied you in a difficult moment, I want to invite you to be part of my community in **Ko-fi**.

In my profile, you'll find ways to collaborate and access exclusive content and personalized services designed to help you move forward with clarity and confidence in your path as a developer.

Perfect! I understand: you want to clarify that if someone signs up to support you directly as a content creator on Ko-fi, they must use the **"**Support" **option**, not the option to create their own Ko-fi as a creator. Here is the corrected and clarified text:

How to support myself in Ko-fi?

<u>If you want to support me, you have two options:</u>

1. **Registering on Ko-fi as a buyer/supporter You can create an account on** Ko-fi using the **"Support"** or **"Buy a Coffee"** option, not as a content creator. This option allows you to follow me and support my work on a recurring or one-off basis, keeping a history of your supports and receiving updates on my news.
2. **Through the direct links in this eBook Throughout the ebook you will find links to specific products, services or contributions. You can click on them and make your support without registering** on the platform. This option is ideal if you want to contribute quickly and easily.

Note: Both forms of support are equally valid and help me continue to create quality content and services. Thank you for your support!

Join here: https://ko-fi.com/aliciaortegamunoz

What can you find in my Ko-fi?

- **Comprehensive mentoring** (weekly or one-off) to accompany you at every stage of your professional development.
- **Optimization of LinkedIn and GitHub-GitLab profiles.**
- **Free specialized content** to help you progress in tech with focus and strategy. Such as **Technical Templates** to optimize your professional profile and **Mini Guides** to point you in the right direction.
- **Tech Professional Brand Plan:** It is designed to help you build your personal brand as a developer from scratch, with a clear, practical and emotionally sustainable roadmap.
- **eBook store** with practical and accessible resources.
- **Memberships with exclusive content** and **discount on all my eBooks**.
- **Personal blog** where I share reflections, learnings, successes and real mistakes of the tech path.
- And so many more things I'm creating for you!

Tech Professional Brand Plan: The 11 phases to build your professional path

This plan is the heart of my accompaniment as a mentor. It's designed to help you build your personal brand as a developer from scratch, with a clear, practical, and emotionally sustainable roadmap. Here are the 11 phases that we work on in depth:

1. **Definition of objectives and professional vision**
   We clarify what type of life and career you want to build, your goals, values and motivations.
2. **Initial diagnosis of skills and knowledge**
   We evaluate your starting point: what you know, what you are missing and what you are interested in learning.
3. **Personalized technical roadmap**
   We design a learning plan adapted to your interests and the current job market.
4. **Professional profile optimization**
   We enhance your LinkedIn, GitHub/GitLab, and other key platforms so that they communicate your real value.

5. **Strategic choice of projects for the portfolio**
   We select and develop projects that showcase your skills and position you as a professional.
6. **CV and cover letter writing**
   We create clear, coherent documents adapted to each type of job offer.
7. **HR & Technical Interview Preparation**
   We practice answers, review frequently asked questions, and work on how to communicate your knowledge clearly and confidently.
8. **Follow-up of selection processes**
   You learn to record, analyze and learn from each interview, technical test and contact with companies.
9. **Consolidation of your digital presence**. **Professional tech brand.**
   I teach you how to share content, participate in discussions and add value in tech communities.
10. **Strategies for networking and collaboration**
    I help you connect with other professionals, mentors and real opportunities for growth.
11. **Support in the transition to the first job**
    I support you in adapting to your new role, in emotional management and in planning your next steps.

No matter your specialty: The accompaniment is tailored to you

One of the strengths of this plan is its **flexibility**. It doesn't matter if you are focusing on **FrontEnd, BackEnd, FullStack, Cybersecurity, Testing, DevOps** or any other area, the accompaniment adapts to your profile, your interests and your learning pace.

Although my main specialization is in **Java and microservices**, I have a transversal vision of development and technology thanks to my experience in different professional contexts.

In addition, during a stage of my working life, I was **a career counselor** and **trainer in computer science**, which gave me a more human, inclusive and pedagogical perspective of technological learning.

That experience taught me that each person has a rhythm, a story and a unique way of learning. That's why my mentorships are not only focused on the technical, but also on helping you build a **solid personal brand**, a **clear professional strategy** and real **confidence** in your abilities.

How does mentoring work?

All mentoring, both comprehensive with follow-up and one-off, is carried out **through email**. This format allows me to give you detailed, personalized and well-thought-out answers, adapted to your situation and learning pace.

I respond on weekends, as I combine this project with my personal life and my work as **a microservices developer in Java**. This allows me to maintain a healthy balance and ensure that each response has the attention and care it deserves.

<u>This email mentoring system is ideal if:</u>

- You need technical or emotional guidance in your career transition process.
- You want to move forward with autonomy, but with accompaniment.

Your growth is important to me and even if we are not in real time, you will be accompanied every step of the way.

This plan is part of the **exclusive memberships** available on my Ko-fi profile, along with other resources such as templates, mini-guides, free specialized content, and discounts on all my eBooks.

Join here: https://ko-fi.com/aliciaortegamunoz/

Why join?

Because your growth deserves accompaniment. Because you are not alone on this path. Because building a career in technology is not just about learning to code, it's about learning to trust yourself, to communicate your value, to make decisions clearly, and to support yourself emotionally at every stage of the process.

Joining my community on **Ko-fi** means accessing much more than technical content. It is to be part of a space where learning is human, progress is shared and each step is accompanied. There are no magic formulas or empty promises here. There is structure, strategy and empathy.

I support this project in my free time, while I look for a job as a software developer specializing in microservices. I combine it with my personal life, my rest and my creative process. That's why every resource I create, every mentorship I offer, and every word I write has intention, care, and purpose.

Join here: https://ko-fi.com/aliciaortegamunoz/

Your support not only allows me to continue creating, it allows more people to access resources that transform their professional lives. It allows this project to continue growing without losing its essence.

Thanks

I want to start by expressing a very special thank you to my mother, **Genara**, for her unconditional love, her infinite patience and her constant support. Your presence has been my refuge and my source of strength in the most challenging moments of this process. None of this would have been possible without his example of dedication, perseverance and tenderness.

To my aunt **Mari Carmen**, thank you for always believing in me, for accompanying me in my blocks, for encouraging me when I needed it most and reminding me, in every word and gesture, that I can achieve everything I set out to do.

This path has undoubtedly been both a professional and personal journey. For this, I also want to thank my dear friends from the bottom of my heart:

Esther, who has always been there to listen to me and support me in each new project.

Estrella, who has never stopped believing in me or my dreams, even in the moments when I doubted them.

Teresa, my most optimistic and spiritual friend. Their company, their unconditional support and their encouragement during this stage full of uncertainty, evolution and learning have been an invaluable emotional pillar.

My sincere thanks also go to all the people who have been an essential part of my journey as a mentor and community leader. To my colleagues and friends in the tech world, thank you for motivating me to share my knowledge, for trusting me, and for the constant encouragement that drives me to continue growing.

A very special recognition to **Iván**, my mentor in the most recent stage of my training. Their close guidance, unconditional support, and ability to understand both my strengths and insecurities have been instrumental in my technical and personal development.

To **Ricardo**, for his constant commitment and generosity in resolving doubts both at the group and individual level in our communities. Their collaboration has been essential for the proper functioning and continuity of the groups, especially at times when I could not actively participate. I deeply thank you for your trust in me and in the vision that drives me.

To **Jairo**, who helped me significantly improve the writing of my LinkedIn profile. Thank you for giving me a new perspective, for the rich draft I adapted with my personal touch, and for helping me express my professional value more clearly. Also thank you for continuing to accompany me with your support in each new goal.

To **Dani Pica**, for providing technical documentation to so many members of our communities, for leading and co-directing a Telegram group without owners or administrators, for offering solutions in critical situations and for reinforcing a good climate in group discussions with reflections that provide clarity and maturity. Thank you for your fellowship and for your key role in maintaining the collaborative spirit.

Ricardo and Jairo have been the most active contributors to the *Programming for Beginners* and *I Want to Learn Java* communities. Their dedication, availability and commitment to those who are just starting out in programming are a constant source of inspiration. Their presence has been key to building collaborative, respectful and enriching learning spaces.

I also want to thank **Narciso** for his valuable help in improving and optimizing my LinkedIn profile, for his advice in times of doubt, and for trusting me even when I didn't. His proposal to write the second part of this eBook, just as he was finishing the first, was a spark of inspiration. It has taught me to imagine futures that I didn't know were within my grasp. Their support, vision, and faith in my potential have left an indelible mark on my path.

I also extend my gratitude to all open **education platforms** and **technical content creators** who, with generosity and vocation, share their knowledge online. Their resources have been instrumental in enriching learning within our communities and have contributed significantly to my continuing education.

I have not traveled this path alone. To all the people who, in one way or another, have been part of this stage of my life, **thank you**. Thank you for joining me, for inspiring me, and for believing in this mission of sharing, teaching, and growing together.

Part I: Starting the Path

From Confusion to Clarity: My Personal Process for Finding Direction in the World of Programming.

1. Trajectory: From confusion to clarity

STAGE 1: Confusion (2022–2023)

My interest in technology arose in a practical way, through the daily use of **GNU/Linux** and its diverse ecosystem. I experimented with several distributions such as *Ubuntu, Linux Mint, Edubuntu, Debian* and *Devuan* (a fork of Debian), adapting to different desktop environments: **GNOME, KDE, Cinnamon, MATE and XFCE**, according to the needs of each system and user.

During this period, I worked as a **Project Manager Professional (PMP)** in the development of IoT prototypes **using** ARM chips**, collaborating on software projects for devices such as tablets and mobiles.**

These experiences, focused on Linux environments, awakened in me the curiosity to understand the code behind the products, it was then that I decided to learn to program.

Organizing My Learning Environment

To facilitate my learning, I made a strategic decision, I use **two laptops**, each with a different operating system.

One with **Ubuntu** intended for my self-taught Linux practices.

Another with **Windows**, dedicated exclusively to Java development.

Although I knew I could use both systems on a single computer using dual boot, I preferred to physically separate them as a way to **compartmentalize my learning goals mentally and** technically. This decision helped me stay focused and avoid distractions.

SPECIALIZATION: BackEnd, FrontEnd or Full-Stack?

In the early stages, I was more attracted to the **BackEnd**, although I also considered training as a **Full-Stack Developer**, a specialty that is in high demand in the job market. However, over time, I identified a common mistake that I also made:

Mistake: Aspiring to be Full-Stack as a Short-Term Goal

Full-Stack profiles are typically aimed at **semi-senior or senior** professionals, as they require **years of experience** in both server-side and client-side development. Pretending to cover everything in the first 6 months or even the first year of training is unrealistic and can lead to unnecessary frustration.

The lesson was clear, you must **adapt your goals to your current level.**
The most sensible thing to do is to opt for entry-level positions such as **internships, interns or juniors**, while continuing with technical training and the development of personal projects.

Technical Learning: Start with the Right Thing

Another major mistake I had to overcome was my initial approach on how to learn to code:

Mistake: Learning programming without a proper environment

I started trying to learn **PHP BackEnd** by copying code snippets into notebooks, as if it were a theoretical subject. This method, although well-intentioned, was of little use. I came from sectors where you learned by memorizing notes, but I soon understood that programming requires **active practice and a functional environment**.

I decided to take a technical step, installed and configured **XAMPP on Windows**. Although it was a complex process, it was also rewarding. It allowed me to work locally with real technologies and experiment with **Laravel**, a framework that marked **my first major milestone of hands-on learning**.

REFLECTION-CONCLUSION

Learning to code is not about memorizing code, but about **practicing in real environments**, understanding what you do and **making mistakes to learn**.

It is essential to familiarize yourself with tools such as **IDEs (Integrated Development Environments),** as they make it easier to write, debug, and understand code.

Similarly, developing a solid foundation in **logic and programming fundamentals** is essential to understanding what you're writing, how it works, and how you can resolve errors when they arise.

This first stage taught me that true learning begins when you stop looking for magical methods and accept constant effort as part of the path. It's only when you have clarity about your approach, your goals, and your way of learning that you begin to move forward firmly.

2. Learning Path:

From Foundation to Microservices: How to build a solid foundation and move towards more complex architectures.

STAGE 2: Clarity (2023)

In 2023 I decided to take a firm step towards my professional development as a programmer. Up until that point I had explored in a self-taught way, but I felt the need to better structure my learning and move forward with a more solid guide. For this reason, I opted for specialized training with the accompaniment of tutors through **SEPE training programs**.

FrontEnd Course – 250 hours (June – August 2023)

<u>This course was a turning point. It allowed me to:</u>

- Discovering what he really enjoyed doing.
- Identify the areas in which it excelled.
- I confirmed that my passion leaned more towards functional development than visual.
- Understanding that **Java** was the language that best aligned with my long-term interests.

<u>I learned the pillars of web development:</u>

- **HTML and CSS3** for interface structure and styles.
- **JavaScript** to start working on client-side programming logic.

In addition, it was during this period that I adopted a **quarterly planning system**, which allowed me to balance guided learning with self-taught exploration of new topics and tools.

Backend Course – 300 hours (conducted in parallel)

While studying FrontEnd, I decided to complement my knowledge with a training in BackEnd using **PHP**. This experience was fundamental to:

- Understand how **relational databases work**, working with **MySQL** and **PostgreSQL**.
- Gain a more complete view of how the server and client interact.
- Laying a foundation that would later help me specialize in **Java BackEnd**, appreciating the value of **strong typing** and **real-time compilation**, which allow me to build more robust and easy-to-debug software.

Quarterly Planning: Moving Forward with Structure

SEPTEMBER – DECEMBER 2023

Java Logic and Programming Fundamentals Course (250 hours)

This course marked my first contact with a **professional IDE: Eclipse**. The structure of the course helped me to advance progressively:

1. Introduction with basic concepts applied through practical examples.
2. Integrative exercises that connected different thematic blocks.
3. More structured activities that simulated real applications.

Thanks to this training:

- I strengthened my **logical thinking**.
- I deeply understood the **syntax** and **key structures of the Java language**.
- I began to develop a personal study methodology based on constant practice and solving mistakes autonomously.

JANUARY – MARCH 2024

Monolithic Application Development – Basic Level

During this quarter:

- I switched to **NetBeans** as a new development environment, which forced me to step out of my comfort zone and adapt. I learned that, in the professional environment, **flexibility with development environments is essential**.
- I started to **structure the code better**, organizing it into **packages**, and to use classes and interfaces in a more elaborate way .
- I developed small applications with **basic user interfaces** and **terminal outputs**, which allowed me to check the operation in real time and work on debugging skills.

MARCH – JUNE 2024

Development of Monolithic Applications – Intermediate Level

As the complexity of the projects increased, so did my skills:

- Improved code organization and scalability.
- I integrated functionalities in logical and presentation layers.
- I started working with **interaction between multiple classes** and handling **more complex internal processes**, which prepared me to start using more robust frameworks.

This was a key moment, I began to really feel like a developer, with more confidence in my abilities and with the certainty that I could face greater technical challenges.

SEPTEMBER – DECEMBER 2024

Introduction to Spring Boot and Spring Tool Suite 4 IDE Eclipse

This quarter represented a great technical and conceptual leap in my training. I worked with:

- **Spring Boot**, a modern framework that facilitates the development of business applications in a fast and structured way.
- **Spring Tool Suite 4**, a development environment focused on Spring applications, ideal for configuring servers, database connections (**H2, MySQL, PostgreSQL**) and facilitating compilation and testing tasks.

I also learned how to:

- Manage dependencies with **Maven**, selecting libraries according to the needs of the project.
- Develop **projects with multiple dependencies**, understand how **to build, test, and debug** efficiently.
- **To start building my microservices portfolio with Java**, which would realistically reflect my technical skills and progress as a backend developer.

This second stage represented my transition from a self-taught learner to **a developer in solid and constant training**. I discovered the value of moving forward step by step, of respecting learning processes and of not trying to run before walking.

I understood that each phase has its function, and that mastering the essentials has allowed me to confidently reach technologies such as Spring Boot and microservices.

JANUARY-DECEMBER 2025

Practical consolidation: From simple projects to productive microservices

Throughout 2025 I focused my training on **project development with Spring Tool Suite (STS) and Eclipse**, ranging from simple applications to microservices connected to real databases. It was a key year to **consolidate technical skills, apply good practices** and gain autonomy as a developer.

Adaptation to the development environment

I started with small projects to familiarize myself with **STS Eclipse**, a more robust environment with many more functionalities than the previous IDEs I had used. To facilitate my adaptation:

I customized the interface with a **dark theme**, which I usually do with any software I use, as working long hours with white screens can lead to eye strain.

<u>I integrated essential tools for modern development, such as:</u>

- **Lombok**, to simplify code by reducing the need for manual getters/setters.
- **JUnit**, for unit tests.
- **Sonar Lint**, code quality analysis extension.
- **Mockito and Cucumber**, which I explored to expand my knowledge in testing (although I later prioritized JUnit in daily use).

Stage One: Basic Projects and Databases

<u>The first projects focused on:</u>

- Learn how to **select the necessary dependencies** using Maven.
- Understand how projects connect to different **relational databases**.

In this phase I mainly used **H2 Database**, an in-memory database ideal for test and development environments.

<u>This environment allowed me to:</u>

- Practice without setup complications.
- Simulate pre-production scenarios.
- Focus on the logic and structure of microservices before scaling to more real-world environments.

I also started using **dependency injection**, a key concept in modern applications. Although it was abstract at first, with practice it became a natural part of the development process.

Second stage: Microservices with real databases

After the test stage with H2, I went on to work with **MySQL** and **PostgreSQL**.

<u>In this phase:</u>

- I created **separate microservices** connected to each of these databases.
- I didn't integrate them with each other, as the goal was not to create a connected services architecture, but **to understand and practice the connection between a microservice and its database** in an isolated and focused way.

To validate the functionalities and test the APIs developed, I started working with **Postman**, a fundamental tool to ensure correct communication between client and server.

Readjustment of priorities: The transversal is also technical

An important learning from this stage was to realize that I had **underestimated the importance of certain transversal knowledge**, which I initially thought I would leave for the end. However, as I advanced in the development of real projects, I understood that I had to incorporate them much earlier.

<u>Cross-cutting tools that I integrated:</u>

Testing con Java

- I learned and practiced with *JUnit*, *Mockito* and *Cucumber*, doing small projects to understand how they work.
- On a day-to-day basis, I focused on the practical use of **JUnit** and the **Sonar Lint** extension in STS, ensuring that the code is bug-free during development.

Picnic

- I started using **Jira**, a common tool in collaborative work environments.
- I learned how to manage tasks visually by status: "pending," "in process," and "completed."
- Although its use was basic, I registered and set up a profile with its **personal URL**, so that I can display it if requested in interviews or work environments.
- Its learning curve is accessible, and I am confident that I will be able to expand its use as teamwork requires.

Consolidation: Projects with different dependencies

I continued to develop **microservices projects with different levels of complexity and variety of dependencies**.

<u>My focus was on:</u>

- **Identify which libraries and configurations are most in demand** in current job offers.
- Practice especially with those that are repeated in most ads for junior positions in Java development.
- Accept that it is not necessary (or realistic) to learn all possible dependencies, but to focus on **those that give more value in the labor market**.

This year was the point of consolidation of my profile as a developer. I went from understanding concepts to **applying them in contexts closer to real production**, developing a more professional and strategic approach.

- I learned to **adapt to new tools** without getting frustrated.
- I understood the importance of **transversal processes**, such as testing and task management.
- I began to create a **more complete portfolio focused on industry standards**.

My ultimate goal remains clear, to continue building a solid technical foundation, with a continuous learning mindset, to confidently join the job market as a Java developer specializing in microservices.

Difference between projects to learn and star projects

A very useful distinction for organizing your portfolio is to separate your projects into two clear categories.

1. Projects to learn and meet labor market requirements

You build these projects thinking about learning and mastering the technologies and skills that frequently appear in job offers. Many times, the lists of requirements are long and can seem unattainable, but if you notice that certain points are repeated in various offers, those are the ones that really matter to move forward.

What are they for?

- Familiarize yourself with key concepts.
- Practice design patterns and best practices.
- Master frameworks and libraries.
- Gain confidence in the use of technologies.

They don't need to be perfect or very complex, but they do need to be complete and functional. Think of them as necessary stepping stones to strengthen your base.

2. Star projects for your CV and public portfolio

These are the projects that you will proudly show off and that will capture the attention of the technical teams. They are robust, well-designed projects with clean code, clear documentation, automated testing, deployment in production, and a solid architecture.

These projects require time and dedication, but they reflect your professional level and your commitment to quality.

I am currently developing a project with **Spring Cloud**, composed of two microservices. One with **PostgreSQL** and the other with **MySQL**, using **Docker** and **Kubernetes**.

This project will allow me to apply the knowledge gained in other repositories of my GitHub profile, such as **Eureka**. Both microservices work correctly and communicate using **OpenFeign**. Each microservice has its own separate database: one with PostgreSQL and the other with MySQL.

I will most likely include this repository in my **CV,** since, so far, it represents one of the most complete projects I have developed.

Below, I will briefly describe the three-star projects that will stand out in my CV:

Examples of my star projects and why I chose them

Microservices Project with Eureka + Docker + Kubernetes

I already have it partially working.

- **Learning Docker and Kubernetes** in this context prepares me directly for technical interviews.
- I can document it and upload it as a star project on my CV and GitHub.

Clear goal: Have a microservices environment with Eureka logging, **Docker containers**, and basic orchestration in **Kubernetes with Minikube or kind**.

What I have to prepare here:

- Dockerize microservices.
- Create docker-compose.yml files initially.
- Configure deployment. yaml and service.yaml for Kubernetes.
- Document the deployment in the README.

Project with Kafka in English

Ideal as a technical reinforcement project and interview material in English.

Ideal for Java backend profiles. Businesses value many realistic projects with distributed microservices and messaging queues. It also includes **Docker and Kubernetes**, as **Kafka** runs in containers locally. This project in English is a way to practice and teach recruiters that I can also do projects in English indirectly.

Clear goal: Create a Java architecture with **Kafka**, include queues, message consumption and describe it in English with **technical README**.

- ➢ It's a good way to practice technical vocabulary in English + use it as a distributed systems mastery demo.
- ➢ There is a lot of demand in job offers in the requirements, so it is a strong point to have experience.

Ubuntu Project + Ollama + ChatGPT + UI + Docker

A local AI mini stack with interface to interact with models, ideal for prototypes and personal tools.

Docker to practice on Ubuntu, make repositories, commits with git.

Clear goal*:* Create a local AI mini-stack with an interface to interact with models, personal tool type for prototyping, captures and documentation.

Document the entire process: Installation, commands, resolved bugs, and creation of a basic UI. This trains me to write technical documentation and share valuable knowledge in Ubuntu.

The entire process from installation, to commands, screenshots, bug fixes, and a basic UI. That is already valuable and serves others.

REFLECTION-CONCLUSION:

You don't need to be an expert in everything from the start. The important thing is to show that you can learn, solve problems and explain what you do. The structure and documentation of the project reflects your professional level.

I currently have more than **24 repositories** on GitHub and they will continue to grow in parallel with my work.

Finalize the README.md of pending projects on GitHub, put the LinkedIn links. **And optimize my LinkedIn and GitHub profiles** by adding the new skills on LinkedIn and the same in the About Me.

Learn how to use Docker and Kubernetes for microservices deployment for the 3-star projects for the CV.

Send CVs along with the link to my GitHub profile and the corresponding links to my most outstanding star projects with a brief explanation and with the stack used to develop the Java microservices projects.

It is necessary to make the **cover letters highlighting my personal merits,** I will surely make a general draft telling the most important and I will adapt it to each job offer.

Optimize my LinkedIn profile by putting the star projects in the section provided by LinkedIn to put the link, screenshots, description and more details.

The **About Me** section is my presentation as a professional with my strengths, merits, skills, I have a brief description of who I am and what I do in a coherent and structured way. Warm and professional.

In this ebook is my **last update of the profile** so that you could get to know me. It was no coincidence, I have put it this way to serve as a reference. And of inspiration regardless of the specialty. I have been able to write extensively in this ebook because I do not have a character limit like on LinkedIn

Develop more projects aimed at scalable and resilient architecture in real environments.

Phases of learning from beginner to professional:

Short and medium-term goals to structure your growth.

The phases you will have to go through:

To give you a clear vision, here is a summary of the phases we all must go through. Keep in mind that some things may vary depending on the job market, especially in Latin America, but this is my experience as a Spaniard. Then, each person adapts their strategy to their circumstances.

In this ebook we are going to focus from Phase 1 to 5 so as not to get lost on the short- and medium-term objectives.

Defining your career goal

Phase 1: Logic and programming fundamentals.

Objective: To understand how programs work and how to solve problems logically, without memorizing.

Key topics: Algorithms, data types, control structures (if, for, while), functions, collections, pseudocode, flowcharts.

Tools: JavaScript, Java, C#, C++,

Editors: VS Code, IntelliJ, NetBeans, Eclipse or STS Eclipse for Spring Framework.

Phase 2: Specialty Choice + Market Research

Objective: Define your professional route and find out what job offers are asking for.

Steps: Choose specialty (web, mobile, AI, video games, etc.), follow RoadMap, analyze offers on LinkedIn, InfoJobs, Tecnoempleo.

Phase 3: Small and medium-sized projects

Objective: To put theory into practice and gain experience.

Examples: Calculator, task manager, basic games, CRUD.

Focus: Clean, organized code, with comments and basic use of Git.

Phase 4: Advanced Projects + Portfolio

Objective: Build a professional portfolio with projects that reflect your real skills.

It should include: Structured code, professional README, screenshots or demos, use of real tools.

Phase 5: Profile Optimization

Objective: Create an online presence that attracts opportunities.

Checklist: Tidy GitHub, updated LinkedIn, possible personal page.

Phase 6: Sending CVs and cover letters

Objective: To adapt each application to stand out.

Tips: Customize CV, include links to projects, short and specific letter.

Phase 7: Phone interview

Objective: Show motivation and align expectations.

Tips: Safety, explain your learning and projects, genuine interest.

Phase 8: Face-to-face technical interview

Objective: Demonstrate your skills in technical challenges.

Includes: Algorithms, explain decisions, mini-applications.

Phase 9: Offer or pre-contract

Objective: To evaluate conditions before signing.

Check out: Contract, working day, salary, modality, probationary period.

Phase 10: Trial period

Objective: Adapt and demonstrate your contribution.

Tips: Learn fast, be collaborative, ask for feedback.

Phase 11: Indefinite contract

Objective: To achieve stability and continue to grow.

Next steps: Continuous training, open source, mentoring, constant updating.

Remember, this process is a constant learning, and no one expects you to master everything at once. The fundamental thing is not to skip key phases and learn to respect your own rhythm, with patience and perseverance.

Defining your career goal

What kind of programmer or developer do you want to be?

Programmer vs Developer: A Difference That Can Define Your Career

Although they are often used synonymously, "programmer" and "developer" do not mean the same thing. **The difference may seem subtle**, but it has a profound impact on the professional role, the responsibilities assumed and the way we face technical work.

In practice, many companies and teams use both terms interchangeably, especially in job offers. This lack of precision leads to confusion: what is really expected of you when you apply for a position? Just write code or also design solutions, make decisions and think about the product?

Still, understanding the difference can mark a **turning point in your professional growth**. Not only does it allow you to chart a clearer path, it also prepares you to take on more strategic responsibilities over time.

Choosing your path in technology: Between doubts, mistakes and discoveries

With the above guidelines you already know how to differentiate between programmer and developer, and you also know the different profiles and specializations that exist. But let's be honest.

Deep down, you already know what catches your attention the most. There's something you visualize when you imagine yourself working on what you like. Something that excites you.

And if you're still not sure, don't worry. It's completely normal to feel lost at first. It happened to me too.

Not everything was linear. I was wrong, I changed course, I turned back and took new paths.

But every mistake was learning.

No time was really wasted.

On the contrary, each misstep gave me more clarity.

Figuring out what you like takes time

As soon as you start programming for real, with real or practical projects, you will notice:

- Which parts you enjoy the most.
- Which ones bore or frustrate you.
- What kind of problems you like to solve.

And if the path you took doesn't convince you, that's okay. You can redirect it. You don't have to know everything from day one. This industry is so big that there is always room to reinvent itself.

If you are programming today, ask yourself: do you want to just execute, or do you want to design, decide, build?

If you already consider yourself a developer, remember that this implies a broad vision, the ability to collaborate, and critical thinking.

Don't compare yourself with those who say they got a job in record time, each story is different, and yours also has value.

Keep training, building, learning. This is a race for those who don't give up.

What is the difference between programmer and developer?

Programmer

A programmer is someone who writes code. Its focus is on the technical implementation of instructions following specifications.

Key features:

- It concentrates on the source code.
- Receive specific tasks, for example: "program this module or function".
- It is not necessarily involved in the overall design of the system or strategic technical decisions.
- Their role can be more executive than creative.

Example: If a team decides to build an application, the programmer can be exclusively responsible for coding a specific functionality (such as the login screen), following an already established guide.

Developer

A developer goes beyond writing code. Participate in the entire software development lifecycle:

- Analysis of requirements.
- Design.
- Programming.
- Tests.
- Maintenance and continuous improvement.

Key features:

- Understand the full context of the project.
- Make decisions about architecture, user experience, scalability, etc.
- He usually proposes solutions, not just implements them.
- They can work more autonomously and have a more strategic and product vision.

Example: A developer might design how an entire functionality will work, choose the right technologies, coordinate with other roles (design, QA, client), and then program or delegate it.

In practice, many companies use both terms synonymously, and not all of them make a clear distinction. However, understanding the difference can help you grow professionally:

If you are a programmer, you can aspire to be a developer by expanding your knowledge in design, architecture and product.

If you present yourself as a developer, you are expected to have a more global vision and be able to make technical decisions beyond the code.

Defining your specialization doesn't mean you limit yourself forever, but it does help you have a clear starting direction. Here I detail the main types of programmers with their characteristics, key technologies, and ideal profiles:

The importance of autonomy in programming. Why self-discipline and autonomy are essential in the learning process.

How to stand out in the job market as a programmer or developer?

Recommendations to stand out in a competitive market.

Being a good programmer or developer goes far beyond writing code. It implies having a **comprehensive vision of the technical role**, being prepared to solve real problems, adapting to constant change and knowing how to communicate your ideas. In such a competitive environment, those who manage to excel are those who combine **deep technical knowledge with soft skills and a proactive attitude of continuous learning**.

Master Fundamental Technical Skills

Technical knowledge is the foundation of your career. Without it, you can't build, but with it alone, you won't be able to stand out.

Recommendations:

Programming languages: Focus on mastering key languages according to your specialty:

- **BackEnd:** Java, Python, C#, Go, Node.js
- **FrontEnd:** JavaScript, TypeScript, HTML5, CSS3
- **FullStack:** Combination of both and integrative frameworks

Frameworks and tools:

- Spring Boot, Django, Express (BackEnd)
- React, Angular, Vue.js (FrontEnd)
- Docker, Kubernetes, Jenkins, GitHub Actions (DevOps/Deployment)

Databases:

- Relational: MySQL, PostgreSQL
- NoSQL: MongoDB, Redis, Firebase

Version Control:

- Learn Git in depth: Branches, merges, pull requests, rebases
- Use collaborative platforms like GitHub, GitLab, or Bitbucket

The key is not only to know how to use these technologies, but also to understand when and why to apply them.

Define a Clear Career Path

Having a clear purpose and goals helps you make strategic decisions in your learning and career.

Examples of routers:

- **FrontEnd Developer** → UX/UI + React/Vue + Testing
- **Backend Developer** → Spring Boot + Docker + Clean Architecture
- **Full Stack** → MERN Stack + DevOps + API Design
- **Mobile Developer** → Flutter or React Native + Firebase
- **Cloud & DevOps Engineer** → Terraform + CI/CD + Kubernetes

Set quarterly goals and evaluate your progress constantly.

Become a well-rounded professional

Standing out in the job market as a programmer is not achieved only with technical talent. It involves having vision, adaptability, and interpersonal skills.

The developer that becomes indispensable is one that combines:

- Solid and up-to-date technical knowledge
- Critical thinking and solution focus
- Communication and collaboration skills
- Constant learning capacity
- Professional presence on platforms such as LinkedIn and GitHub

The key is to **build value, demonstrate it and communicate it strategically**.

If you invest in yourself, in your knowledge and in your way of presenting yourself to the world, you will not only find work.

You'll find opportunities aligned with your growth, passion, and purpose.

Develops Critical Thinking and Problem Solving

Employers are looking for developers who think like **solution architects**, not just task executors.

How to achieve it:

- Practice platform problem-solving.
- Learn about design patterns and scalable architectures.
- Improve your ability to read, understand, and optimize other people's code.
- He studies agile methodologies, especially Scrum and Kanban. ***It's not just "what you do," but "how you do it" and "why you make every decision."***

Build a Technical Personal Brand

Differentiating yourself is key. To do this, you must **show what you know how to do and communicate it strategically**.

Actions:

- Keep your **LinkedIn profile** up to date, with measurable accomplishments and technical posts.
- Use **GitHub** to showcase your most relevant projects and the way you work.
- Consider having your **personal portfolio online** (simple website with your CV, links, featured projects).
- Write technical articles or tutorials about your learning, in addition to reinforcing your knowledge, it positions you as a reference.

Improve Your Communication Skills

Knowing how to communicate is as important as knowing how to program. You'll be working with teams, clients, testers, and technical leaders.

Focus on:

- Explain technical solutions in a clear and structured way.
- Actively listen and ask intelligent questions.
- Participate in meetings, retrospectives, and project presentations.
- Use collaboration tools (Slack, Trello, Notion, Jira, etc.) fluently.

*A programmer who knows how to communicate well multiplies his value in any team***.**

Encourage Collaboration and Teamwork

Software development is a collective effort. Those who know how to contribute, receive feedback and adapt stand out.

Councils:

- She works on collaborative projects (open source, freelancing, bootcamps).
- Give and receive code reviews with respect and objectivity.
- Learn how to document your processes and decisions well.
- Be proactive: Offer help, seek improvements, share knowledge.

The learning curve in tech is not linear: It changes and that's completely normal

When you start in technology, at first everything moves very quickly.

It seems that everything is going well:

You learn new concepts, do exercises, follow tutorials... and you see concrete results. But there comes a point where everything becomes more complex, and many people get frustrated. The important thing is to understand that this is normal and part of the real learning process.

Phase 1: The start is fast and motivating

You start with introductory courses, simple exercises, basic examples. You begin to solve problems and understand the fundamentals. You feel confident, you even think you're close to being ready to work. Progress is constant and there are no major blockages.

Why?

Because you work in controlled contexts, with low technical complexity. The learning curve is fast, and that motivates you a lot.

Phase 2: Real projects, more level, more challenges

You want to apply what you learned in your own projects.

That's when you realize that you need much more: Tools, logic, structure and good practices. You find that job postings require skills that weren't in the core courses

New challenges appear:

- Testing.
- Documentation.
- Architecture.
- Code organization.
- Integration with other systems.

This stage takes more time and demands more mental effort. You're no longer solving exercises, you're solving real problems. But this is not a step backwards. It's evolution. This is where you really learn

You begin to develop deep skills:

- Autonomy.
- Frustration tolerance.
- Critical thinking.
- Ability to integrate multiple knowledge into a single project.

This level is more challenging, yes, but also more formative.

REFLECTION-CONCLUSION

At first everything seems easy and fast, but it's because you're in the basics. Then everything gets slower and more complex and that's where you're really growing.

If you're at this stage:

It's normal for it to take longer. It's normal to get frustrated with mistakes you don't understand. It's part of the path to a real professional level.

Your growth does not depend only on a degree, but on what you are able to build with what you learn.

My personal story: From programming out of curiosity to developing with purpose

When I started programming with JavaScript, I knew that I didn't just want to chop code, I wanted to develop. I was attracted to the challenge of building complete systems and especially, I was fascinated by everything that happened on the backend side, that invisible part that holds an application from within.

It didn't take me long to realize that software development goes far beyond web design or pretty interfaces. **I wanted to create robust applications, well-thought-out business logics, scalable structures.**

That's when I discovered Java, a language that, while many said was difficult, had everything that solidity, versatility, and presence in large enterprise systems was looking for. **Specifically, I wanted to specialize in Java microservices Software Developer**

And yes, it's true, learning programming is not something that is achieved in 6 months with a miraculous course. This is a long-distance race.

It requires perseverance, resilience and many hours of real work. Whoever says otherwise, probably selling you a course with aggressive marketing is not being honest.

Each person lives their own battle

We all fight our own battles. Frustrations, rejections, doubts, pauses... but, even so, we keep going. We fight every day to get closer to a goal that goes beyond a simple job, to work in something that we are passionate about. And for that alone, we are already doing something brave. That silent, constant and often invisible effort is an act of courage.

The reality of the tech sector: Lights and shadows

The technology sector is often associated with **attractive salaries, international opportunities, and constant professional growth**. And yes, that's all true. But it is also true that, for those who are training or reconverting professionally, **access can seem like a steep mountain**:

- **Job offers that ask for** "5 years of experience with technologies that have only two".
- Endless lists of requirements: Frameworks, tools, methodologies, soft skills, languages, testing, agile methodologies...
- **Technical and demanding selection processes**, where not only knowledge is evaluated, but also the ability to solve problems under pressure.

This can create a sense of never being prepared enough, and it's something that many people struggle with in silence.

Preparing as a specialist... while remaining an apprentice

Those of us who aspire to work in technology don't just need to learn to code. We need to develop a **specialised, resilient, constantly evolving professional profile** that combines technical skills with soft skills such as communication, autonomy and adaptability.

The labor market is constantly changing. Technologies are advancing fast. And so do the needs of companies. Therefore, beyond "finishing studying", the important thing is to adopt an **attitude of continuous learning**.

It is not about knowing everything, but about being willing to learn what it takes.

Not complying with everything ≠ not being prepared

Many times, we stop at a job offer, a course or an opportunity and think twice when we see the list of requirements: "Oh, but if I don't meet this, that, that, that... is it good for me?" The truth is that almost no one complies 100%. And that's okay.

Companies put the ideal, their "dream version" of the perfect candidate. But what they really value is not that you are perfect, but that you show that you are willing to grow, that you have initiative, that you learn quickly. That weighs more than many ticked boxes.

What does it mean to be truly prepared?

Initiative: Seeing something that is missing, something that you can contribute, and taking the step to do it, without waiting to be told. There you show that you are not only "complying", you are contributing.

Commitment: To keep going, even when things are missing, even if everything is not clear, even if you feel insecure. Don't give up.

Adaptability: Accept that there will be new, unexpected things that you will have to learn as you go. Be flexible between the known and the new.

Why don't you need to meet all the requirements?

1. Because the requirements are usually "the ideal" not "the necessary". Many companies would incorporate someone who fulfills a large part, but who is eager to learn the rest.
2. Because what really matters many times is **how you deal with what you don't know**. If they see you with a curious attitude, looking to learn and improve, that transmits much more value than just "being ready".
3. Because every step count, no matter how small it may seem. Every new skill, every project you make, even if it's modest, every mistake you learn from... they all add up.

How to continue growing?

Take an honest inventory of what you know and what you lack. And pick one or two gaps to work on now. Don't try to fix everything at once.

Surround yourself with people who know more, who challenge you, who inspire you. Ask for help, mentors, peers.

Learn from feedback: Seek constructive criticism, from your professors, from colleagues, from customers. Let them know what they can improve in your work.

Be consistent, though not perfect. If you do something every day, read a fragment, polish an idea, practice something new, after a while you will see the big change.

Celebrate what you've already accomplished: When you look back you realize how far you've come. That gives encouragement to continue.

REFLECTION-CONCLUSION

Don't compare yourself to those who seem to have it all. Don't run a race that isn't yours.

That constant comparison with others, what they publish, what they demonstrate, is like always looking outwards, losing sight of your own path. Each person has their rhythm, their stumbling blocks, their silences. What you see in an image (a post, an achievement, a profile) is just the cropping that person decided to show. Behind it may be doubts, mistakes, disorders that are not seen.

Your career is not theirs. It's yours. With your times, with your tools, with your unique learning. If you run a race that isn't yours, you'll always be lagging behind, feeling like you're not enough.

In the meantime, you can jog, move at your own pace, calmly, but steadily, looking inward: what do I need to learn today? What do I need? What am I good at already?

What matters is that you are moving, walking towards what you want, learning day by day.

It doesn't matter if the march is slow, if some days you take two steps forward and one step back. The essential thing is to move. Don't let your life get stuck in a place where you know you want to get out. That daily movement, even if it seems minimal: reading a chapter, translating an idea into action, polishing a piece of writing, talking to someone, practicing a high skill. Because every second invested in growth, even if you don't see immediate results, becomes a firm foundation for what is to come.

And remember: What seems little to you today, tomorrow you will look at with pride, because it was necessary to get here.

Maybe today you think "but I have made so little progress..." But in months, when you look back at where you were, you'll see that that "little" was a decisive step. That every correction, every failed essay, every obligatory reading was worth it.

That "little" is the constant heartbeat of your progress, often invisible, but real. And when you achieve bigger achievements, those "few" will be anecdotes of how you started. They will be the testimony that you did not stop.

What matters is not that tomorrow you are perfect, but that today you have come a little closer than yesterday.

Perfection is a trap: you wait for everything to be "ready" before showing, before you launch, before you move forward. But real change happens when you commit to continuous progress, to the "a little more than yesterday." It doesn't matter if the progress is small, slow, imperfect. What matters is that there is progress. That you look back and see a line that did not stop, that continued to grow.

With humility, discipline and curiosity, your effort will be what makes the difference.

Define your path and be honest with yourself

Don't compare yourself with those who say they got a job in record time. Every story is different, every process is unique, and **your path has value too**. The most important thing is that you continue to train, build, learn. This is a long-distance race, not a competition.

It is a profession for people who do not give up, who know that each step adds up, even if it is not immediately noticeable.

My Personal Situation: What I Have, What I Don't and What I Decided to Do

I do not have a Vocational Training or a university degree in computer science. I started with some official certification courses, but most of my learning has been self-taught or through specialized courses and master's degrees **without formal qualifications**.

I would have liked to have been able to pursue a higher degree in multiplatform application development, but my personal circumstances did not allow me to do so. Even so, I learned that, in this sector, in addition to the degree, it is highly valued that you are decisive, self-taught, curious and that you have real experience with projects.

Because, let's be honest. You can have many degrees and not have developed any practical project on your own. And that leaves you without an essential part of what companies are looking for, knowing how to apply what you know.

Changing focus and developing autonomy

I had to adapt. Learn to solve mistakes on my own. There were many moments of frustration, especially when I ran into bugs that I didn't know how to solve.

It was then that I understood the importance of testing, **JUnit and SonarLite** helped me to detect errors and correct them from the beginning, integrating good practices from the early stages of development.

<u>I remember one of my mentors telling me something that left a mark on me:</u>

"At work, the important thing is to solve mistakes. The warnings may be there, but they don't block the project. There will be time to solve them in pre-production."

That taught me to **prioritize, manage frustration, and focus on what matters**. I learned that sometimes it takes longer, sometimes less, but if you insist, in the end you succeed.

My strategy to stand out without an official degree

It was clear to me that, in order to excel, I needed to **design my own path**. I wanted to compete with people who did have degrees, so I built my **personal professional tech brand**, focused on three pillars:

1. LinkedIn

Strategically updated. I publish valuable content, reflections, advances, mistakes that are transformed into learning. I participate in relevant debates, I contribute from humility and real experience. I review and adapt the *"About Me"* section as I learn as I progress.

2. GitHub

My portfolio contains functional, well-documented projects with explanatory README.md, screenshots and relevant links. My star projects are carefully selected, they are not generic, but representative of my skills and current technical level.

3. Mentoring and own content

I lead and actively participate as a mentor in two Telegram communities. I bring experience advising others, designing personalized training routes, improving profiles and guiding those who are in the initial phases.

In addition, **I write and publish eBook's** for beginner programmers, in Spanish and English, and I am developing increasingly technical and specialized materials. This not only allows me to teach, but also to reinforce my learning and strengthen my professional profile.

Everyone has their own means: Find yours

I have always liked to write. That's why eBooks are my means of expression. They allow me to connect with people who feel the same way I felt when I started. They make me happy. They make me feel useful. But that's *my* channel. Maybe yours is different:

- ✓ Do you like to talk? Maybe your place is on **YouTube** or **Twitch**.
- ✓ Do you love to write? Maybe you're into a **technical blog** or writing eBook's.
- ✓ Are you good visually? You could share your progress on **Instagram** or **X**.

The important thing is that you choose the format in which you feel most comfortable to communicate your differential value as a tech professional. And that content is backed by your actual work, your projects on GitHub, and an attitude of continuous learning.

About the projects you share

Not all projects are worth putting on your CV.

There is a clear difference between:

- **Small or medium-sized projects**: They serve to consolidate knowledge, they are generic and repetitive, ideal for practice.
- **Large or complex projects**: They show a more advanced technical level, with integration of multiple tools, complex logic and solid structure. They are the ones that should go in your professional portfolio.

Choosing your star projects generates doubts, it's normal. But don't worry, over time, you'll know which ones best represent your evolution as a developer.

My confidence comes from practice

I can say with conviction that I am confident in my knowledge and experience doing microservices with **Spring Tool Suite (STS).** Each project has been a challenge, each bug a lesson, each advance one more step towards my professional goal.

I don't pretend to know everything, but I know what I've achieved, and I prepare myself day by day to face technical interviews with confidence.

I started by laying the solid foundations:

Study Java understand object-oriented programming, handle collections, exceptions, threads. **I used STS to build small projects**: REST controllers, database access with Spring Data, **CRUD** operations. That stage gave me the security of knowing how endpoints work, how to map entities, how to structure a project with clear responsibility, and use essential tools like Maven and Git.

Then I took the step to standalone microservices:

Separate responsibilities, design API contracts, manage communication between services (REST, events), anticipate failures and define patterns such as retry, circuit breaker or fallback. In this phase I understood the value of decoupling, clean design, fault tolerance, and facing scenarios in which a service does not respond.

Finally, I understood that code must live beyond the local environment:

Getting into deployments, **packaging with Docker**, configuring different environments (local, test, production), managing external configuration profiles. In each deployment I learned lessons:

Settings, failures, error messages, logs, unit and integrated tests, versions. I could see that a project is not only successful if it runs, but if it can be maintained, scaled, and survived in the face of failures.

Along with all that, I have worked on specific preparation for interviews. I've analyzed typical questions: How to design a microservice "on the fly", how to handle performance or failures, how to justify architecture decisions and trade-offs between simplicity and robustness.

I have done mock interview exercises, coding challenges, I read experiences of other candidates. Practicing not only coding, but explaining what you do, the "why" behind every decision, is a vital part of training.

Today I have managed to build functional microservices that communicate with each other, deal with real bugs (dependencies, configuration, integration, JSON errors, latency), use Docker as an environment, write tests to ensure basic functionality and develop code with quality: That can be read, maintained, extended. And that gives me confidence.

But I also know that this is a path that follows. That's why I now focus on advanced architecture (messaging, events, resiliency with tools like Resilience4J), performance, scalability, deeper testing (integration, contracts, load testing), security (authentication, authorization, encryption), and basic DevOps (CI/CD, automation, cloud deployment).

This path that I have traveled and that I follow, you can adapt it to you. Assess where you are now, identify your strengths and weaknesses, define concrete goals (short and medium term), build small projects that combine different challenges, document your decisions, explain your process, and practice mock interviews.

This way you will build your own journey, with purpose and firmness.

Phase 1: Logic and programming fundamentals.

Why is it important to learn logic and programming fundamentals before choosing a major?

Learning logic and programming fundamentals before specializing is an essential step for anyone aspiring to build a solid career in software development.

This approach not only establishes a robust technical foundation, but also facilitates the acquisition of new knowledge, improves decision-making, and prevents bad habits that may be difficult to correct later.

Below are the key reasons why mastering the fundamentals is an investment that makes a big difference:

1. Solid foundation for any language or specialty

Universal foundations: Concepts such as variables, conditionals, loops, functions, and data structures are common to all programming languages. If you understand how they work, you can adapt to technologies such as Java, Python, JavaScript, C#, among others, simply by learning their syntax.

Language independence**:** With a deep understanding of the fundamental principles, changing or learning new languages becomes easier and more natural. This versatility allows you to stay relevant in a changing technology environment.

2. Development of logical thinking

Algorithmic thinking: Learning logic trains you to break down complex problems into simple, orderly steps. This allows you to design clearer, more efficient, and scalable solutions.

Better decision-making: A good command of logic helps to evaluate technical alternatives objectively, choosing solutions based on technical criteria and not on assumptions.

3. Smooth transition to any specialty

Versatility: A solid foundation allows you to explore areas such as web development, mobile apps, artificial intelligence, data science, or process automation without feeling lost.

Adaptability: By understanding the principles behind tools and frameworks, you can learn them more quickly and use them more effectively.

4. Deep understanding of advanced concepts

Assimilation of complex topics: Concepts such as data structures, object-oriented programming, design patterns, concurrency, or software architectures become more accessible when you already have internalized the fundamentals.

Optimization and debugging: Mastering the basics allows you to write cleaner code, identify bugs quickly, and improve the performance of your programs from a conscious approach.

5. Global Understanding of Software Development

Comprehensive view of the process: Knowing how a program works from the inside helps you understand each stage of the software life cycle: from design to maintenance and evolution.

Greater autonomy: This vision allows you to make informed technical decisions, without relying exclusively on tools, wizards, or templates.

6. Advantage in technical interviews

Algorithmic problem-solving: Many job interviews assess your logical ability through problems that require structured thinking, not just knowledge of tools.

Demonstrate real understanding: Companies value candidates who understand the "why" of what they do, not just the "how."

7. Prevention of bad habits

Sustainable code: Without foundations, it is easy to fall into bad practices: copying without understanding, structuring the code badly, not validating inputs, among others. A solid foundation prevents these mistakes from the start.

Critical mindset: Understanding the essentials allows you to evaluate technologies and tools judiciously, without blindly relying on external solutions.

8. Long-term motivation and persistence

Frustration reduction: The more you understand what you're doing, the easier it is to overcome technical blocks. This keeps you motivated and with a positive attitude towards learning.

Natural progression: Each new concept connects with the previous ones, forming a mental map that gives you confidence and clarity.

REFLECTION-CONCLUSION

Mastering the logic and fundamentals of programming isn't just an early stage: it' **s the cornerstone of your professional development**.

It prepares you for:

- Learn new languages and technologies more quickly.
- Successfully adapt to a constantly changing market.
- Solve problems with technical criteria and efficiency.
- Understand, optimize, and maintain your code securely.
- Move toward specializations with greater clarity and fewer errors.

Investing time in strengthening this foundation will not only make you more competent, but also more independent, adaptable, and resilient. It is the best starting point to build a solid, sustainable and meaningful technical career.

The importance of not skipping steps in learning programming

Learning to code is a progressive process that requires patience, discipline, and a structured strategy. Skipping stages, for example, avoiding basic exercises to move directly to complex projects, can have negative consequences in both the short and long term.

Programming mastery isn't built overnight. Respecting a progressive learning sequence, from the fundamentals to the real projects, is key to:

- Consolidate knowledge clearly.
- Avoid unnecessary blockages and frustrations.
- Develop a structured mindset for solving problems.
- Build a coherent and professional portfolio.

Skipping steps can lead to confusion, gridlock, and messy code. Instead, moving forward with patience and structure ensures deep learning, greater autonomy, and a more fulfilling career in the long run.

Here's why it's crucial to respect a logical and gradual progression in the learning process:

1. Consolidation of the fundamentals

Understand before applying: The bases, variables, conditionals, functions, loops, are the foundations of any development. Trying to work without understanding them leads to unnecessary frustrations.

Reinforce the technical base: The constant practice of basic exercises allows you to assimilate concepts until they become natural, facilitating problem solving later on.

2. Avoid frustration and cognitive overload

Large projects can be overwhelming: Tackling complex tasks without technical preparation can result in constant mistakes, wasted time, and demotivation.

Small achievements, big motivation: Solving simple challenges boosts confidence and builds a positive mindset of steady progress.

3. Progressive development of problem-solving skills

Structured reasoning: Solving small problems strengthens your logic, ability to refine and validate ideas.

Scale with criteria: Every basic problem solved is a step towards building more complex and efficient solutions.

4. Better understanding of tools and technologies

Controlled learning**:** Familiarizing yourself with tools such as text editors, version control or terminals in a simple environment avoids technical saturation.

Avoid "paralysis by analysis": Introducing many tools at once without clear foundations can lead to confusion and frustration.

5. Learn from mistakes and continuously improve

Iterate with low risk: Making mistakes in small exercises is less expensive and allows for safe learning.

Continuous improvement: This approach trains you to iterate, debug, and improve, key skills to adapt to real, collaborative environments.

6. Preparation for real projects

Practical application of theory: Small projects allow you to connect abstract concepts with real scenarios.

Gradual evolution: Going from simple to complex improves your efficiency, technical safety, and ability to adapt to new technologies.

7. Building a Strong Professional Portfolio

Demonstrate real knowledge: Well-thought-out, yet simple, projects reflect mastery of best practices and attention to detail.

Professional habits from the beginning: Organizing, documenting and versioning correctly from the first projects are skills that will make you stand out from other candidates.

8. Increased confidence and autonomy

Evidence of progress: Seeing your progress in practical exercises builds real technical confidence.

Constant motivation: Each step completed strengthens your determination to continue learning and overcoming new challenges.

Why does this happen?

There are two common reasons:

Lack of knowledge: Many people do not know what programming logic is and therefore do not understand its importance.

Impatience: They want to go straight to programming in the language they've chosen, without first building a solid foundation.

REFLECTION-CONCLUSION

Every blockage at the beginning has a cause

When you start programming, reading other people's code or doing exercises, it is very common for blockages, frustrations, insecurities to appear. You feel like you don't understand what you see, that you don't know where it's going, that other people seem to be moving faster.

Part of that discomfort comes from trying to build something complex without having the foundations well established. Going back to the origin means taking up the fundamentals, signing it, making sure you really understand it, before continuing to build.

Essentials to get back to basics

Here are the steps that make the difference:

1. **Basic and intensive course in programming logic:** A well-structured course can give you the foundation you need: control structures (if/else), loops (for, while), functions/methods, basic operations. About 60 hours of good material and practice can solidify the idea of what you are doing when you "program".
2. **Read and consult technical documentation on fundamental concepts:** Not only tutorials that show "how to do something", but official documentation, books, articles that explain what a loop means, what a function does, how errors are handled, what recursion is, etc.
3. **Understand concepts, don't memorize them:** By memorizing you can copy, but if you don't understand, when faced with a new problem you won't know how to adapt. Understanding implies knowing why certain structures are used, what consequences they have, how they are applied in different scenarios.
4. **Learning to reason before programming This includes skills such as:**

    - Break down big problems into small pieces.
    - Propose the solution in steps (pseudocode, diagrams, diagrams).
    - Foresee extreme cases, mistakes, what happens if something goes wrong.
    - Think about what data you need, how you transform it, how you present it.

Why doesn't language matter so much at first?

All languages share a general structure: Variables, data types, expressions, conditionals, loops, functions, perhaps error handling. What is really important is not the syntax of "how X is written in that language", but to know what is happening "behind the code", what is evaluated first, how execution flows, where errors can arise, how they are handled.

Choosing a language that you like or that is accessible to you is fine, but what you build with it, what you think using universal conceptual tools, that is what will accompany you when you change languages, start microservices, learn frameworks, etc.

Practical tips to put it into action

- Do simple logic exercises: Pseudocode, flowcharts, solve logic puzzles.
- Use tools like **PSeInt** to write pseudocode and see the logical result without worrying about actual syntax yet. (It serves to understand programming logic without distraction).
- Read official technical documentation: Language manuals, style guides, articles on clean code structure.
- Dedicate sessions only to "thinking about the problem" before writing code. For example: reading the statement of the exercise, writing down the steps, drawing how the data flows, what functions will do what, foreseeing what may fail.
- When reviewing other people's code, not only read what it does, but *why* it did it that way, what alternatives there were, whether that solution is good or not, what you could change.

My personal experience

In my case, I learned programming logic with **JavaScript**, because it was the first language I studied during my FrontEnd training. When I later decided to major in **Java**, the transition was a bit difficult at first.

But over time I got used to it, **because the logic was the same**, and I already had a solid foundation that helped me adapt faster.

If you're starting from scratch, **invest time in this stage**. It's going to save you a lot of crashes later on. You don't need a degree or a master's degree to understand programming logic, just commitment and constant practice. Take courses, practice exercises, and keep basic documentation handy that you can refer to when you need it.

This is the most important phase to prepare the way to the next stage, specialization and insertion into the labor market.

Phase 2: Specialty Choice + Market Research

Once you've acquired a solid foundation in logic and programming fundamentals, it's time to make a crucial decision, **choose your specialty, and gain an in-depth understanding of the job market you want to access**.

Based on my experience guiding people in my group and individual mentorships, I have detected several common mistakes that can slow down professional growth if not corrected in time.

Frequent profiles and their problems

The academic profile without practical experience

This is a more common case than it seems, people with a solid university education, very good at mathematical logic and with great analytical skills. These skills give you a **real advantage** when it comes to understanding the structure of algorithms and solving complex problems.

However, some of these profiles tend to assume that the **academic degree is enough to directly access a specialized position** without having developed a specific specialty or having built a portfolio that evidences practical experience.

In many cases, they have worked on group projects during their training, but **they do not have individual projects**, and, therefore, they cannot demonstrate autonomy or real technical mastery in the area in which they want to insert.

Reflection and accompaniment

When I work with these people, my focus is never to discredit them, but **to help them become aware** of how the current job market works in technology:

- **The academic degree is an added value, not a substitute for practical experience.**
- It is essential **to specialize** (e.g., in video game development, mobile applications, BackEnd, FrontEnd, cybersecurity, etc.) to stand out in a competitive market.
- It is also key **to research the real job offers** in interest and analyze what the most repeated requirements are: Technologies, frameworks, tools, experience, type of projects.
- And of course, it is essential **to build a professional portfolio** that reflects that specialization through real projects, well documented and published on platforms such as GitHub or GitLab.

At first, this reality can generate rejection or discomfort, especially when there are high expectations based on academic training. But when an **honest, respectful and well-argued vision is offered**, the majority understands and accepts it, even if it is from the initial resignation. The important thing is that they then begin to **reconstruct their strategy for access to employment with greater clarity**.

Before choosing your specialty, take the time to answer these questions:

- What type of development do I like best? BackEnd, FrontEnd, mobile, video games, AI, testing, DevOps...?
- What technologies do I feel most comfortable in?
- What kind of companies or sectors would I like to explore?
- What skills or tools are real job offers in that area asking for?
- Do I have at least 2 or 3 functional projects that clearly reflect that specialty?

Choosing without investigating is a risk. Researching without choosing is a waste of time. Both processes must go hand in hand to design a realistic, strategic action plan aligned with the labor market.

Impatient person

Another profile that I have frequently encountered in mentoring is that of the **impatient** person, who approaches learning programming with a **sprint mentality**, when in reality this is a fundamental process, which requires time, perseverance and focus.

This type of person is often caught up in the **aggressive and unrealistic marketing** of certain formations that promise extraordinary results in a very short time. *"Learn to code in 6 months and get a job right away."* This message generates unrealistic expectations that, sooner or later, translate into frustration and emotional exhaustion.

Many times, they buy several accelerated courses, complete them quickly without paying real attention or doing meaningful practices, and **jump from one to another without consolidating knowledge**. This generates a **spiral of superficial learning**, without direction or clear structure, which hinders real progress.

Reflection and accompaniment

With this profile, the challenge is **to slow down the pace and redirect the focus**, something that is not always easy, since this behavior is usually driven by the anxiety of finding a job as soon as possible, the pressure of the environment or comparison with others.

My job as a mentor is **to help bring order**, structure the process and accompany these people so that they can **regain control** of their learning. I usually begin by explaining clearly that:

- It is not about studying fast, but about **learning well**.
- You can't build a strong career without a **firm and well-understood technical foundation**.
- Not everything that is available on the Internet is useful, there are introductory, generalist, poorly structured or outdated courses. Not everything works for everyone.
- It is essential **to stop and reflect**: What do I want to learn? Why? With what professional objective? How am I going to measure my progress?

Without a defined and realistic learning path, dispersion is guaranteed. And the time lost in that process cannot be recovered.

Recommendation

Instead of accumulating courses, focus on building a **customizable but structured learning path** that includes:

1. A solid foundation in logic and programming fundamentals, if you don't already have one.
2. Clear choice of a specialty.
3. Development of real projects that apply what has been learned.
4. <u>Constant practice with feedback</u>: Clean code, testing, documentation.
5. Transversal training git, testing, agile methodologies and collaborative work.
6. Building your professional profile portfolio + LinkedIn + GitHub.

In addition, there are **many quality online platforms**, with free training, at your own pace, and with or without certification. In the annex to this ebook I include a curated selection of resources with their respective links, so that you can study in the right direction, without wasting time or energy constantly changing paths.

Learning programming is not a race against time, it is a professional transformation that needs focus, patience and commitment to yourself.

"Encompassing" Person

This profile represents people who, with great enthusiasm, want to **learn several programming languages at the same time**, master multiple technologies and develop complex projects, without being fully aware of the **real difficulty of the learning curve** or the **time it takes to consolidate knowledge**.

It is an ambitious profile, and that ambition can be positive if it is well channeled. However, when it is accompanied by unrealistic expectations, such as wanting to master everything in less than a year, it becomes a **trap that generates frustration, exhaustion and dispersion**.

Many times, these people set goals such as:

- Learn FrontEnd, BackEnd, DevOps and testing... all at once.
- Work with Java, Python, JavaScript, C#, Go, and SQL in the same quarter.
- Trying to use multiple frameworks in a single project without mastering any.
- Develop a complex application without having done small or medium-sized projects before.

What happens in practice?

What usually happens is that **they don't finish anything** or don't delve into anything. They jump from course to course, from tutorial to tutorial, and although their intention is good, **the lack of focus blocks them**.

In addition, they do not consider the **transversal knowledge** they need for their projects to really work such as version control (Git), testing, documentation, use of IDEs, good practices, agile methodologies, continuous integration, among others. All that must also be learned... and it takes time.

Accompaniment and reflection.

Making them understand that they must **choose only one path for now** and **postpone the rest for later** is not always easy. Many of these people are not used to **temporarily giving up what interests them**. But if everything is a priority, nothing is.

My recommendation in these cases is clear and direct:

- **Specialize in what you currently like the most.**
- Take your time to **reflect and decide well**.
- Contrast your decision with the **real job market**, reviewing offers and requirements.
- Adjust your training path to **meet most of these requirements** and be employable.

If you want to prepare for all the possible technologies that interest you, it is likely that you will need **10, 15 or even 20 years** to master them with ease. But now you need to work.

Therefore, even if you have many concerns and desires to learn everything, **your priority today should be to specialize in an area and become a decisive, employable person with functional projects.**

Once you get a job and have a solid technical base, you can learn other languages and technologies in your free time or in new professional projects. You'll have a whole race ahead of you to do it. After that, you'll have time to explore everything else. Step by step, without haste, but with direction.

An entrepreneur of applications or video games to get rich

This profile is more common than it seems. These are people who come with a revolutionary business idea, usually related to an app or a video game, with the conviction that it will make them rich quickly. For this reason, they often ask for **private mentoring**, worried that someone could "steal their idea".

They are **in love with their concept** and do not usually detect any error or weakness in it, which prevents them from seeing the technical, economic and business reality that it entails to materialize it.

What happens in practice?

When you objectively explain to them what it means to develop an application or video game from scratch, **at least two years of continuous learning and practice**, plus specific knowledge of the Android/Apple ecosystem or video game engines such as Unity or Unreal, they are usually skeptical or impatient.

<u>They are also unaware of fundamental aspects such as:</u>

- That **one person cannot do everything** (design, development, testing, marketing, publishing, support, maintenance...).
- Who **need initial capital,** even if it is minimal, to meet costs such as:
    - Servers and storage.
    - Publishing in Stores (Google Play, App Store).
    - Brokerage fees (such as PayPal, Stripe, etc.).
    - Advertising, maintenance, analytics, licensing, and more.
- That **publication is not the end of the road**: If the application is successful, the costs and liabilities increase considerably. And in most cases, **they haven't made even a simple budget** or basic knowledge about how to make a **viable business plan**.

How to address it?

In this case, the first thing I try to do as a mentor is to land their idea in a more realistic action plan.

<u>I explain that:</u>

- They must **first be trained technically** before launching into the full development of their idea.
- They need to know and follow a **specific RoadMap** for application or video game development.
- Understanding the legal, economic, and logistical aspects of digital business is a must.
- Most likely, they need **to collaborate with other people** (designers, backend developers, marketing experts, etc.) or hire external services.

Entrepreneurship in technology is exciting, but it is not easy or fast. Success is not guaranteed. Even so, if you decide to move forward on this path and **learn to develop your prototype professionally**, that experience will be **highly valuable** for the job market.

<u>Even if the project doesn't get off the ground as expected, you can:</u>

- Include it in your **portfolio as an advanced and realistic project**.
- Tell your story in interviews and selection processes.
- Demonstrate that you know how **to work towards objectives**, solve complex problems, manage resources and make key decisions.

Because entrepreneurship, even if you do not achieve the expected success, makes you a more prepared and valuable person professionally.

Insecure or confused person

This profile is more common than we imagine, and often not enough is talked about.

There are people who feel insecure because they have started their training in technology at a later stage of their life. I also started "when I grew up", like many other people in the sector. It is completely valid. It's not when you start that's important, it's how you progress.

In a sector that demands more and more specialized professionals, **the best way to prove your worth is not your age or your formal training**, but your **real projects on GitHub or GitLab**, well documented, with commits that reflect your evolution and professional readmes that explain the development process.

The reality is that, regardless of our age, we must all go through the same technical, theoretical and practical tests during the selection processes. Therefore, what really makes the difference is **following a well-planned path, having patience and staying consistent**.

An additional challenge: Reconciling education and personal life

Balancing learning with family life and other responsibilities is a real challenge. It is often a **stage of personal and professional transition**, where time is limited, obligations increase, and doubts intensify.

<u>However, we also have unique strengths:</u>

- We have **work experience in other sectors**.
- We have **emotional maturity** and a greater capacity for commitment.
- And above all, when we set a goal, **we know what is at stake** Our children, our families, our future well-being.

That's why we tend to be more consistent, disciplined, and goal-oriented. We're not here because of fashion or curiosity: We're here because we've made a conscious and courageous decision.

Insecurity and information overload: Common enemies

It's completely normal to feel confused with so much information available. It's not always easy to differentiate between quality content and empty promises "sellouts", especially when you don't yet have the technical knowledge to make that filter.

That is why it is so important to have **good advice**, look for mentors or references who have already gone through that path, and rely on real experiences to make better decisions and avoid unnecessary mistakes.

REFLECTION-CONCLUSION

Having a clear roadmap, a learning path adapted to your specialty, is essential. Don't skip steps. Before you jump into programming with a language, make sure you have a good foundation in programming logic.

I recommend you visit www.roadmap.sh, a visual and professional tool where you will find detailed routes for different technological profiles. You can download them in PDF and always have them at hand as a guide.

If you have felt identified with any of the type people mentioned above, don't worry. I do not intend to judge or label, but to share what I have learned as a mentor and offer real solutions to common problems.

What to do when everything seems like too long a journey?

It is normal to be overwhelmed when seeing everything there is to learn. When I discovered what it meant to specialize in **Java with microservices**, I felt anxious. It seemed like a never-ending road. But then I remembered my personal purpose, **my Ikagai**. That which gives meaning to what I do and motivates me in the long run.

That purpose gave me perspective and allowed me to break down the path into shorter stretches, quarterly goals, weekly tasks, small daily achievements. This is how your progress is built.

First learning leap:

When you go from reading theory, doing examples and small exercises to developing your first **basic projects** (like a calculator). They are essential to consolidate knowledge, but they are not considered valid for the professional portfolio.

Second learning leap:

When you start with **projects of medium difficulty**, which already include databases and require connection and real functionality. This is where Postman comes into play to verify the correct functioning of the backend. A typical example, **your first functional CRUD**.

Insecurity, blockages and frustration have no age. They are an inevitable part of any learning process. But they are also signing that you are getting out of your comfort zone, and, therefore, growing.

Don't compare yourself. Don't beat yourself up. Trust your process. Talent is built with hard work, perseverance and purpose.

Phase 3: Small and medium-sized projects

The main objective of this phase is **to gain practical experience programming and developing small projects**, which allow you to build confidence and fluency with the language and tools you have chosen.

Until now, in previous phases, you are used to learning through examples, exercises and gradual activities that have helped you understand the theory and practice basic concepts.

This stage is essential to consolidate these learnings and begin to leave the comfort zone.

First projects: Confidence and motivation

I remember the excitement I felt when I made my first calculator, several forms of different levels of difficulty and my Landing page for a FrontEnd project with JavaScript.

These projects, although basic, represent a **first leap in learning** because they force you to integrate different concepts and tools so that everything works correctly.

These projects are not usually included in the professional portfolio, but they are crucial to acquire the necessary practice and demonstrate that you know how to apply what you have learned. For example, my first FrontEnd projects were developed in Visual Studio Code and to execute them I used a simple virtual server, easily installed through extensions.

Although I have a repository on GitHub, I haven't uploaded those FrontEnd projects because I currently specialize in Java. On the other hand, I did include all my exercises, examples and, above all, the microservices I developed.

Microservices: Small, Medium, and Advanced

As for my microservices projects, the first ones were small and used in-memory databases like **H2 Database** for pre-production, which allowed me to learn how to configure project-specific dependencies and master key concepts such as **dependency injection**.

Mid-level projects already included more robust databases, such as **MySQL** or **PostgreSQL**, which are commonly used in production environments. Finally, advanced projects combine both databases and consist of several microservices that communicate with each other, for example, using **OpenFeign**.

Ideas for small and medium-sized projects

<u>Small projects:</u> Calculator, task manager, basic games (such as Tic-Tac-Toe or Hangman).

<u>Medium-sized projects:</u> Full CRUD (Create, Read, Update and Delete) with connection to a database, for example, an application to manage users, inventories or appointments.

Clean code and best practices

Beyond the code working, it is essential that it is **easy to read and understand**.

That means:

- **Clean code**: Well-organized, with clear names for variables and functions.
- **Precise and concise comments**, which explain what is really important, without redundancies.
- **Frequent and descriptive commits** in Git, which allow you to professionally track progress, fixes, and new features.

Not only do these details help yourself maintain control and vision of the project, but they also make it easier to collaborate when working in a team.

Teamwork from the start

Even if you are learning and may now be working alone on your projects, it is very important to **acquire good teamwork habits from the beginning**. This includes:

- Use version control (Git) professionally.
- Document the code and decisions made.
- Follow coding standards.
- Practice clear and effective communication.

In this phase, the focus is on **applying what you have learned** and starting to build functional projects that give you the security to move forward to greater challenges. Remember that small and medium-sized projects are not just exercises, but the pillars that prepare you to create a solid portfolio, which will open doors in your tech career.

My mentor recommended me to learn how to use this Fork tool and I have been doing it ever since. To upload the first repository, I use Git-Gui that is installed while installing and configuring Git. You must have previously put the name of the repository on GitHub. To make the commits I continue with Fork.

My personal experience: From pure Java to microservices with Spring Boot

One of the **most significant leaps** in my career as a developer was when I went from working with monolithic projects in "pure" Java to building more complex and professional applications with **Spring Boot** and the entire **Spring Framework ecosystem**.

This change not only implied a transformation on a technical level, but also **an evolution in my mentality**: I began to understand the importance of architecture, code organization, and the scalability of projects.

Overcoming the Spring Ecosystem Challenge

At first, the learning curve was steep. I had a hard time understanding how to write Java within the Spring environment. Everything was different: the configuration, the structure of the projects, the use of annotations, the injection of dependencies, the connection to databases, etc.

To make the process easier, I decided to use **Spring Tool Suite 4 (STS),** an Eclipse-based IDE. I already had previous experience with Eclipse and NetBeans, so STS allowed me to stay in a familiar environment, but better suited to work with Spring.

Gain efficiency with the right tools

In software development, **time is our most valuable resource**. Using an IDE that allows you to create projects with all dependencies configured correctly from the start, such as STS, was key for me. I was able to focus on writing functional code and learning business logic, rather than wasting hours troubleshooting configuration errors.

Tips for becoming more versatile as a developer

One of the most valuable lessons I've learned is this, **versatility is more important than absolute mastery of a single tool.**

Learning to handle different IDEs (such as IntelliJ, Eclipse, VS Code), familiarizing yourself with multiple frameworks and having a foundation in more than one programming language makes you more adaptable. In the real world, companies work with varied tools and stacks, and being flexible allows you to more easily join any team or project.

In addition, the ability to learn autonomously and to get out of your comfort zone is one of the most valued assets by any technical team.

What is Fork and what is it for? Free friendly Git client

Fork is a Git client (a desktop application) that allows you to manage your Git projects and repositories in a visual, intuitive, and user-friendly way, without the need to use the terminal.

It is available for Windows and macOS.

What is Fork for?

1. Visualize branches, commits, and merges clearly. It has a graphical panel that shows you how branches connect, ideal for better understanding Git.

2. Clone, pull, push and fetch easily. You can work with your local and remote repos with just a few clicks.

3. Manage merge conflicts. When there are conflicts, it helps you resolve them visually, without getting bogged down with the terminal.

4. View the change history. You can review old commits, view diffs, and browse the full history of the project.

5. Create branches, stashes, and tags. All in a visual and intuitive way.

6. Integration with GitHub, GitLab, Bitbucket, and others. You can connect your accounts and work directly from the app.

Why use Fork if you already have Git?

While Git can be used only from the terminal, Fork makes it more visual and accessible, especially if you're just starting out or want to save time.

<u>It is ideal for:</u>

- ✓ Beginners who have not yet mastered Git per console.
- ✓ Developers who prefer a visual interface.
- ✓ Review branches and merges in complex projects.
- ✓ Alternatives to Fork (other visual Git clients):
- ✓ GitKraken (more visual, but with limitations in the free version)
- ✓ SourceTree (from Atlassian)
- ✓ GitHub Desktop (simpler, ideal for projects on GitHub)

"Fork is a visual Git client that helps you manage your repositories easily without relying on the terminal. Ideal for viewing branches, committing, resolving conflicts, and working with GitHub, GitLab, or Bitbucket. Very useful for learning Git in a practical way."

These skills prepare you for the real work environment, where collaboration is essential and culture shock can be great if you don't have these foundations.

Phase 4: Advanced Projects + Portfolio

From exercises to projects with real impact

When you reach this stage, it's no longer just about learning, but about **consolidating your experience as a developer**. You've made it through the basic exercises, small and medium-sized projects, and now you're ready to tackle more complex challenges, similar to those you'll encounter in real work environments.

Building advanced projects is not only about applying what you've learned, but also about solving real problems, making informed technical decisions, and simulating the professional work environment.

The challenge of advanced projects

<u>You've already reached a key point:</u>

- You've built up a solid base of knowledge.
- You are fluent in your main language and its associated tools.
- You know how databases, testing, and dependency integration work.

But this is where a new, more demanding reality begins. When you read job offers, you see that it is no longer enough to know how to program well; They ask you for specific frameworks, design patterns, scalable architectures, agile methodologies, soft skills, ability to work in a team, communication skills, among many others.

And it can feel overwhelming. You may be wondering, "How am I going to accomplish all that?", "Where do I start?", "What if I'm not really ready?"

It's completely normal to feel like you're still a long way off. That feeling of impatience, of not being "enough", appears in almost all people who are in this phase. The learning curve becomes steeper, the tasks are more complex, the expectations are higher, and concrete, real results are demanded, not just concepts.

But it is also at this point where the difference really makes itself, between a junior profile that is limited to the basics and one that begins to have a voice, that can hold interviews, propose solutions, adapt to new technologies, face projects with ambiguity, not only with clear instructions.

What skills are starting to be tested

1. **Design, patterns and architecture**
   It is no longer enough for the code to work: It must be well organized, scalable, sustainable. You begin to need to understand design patterns (e.g., MVC, repository, factory, strategy), SOLID principles, separation of responsibilities. Deciding architectures: monoliths vs microservices, how to organize packages/modules, how to handle scalability.
2. **Frameworks, libraries and complementary technologies** You are expected to handle some frameworks with ease, to understand not only "how to use it" but "why choose one or the other". Do not lack knowledge of the tools that accompany professional development: Version control with Git, containers (Docker), CI/CD, automated tests, documentation tools, metrics, logs, monitoring, etc.
3. **Solving real problems:** In this phase it is no longer just "solving textbook exercises", but facing real errors, complicated debugging, integration between several pieces, external dependencies, production errors. Know how to anticipate failures, handle limit cases, design with errors in mind, deal with ambiguous requirements, scope changes, refactoring, technical debt.
4. **Autonomy and responsibility**: You stop working always under detailed supervision; They already require you to be able to take on parts of a project, estimate times, distribute tasks, decide which approach to follow, communicate progress and obstacles. It is highly valued that you can solve problems without having to depend on someone else for every detail.
5. **Soft skills and team context:** Not everything technical. Now it matters how you communicate, how you collaborate, how you learn from your team, how you propose improvements, how you handle feedback. Know how to work with agile methods, standup meetings, sprints, retrospectives, integrate into multidisciplinary teams (developers, testers, designers, DevOps). Be reliable, deliver what is promised, respect quality.
6. **Mindset of growth and constant improvement:** See what you lack as a zone of opportunity, not as a failure. Knowing that you are going to have to learn new versions of frameworks, new tools, new architectures; that technology changes. That making mistakes is part of the process, but that each mistake brings a lesson.

How do you face this challenge and stand out?

Here are some strategies to move more confidently at this stage, so that the "overwhelming" becomes manageable:

- **Make real/personal projects** that force you to combine several pieces: various microservices, integration, databases, testing, deployment. Don't let everything remain in isolated tutorials.
- **Read code from others**: open source projects, complex examples. See how others solve architecture problems, handle patterns, structure modules. Ask yourself: why did they do this like this? What alternatives were there?
- **Practice system design**: even if it's sketching architectures, deciding what services you need, how they communicate, how you would handle scalability or failures. Even if you don't implement everything, thinking about it already makes you very strong.
- **Testing and quality from the start**: write unit tests, integration; catch bugs early; refactor. Do not leave quality for "when there is time".
- **Constant feedback**: ask for code reviews, talk about your decisions, accept criticism; see what you can improve. Share your code with others.
- **Specialize a little but maintain breadth**: have a strong command of your main tools, but also exposure to others: different frameworks, deployment tools, methodologies, secondary languages. That makes you more flexible.
- **Document what you do**: not just code that works, but that others understand: clear names, helpful comments, best practices, patterns that repeat well, logical structure. Being able to explain your decisions, not just make them.

What makes a junior profile "ready"

- That can show real projects, with visible code, where organization, good practices, tests, quality are appreciated.
- <u>That in technical interviews you not only answer syntax questions, but that you can reason:</u> Explain why you decided on a design option, how you would handle a possible problem, what you would do if a requirement changed.
- That you can take on tasks with a reasonable degree of autonomy, solving difficulties and mistakes on your own, without depending on being told "do this, do that".
- <u>That you show attitude:</u> Desire to learn, openness, humility, curiosity, responsibility. That you initiate improvements, propose optimizations, are interested in polishing details, that you are not satisfied with "what works" but what works well.

The need to demonstrate practical experience, even if you have no work experience.

If you haven't yet formally worked in the tech sector, your **GitHub/GitLab portfolio is your most valuable cover letter**. It's not just about uploading finished projects.

<u>What really conveys your professionalism is showing the complete development process:</u>

What will recruiters and technicians evaluate?

1. **Your Activity in commits**
    - Frequency, clarity, and progression.
    - They reflect how your code evolved, what bugs you solved, and how you organized the work.
2. **The functionality of the project**
    - Is it working properly?
    - Does it have a clear purpose?
    - Is it aligned with real problems in the sector or with a specific need?
3. **Maintenance and improvements**
    - Is it a project you created and abandoned?
    - Or have you spent time optimizing it, refactoring the code, adding new features, or resolving bugs?
4. **Clear and professional documentation**
    - A **well-structured README** that explains how it is installed, how it is executed, and what technologies were used.
    - Screenshots, diagrams, technical instructions and even explanatory videos if possible.
    - Take care of technical language and spelling: these are details that make the difference.

What to include in your portfolio?

Advanced level projects with complex functionalities, connection between microservices (if it is your specialty), error handling, validations, testing with JUnit, and good architecture practices.

Use of key tools such as Maven/Gradle, Docker, Swagger, Spring Boot, relational and non-relational databases, etc.

<u>Projects in which your growth is seen:</u> From the first to the most recent, to demonstrate evolution.

Think of your portfolio as a **living, professional project**, not a binder of loose tasks. Organize it, keep it clean and updated, focus on quality rather than quantity.

If you work with passion, strategy, and consistency, your portfolio will speak for you before you arrive for an interview. And if you also accompany it with a **well-worked professional personal brand**, such as a good LinkedIn profile, a technical blog or the publication of eBook's, you will be building a differentiating and powerful presence within the tech sector.

The portfolio is not a static showcase. It's a reflection of your learning process, your technical decisions, and your commitment to professional development

Difference between projects to learn and star projects

One of the best ways to structure your technical portfolio professionally is **to clearly distinguish between projects that you have developed to learn** and **those that represent your current level as a developer**.

This ranking will not only help you understand your progress but will also allow recruiters and technical teams to **clearly see your evolution, your priorities and your specialization**.

1. Projects to Learn and Cover Labor Market Requirements.

These are projects that you develop with the aim of **learning a specific technology**, consolidating key concepts or meeting requirements that you see on a recurring basis in job offers. These projects serve a practical function, they are **applied learning tools**, not necessarily exhibition pieces.

What are they for?

- Understand fundamental programming and development concepts.
- Apply basic design patterns and best practices.
- Practice the use of frameworks, libraries, testing tools, ORMs, databases, etc.
- Gain fluency in the use of tools such as Spring Boot, JPA, Maven, Git, among others.
- Simulate common work environment tasks to train your development and problem-solving logic.

What is expected of them?

- That they are **functional**, although not perfect.
- That they follow basic good practices (clean structure, naming conventions, code organization).
- That they are well documented at a technical level.
- That they demonstrate understanding, not just repetition of tutorials.

<u>Example:</u> A complete CRUD with Spring Boot, connecting to a relational database, with basic up, off, modify, and search operations, using Postman to validate endpoints.

2. Star projects for your CV and public portfolio

These are the projects that **you will proudly show** in interviews, on your LinkedIn profile, GitHub and CV. They represent your technical maturity, your ability to resolve, your way of thinking as a developer and your professional approach throughout the life cycle of a project.

These projects must be prepared to be evaluated by both **technicians** and **non-technical recruiters**. They are your letter of introduction and your biggest argument in interviews.

What should they have?

- A well-defined and scalable architecture (e.g., based on microservices).
- Clean and documented code, following SOLID principles.
- Automated testing with JUnit and tools like Mockito.
- Code quality analysis (e.g., with SonarQube or Sonar Lint).
- Dependency management with Maven or Gradle.
- Connection to production databases (MySQL, PostgreSQL).
- Use version control with well-documented Git (clear and structured commits).
- Professional technical documentation in the README.
- Optional but highly valued: Cloud deployment (Heroku, Vercel, Render, AWS, etc.) or containers with Docker.

Goal: That the technical team can review your code and say, "This is ready for production."

Why is it useful to distinguish between projects for learning and star projects?

Making this distinction is not only useful, but **strategically necessary** if you want to stand out as a technical professional and orient your profile towards real job opportunities.

Here's why:

Because not all projects have the same purpose:

There are projects that fulfill a formative function, they help you master a technology, consolidate knowledge and make mistakes without fear. Others, on the other hand, are projects with a professional vision, designed to show your level, your technical maturity and your way of working in real contexts. Mixing them up or treating them the same can lead to confusion for anyone who reviews your profile.

Because it helps you better communicate your progress:

When you separate your projects into these categories, you can better tell your story as a developer. You show a recruiter or technician how you have evolved, what technologies you have incorporated, and how you have gone from theory to practice in a progressive and coherent way.

Because you save recruiters time and orient your portfolio towards the type of work you want:

A technical recruiter doesn't have time to go through all your repositories. You want to see the projects that really matter. When you define your own star projects and which have been part of the learning process, you are facilitating their work, generating a better impression and focusing your candidacy on the positions you really aspire to.

Because it prevents you from falling into the trap of wanting all your projects to be "perfect" and ending up blocked without publishing anything.

Many aspiring developers get stuck trying to polish every detail of each project. The reality is that not all projects need to be spectacular – some are just stepping stones in your process.

If you try to make everything perfect, you run the risk of not publishing anything and that hurts you more than showing imperfect but functional code. Constant improvement is best demonstrated by the actual evolution of your projects, not by absolute perfection. Organize your portfolio as a map of your professional growth. Be transparent, show your process and select what to highlight judiciously. This is how you communicate maturity, focus, and clarity, three qualities that any tech company will value.

Next: My 3-star projects

Below, I present the three-star projects that I have developed and that are a key part of my professional portfolio and my CV. Each has been carefully selected for its technical level, its applicability in real production environments, and the deep learning they represented during their development.

They are projects designed not only to demonstrate knowledge, but also to face challenges like those posed in technical interviews and current work environments.

Now I am developing a project with STS of 2 microservices with SpringCloud and connected to each other through OpenFeign and then learn Docker, Kubernetes and AWS. What I learn later I will execute in the projects I mention to integrate and scale them with the new functionalities.

The Eureka project already has the 2 microservices connected with OpenFeign and only Dockerize and Kubernetes remain. Most likely, I will put the SpringCloud repository with Docker and Kubernetes in the CV.

Project 1: Microservices with Eureka + Docker + Kubernetes

Technical summary: This project consists of a Java microservices system, with service discovery using **Eureka**, containerization with **Docker**, and basic orchestration with **Kubernetes** (using **Minikube** or **Kind**). It is ideal for showcasing real deployment capabilities in modern, scalable environments.

Technical objetive:

- Dockerize each microservice independently.
- Create a functional docker-compose.yml file for local deployment.
- Implement deployment. yaml and service. yaml for Kubernetes.
- Register the microservices with Eureka Server and test the communication between them.
- Document the entire process in a technically and visually appealing README.md.

Added value: This project simulates a production environment with decoupled architecture, useful for technical interviews and to demonstrate competencies in basic DevOps applied to Java microservices.

Project 2: Distributed Architecture with Kafka in English.

Technical summary: This project implements a distributed architecture using **Kafka** as messaging systems. Microservices exchange events across queues, managing asynchronous data streams in real-time. The project is documented entirely in English.

Technical objective:

- Configure Kafka and/or RabbitMQ with Docker.
- Develop microservices in Java that publish and consume messages.
- Simulate a real-world use case (e.g., orders, notifications, or event processing).
- Document in English to practice technical vocabulary and facilitate international interviews.

Added value: Messaging technologies such as Kafka and RabbitMQ are in high demand in scalable microservices architectures. This project not only strengthens my skills as a Java backend, but also demonstrates technical and linguistic adaptability.

Project 3: Ubuntu + Ollama + ChatGPT + UI + Docker (local AI)

Technical summary: Development of a lightweight **local AI** stack with an interface to interact with language models using **Ollama** (similar to ChatGPT). Run on Ubuntu with **Docker**, with a basic user interface, designed as a personal tool and software prototype.

Technical objetive:

- Install and configure Ollama on Ubuntu.
- Create containers with Docker for on-premises services.
- Design a simple interface to consume the local API.
- Document the technical process step by step: installation, commands, resolved errors, screenshots, etc.

Added value: This project strengthens my skills in DevOps, technical documentation and use of modern tools. It's an opportunity to apply knowledge of artificial intelligence, UI, and Linux environments, as well as bring useful content to the community.

What am I currently working on?

I currently have more than **24 active repositories on GitHub**, many in the process of improvement and documentation.

I'm working on:

- Finalize the technical and visual README.md of my projects.
- Learn and apply Docker and Kubernetes in the full deployment of the three flagship projects.
- Optimize my GitHub profile to be understandable, clean, technical, and visually appealing.
- Send CVs with direct links to my featured projects on GitHub, including brief descriptions and stack used.
- Write **personalized cover letters** aligned with my personal tech brand, highlighting my merits and my differential value.
- Update my LinkedIn profile:
    - Add the star projects in the corresponding section (with links, screenshots, and descriptions).
    - Update technical skills and the "About Me" section with a warm, structured, and professional presentation.

Next steps

Develop more projects aimed at **scalable and resilient architectures**, simulating real high-demand scenarios.

To continue training and adapting to changes in the sector, keeping my portfolio updated and functional.

You don't need to be an expert in everything from the start. The important thing is to demonstrate your ability to learn, your ability to solve problems, and your commitment to quality. The structure, documentation and evolution of your projects speak of your professional level better than any degree.

Understanding the selection team

In a selection process for a technical position, **two different profiles** with complementary functions are usually involved. Understanding this is critical to knowing how to apply effectively.

1. Human Resources and Recruiters

These professionals are the first to filter profiles.

Its approach is more generalist:

What do you evaluate?

- Work experience, even if not directly in the tech sector.
- Academic and complementary training.
- Soft skills: communication, teamwork, conflict resolution.
- Compliance with the **minimum requirements** of the offer.
- Clarity in the CV and consistency in the LinkedIn profile.

Important: They do not always have in-depth technical knowledge. That's why it's essential that your CV and LinkedIn profile are written clearly, professionally and adapted to the language they use. They also highly value proactivity, order, and presentation.

2. Technical Team / Hiring Managers

They are the ones who delve into your profile from a technical and practical perspective. Their approach is much more specific:

What do you evaluate?

- Your **repository on GitHub or GitLab**: Structure, commits, code clarity, best practices, documentation, testing, deployment, etc.
- Ability to solve real problems and write maintainable code.
- Theoretical and practical technical tests.
- Experience in projects similar to those of the position.
- Technical level in live technical interviews or with automated systems.

Key: This is where your work on your own projects **makes a difference**. Your GitHub becomes your technical cover letter. **It's not enough to have projects uploaded**: they must be organized, documented, functional and reflect your evolution.

Public vs Private: What to show?

Public repositories: They are your professional showcase. Here you should include your **star projects**, well documented, with progressive commits and technical README. This is what the technical team will see without having to ask for access.

Private repositories: You can reserve certain projects privately (for example, more experimental or confidential personal projects). If a recruiter or technician asks you during the process, you can give them temporary access. Make sure they're also organized if you choose to show them.

Align your profile with the two stages of the selection process:

First, convince with a clear, well-structured CV and a professional LinkedIn profile.

Then, demonstrate with your visible work that you have the required technical level.

Your projects are more than lines of code, they are living proof of your learning, your way of thinking and your value as a professional.

How many projects to include in your CV?

You don't need to include **all** your projects. Ideally, you should select **between 3- and 4-star projects** that really represent:

- Your current technical level.
- The **specialty** you want to dedicate yourself to.
- Your ability to solve real problems and work professionally.

Each project must have a clear purpose and communicate **what you know how to do** and **how you think as a developer**.

What information should each project include in the CV?

For each project you decide to highlight, add:

- **Name of the project** and a brief description (maximum 2 lines).
- **Technology stack** used (languages, frameworks, databases, deployment tools).
- **What you learned,** or **a specific achievement** achieved during its development (example: integration with external APIs, cloud deployment, performance improvement, complex bug fixing, etc.).

Keys to a purposeful portfolio

Your portfolio isn't just a collection of projects. It' **s a strategic tool** that communicates your value as a professional.

<u>How to build it with intention:</u>

Align your projects with your career goals

- If you're aiming for the backend: demonstrate mastery of REST APIs, databases, security, testing, and architecture.
- If you're aiming for FrontEnd: focus on good UX/UI practices, responsive design, accessibility, and modern frameworks.

Updates frequently: Shows constant evolution. A portfolio that is unchanged in months conveys a lack of activity. You can:

- Add new functionalities.
- Improve performance.
- Refactor code.
- Add tests or IC/CD.

Be transparent In READMEs, she explains:

- What problems you encountered.
- How you solved them.
- What you learned during the process.

This shows professional maturity and your ability to self-evaluate.

Quality over quantity: It is better to have 3 **well-structured, functional and documented projects** than 10 unfinished or with errors.

Add extras that make you stand out: If the job offers you are interested in ask for:

- Automated testing (JUnit, Jest, etc.)
- Continuous integration (GitHub Actions, Jenkins...)
- Cloud deployment (AWS, Vercel, Render...)

Try to include at least one of these elements in your star project. That can set you apart from other candidates with the same stack.

Building a good portfolio isn't just about programming.

<u>It is also:</u>

- Make strategic decisions.
- Reflect on your professional purpose.
- Show how you face challenges.
- Communicate your evolution.

Your portfolio will speak for you before the technical interview. Use it as a powerful tool to tell your story as a developer.

Your portfolio says more than your CV

In today's technological environment, a **CV alone is not enough** to stand out. Many candidates claim to know how to use React, Java or Python, but that no longer **impresses**. What really makes the difference is to demonstrate it with **real, functional and well-structured projects**.

Your portfolio is tangible proof of your professional ability.

What does a good portfolio show?

A well-made portfolio isn't just a list of projects. It's a **window into the way you think, work, and problem-solve.** It conveys much more than your technical knowledge. Through it, recruiters and technical teams can observe:

Your sense of organization

How you structure your code, your repository, and the documentation reveals your level of professionalism. If someone new can understand your project easily, you're ready to work as a team.

Your creativity

Original projects or ingenious solutions show that **you not only follow tutorials** but are also able to contribute ideas and value.

Your commitment to quality

Clean, well-documented code with automated testing, error control, and active maintenance speaks to your **work ethic and professional commitment**.

Build a portfolio that tells your story

Your portfolio should help tell your professional story better than any traditional CV.

Showcase projects that:

- **Solve real problems** or simulate professional use cases.
- Include **clear explanations** and documentation that makes it easier to understand.
- Use **technologies relevant** to the labor market.
- Reflect **progressive learning**, with improvements over time.

Your portfolio: Your best professional positioning tool

In the technology sector, your portfolio is much more than just a compilation of projects. It's a tangible representation of your journey, your effort, and your evolution as a developer. It is, without a doubt, the best way to position yourself professionally and differentiate yourself.

A well-constructed portfolio says much more than any resume. Before you speak in an interview, you can already convey your technical level, your mindset of continuous improvement and your way of solving problems.

What should a professional portfolio convey?

Constant activity and evolution:

A good portfolio is alive. It is not a static repository, it must show that you are on the move, that you study, improve, correct and update. Regular activity in your commits, the incorporation of new functionalities and code optimization demonstrate technical maturity and professional commitment.

Well-structured technical documentation:

Documentation is key. It's not enough for your code to work: it needs to be understandable. A good README should explain what your project does, how it is executed, what technologies it was developed with and what your process was. It's also helpful to include screenshots, relevant commands, possible errors found, and how you solved them. This shows your ability to communicate clearly, a skill that is highly valued in collaborative environments.

Quality over quantity:

It is not about having dozens of projects. Ideally, you should have three or four who really represent the best of your work. Each must be well structured, clean, functional and documented. It is preferable to have a small set of solid and professional projects than many unfinished or irrelevant ones.

Purposeful Projects:

Your portfolio should be consistent. Each project should be linked to your professional profile and the career goals you are pursuing. If you want to work as a backend developer, focus on projects that demonstrate mastery in robust architectures, microservices, databases, REST APIs, unit testing, and deployment. If your profile is FrontEnd, your portfolio should reflect attention to visual detail, accessibility, interaction, and user experience.

The portfolio as a strategic tool

A professional portfolio serves multiple functions within your job search and career growth strategy:

- Reinforce your CV with real evidence of your skills.
- It gives you confidence during interviews by having projects to explain in depth.
- Support your digital presence, especially on LinkedIn, GitHub, and your personal page.
- It serves as a basis for technical networking, demonstrating your value in community.

Beyond appearance, a strategic portfolio communicates strength, maturity, and professional direction. It's a positioning tool: it makes you visible, differentiates you, and prepares you for demanding environments.

A good portfolio not only teaches what you know, but how you apply it, how you think, and how you learn. Building it takes time, but it's a must-have investment if you want to stand out in a competitive job market.

The projects you include should be aligned with what you want to achieve. You don't need to know everything or have worked in large companies. What really matters is that you show initiative, perseverance, an improvement-oriented mindset and the ability to solve problems autonomously and professionally.

This portfolio will be your ally. It will open doors for you, support your interviews, and reflect your evolution as a developer. Take care of it, keep it updated, and use it with purpose.

Projects that solve real problems

When selecting projects that will be part of your portfolio, prioritize those that **respond to a specific need or solve a specific problem**. This not only shows that you know how to code, but that you understand how to apply technology in a useful, practical, and impactful way.

A technically correct project is not always enough. What really makes it valuable is its **ability to generate real value**, whether it's for end users, work teams, companies, or even for yourself as a developer.

For example, a basic task management app can be useful to practice, but if you give it a more comprehensive approach, such as adding collaborative functionalities between users, sending automatic reminders, syncing with external calendars, or allowing customization by tags and priorities, you're showing a **deep understanding of the problem**, an ability to design more elaborate solutions, and attention to user experience (UX).

Why are these projects more attractive to recruiters?

1. They simulate **real work environments**, where complex and multifaceted problems are solved.
2. They show that you have **product vision**, not just technical ability.
3. They reflect **analytical skills**, empathy with the user and informed decision-making.
4. They show that you are capable of **designing, implementing, maintaining, and improving** a solution over time.

This type of project also allows you to develop and showcase **transversal skills** that are highly valued in the work environment: teamwork, user-centered design, scalable architecture, data management, service integration, process automation and technical documentation.

Quality over quantity: Less is more if it's done right.

A common mistake when building a portfolio is to focus on quantity. But in technology, as in many other fields, **quality is what really makes the difference**.

Instead of filling your repository with a dozen incomplete or messy projects, focus on **developing three to five solid, functional, and well-documented projects**. A single excellent project can open more doors than ten mediocre ones.

Characteristics of a well-crafted project

For a project to convey professionalism and commitment, it must meet certain criteria:

- **Full functionality:** That meets the objectives it sets out, without obvious errors.
- **Clean and maintainable code:** Clearly structured, with descriptive names, organized logic, and consistent style.
- **Tidy structure:** Well-organized folders, separation between layers (e.g., model, controller, services), and use of good architectural practices.
- **Effective version control:** Clear and frequent commits that show evolution and troubleshooting, ideally with well-managed branches.
- **Clear technical documentation:** Complete README including description, installation instructions, technologies used, functionalities, possible improvements, and additional notes such as known errors or design decisions.
- **Testing and deployment (when possible):** Automated testing, continuous integration, and deployment on-premises, cloud, or container (e.g., Docker/Kubernetes) are a big plus.

A quality portfolio builds trust

Recruiters and technical teams don't have time to review large amounts of code. What they are looking for is **consistency, professionalism and depth** in your skills. When they find a project that reflects this, they understand that you are a person capable of working in a real environment, with a team mentality and attention to detail.

In short, each project in your portfolio should be a **key piece that communicates what you know how to do, how you do it, and where you want to go**. Bet on the quality, purpose, and value you bring with every line of code.

Essentials for every project

For each project you include in your portfolio to be **impactful, easy to evaluate, and professional**, it is essential that it contains certain key elements. Not only will these make it easier to understand your work, but they will also show your level of detail, organization, and commitment.

1. Clear and concise description

The description is the first impression that anyone who visits your portfolio will have. It must explain in a simple and direct way:

- **What the project does**: its main function and key features.
- **Who it's for**: who benefits from or uses this solution.
- **What problem it** solves: the context or need that motivated its development.

An accessible explanation, even for people without deep technical knowledge, helps to quickly communicate the real value of your work and its practical impact.

2. Online screenshots or demos

Including functional images or short videos that show the interface and behavior of the project makes it easy to understand immediately.

- **Screenshots**: show the visual appearance and flow of the application.
- **Demo deployed online**: if possible, include a link to a working version on the web or on a public server (e.g. GitHub Pages, Heroku, Vercel). This shows that you know how to publish and maintain projects in production or accessible environments.

The ability to visually explore the project lowers the barrier for testers and recruiters, who can validate the functionality without installing anything.

3. Link to the repository on GitHub or another platform

<u>The source code must be available and professionally organized:</u> A clean repository, with a logical folder structure.

A **complete and well-written README.md file**, which explains:

- What the project does.
- How to install and run it step by step.
- Technologies and tools used.
- Technical challenges you faced and how you overcame them.
- Potential future enhancements or pending features.

A quality README is a clear sign of professionalism and makes it easier for other developers or recruiters to understand, use, and evaluate your project autonomously.

4. Technologies used and key learnings

<u>Beyond listing frameworks or languages, it is very valuable that you explain:</u>

- What technologies you used and why.
- What new skills or concepts you learned during development.
- What technical difficulties you faced and how you solved them.

<u>You can add:</u>

- "I implemented global health management with Redux to optimize performance and simplify data flow."
- "I designed efficient RESTful APIs, reducing response times by 30%."
- "I learned how to implement unit and integration testing with Jest and Cypress."

This approach shows that you not only know the tools, but that you reflect on your learning and are able to evolve continuously.

Including these elements in each project in your portfolio will not only make it look more professional, but also make it easier for recruiters and technical teams to assess your real experience.

Remember that a portfolio is a living tool that speaks for you before any interview, so invest time in taking care of these details and clearly communicating what you contribute with each line of code.

What kind of projects should you include in your portfolio?

Not all projects have the same weight or value for your portfolio. It is essential to select those that best reflect your strengths, skills and, above all, that are aligned with the professional profile you want to project and achieve.

CRUD Projects (Create, Read, Update, Delete)

These projects are the fundamental foundation for any developer, as they demonstrate that you know how to manipulate data and build essential functionalities. However, to make them stand out, try to add additional functionality or enhancements that show creativity and more advanced mastery, such as complex validations, state management, or integration with external APIs.

APIs or backend services

Developing your own API or microservice is a great way to show that you understand server logic, communication between systems, and software architecture. This type of project is usually highly valued in backend or FullStack profiles, and reflects your ability to design scalable and efficient systems.

FullStack Projects

Combining FrontEnd and BackEnd in the same project is a powerful demonstration of versatility. This type of work reflects your ability to build complete solutions that span the entire development cycle, from the user interface to data management and logic on the server.

Clone of existing applications

Creating functional replicas of popular apps, such as a basic clone of Twitter, Trello, or Instagram, helps you understand best practices, design patterns, and technologies used in the industry. In addition, it allows you to demonstrate technical skills applied to recognizable and relevant scenarios.

Freelance or volunteer projects

If you've worked on real client projects or community initiatives, include those. This experience evidences your ability to work in a team, manage expectations and deliver value in professional contexts, showing commitment and responsibility.

How to Present Your Portfolio

Your portfolio should be much more than just a list of links to repositories. It is key to have a simple, professional and well-organized website, which allows visitors to explore your projects easily and deeply.

Essential features of your portfolio website

Clear organization: Present your projects in an orderly way, with sections or categories that make it easy to navigate.

Detailed descriptions: Clearly explain what each project does, who it was developed for, and what problems it solves.

Images or demos: Add screenshots or links to online demos so that the visitor can appreciate the real operation without having to download anything.

Links to code: Include direct links to your repositories, making sure they're complete and documented.

Technical details and learnings: Highlight the technologies used and what new skills you acquired in each project.

Recommended platforms to deploy your portfolio

GitHub Pages**:** Ideal for static projects with direct integration to repositories.

Netlify or Vercel: Perfect for modern applications with automatic deployments and advanced features.

Custom sites: If you want to demonstrate web design and development skills, creating a portfolio from scratch is an excellent cover letter.

Best practices for design and experience:

- It uses a clean and professional design, with legible fonts and a harmonious color palette.
- It makes it easy to navigate so that the user can quickly find what they are looking for.
- It reflects your personality and professional style, but without overloading the design.
- Include your contact information and links to professional profiles, such as LinkedIn or Twitter.

The strategic importance of your portfolio

Today, a traditional resume may list your knowledge, but it can't demonstrate how you apply that knowledge in real projects. Your portfolio is tangible evidence of your ability to transform ideas into functional solutions.

In addition, it shows your way of thinking, your coding style, your creativity and your ability to face and solve specific problems.

That's why your portfolio is your most powerful cover letter. It is not enough to include technical exercises or demos without context; Recruiters especially value projects that solve real problems and provide tangible value to users or companies.

Key guidelines for your portfolio:

- Prioritize projects that solve real problems and have practical impact.
- He prefers to have a few well-prepared and complete projects than many unfinished or undocumented ones.
- For each project, include a clear description, images or demos, links to the code, and reflections on the learnings obtained.
- Align your projects with the professional profile you want to develop, and the job offers you aspire to.
- It presents everything on a neat, professional, and easy-to-navigate website that provides a positive visitor experience.

Building a strong and consistent portfolio will not only make you stand out from other candidates, but it will also give you confidence to confidently present yourself in interviews and new opportunities.

Your portfolio will be the key that opens many doors in your technological career, because it will speak with facts and demonstrate your true professional potential.

Phase 5: Profile Optimization

The optimization of online profiles is a recurring and crucial topic in both group and individual mentoring.

However, many professionals do not give it the importance it really deserves nor are they aware of the impact that their digital presence has on the construction of their Professional Personal Brand within the technology sector.

The reality of our online identity

Everyone, without exception, has a digital identity. Some intentionally build and strengthen it while training and developing professionally, but others simply leave it to chance, without paying attention to what they transmit or how they can be perceived.

Recruiters and recruitment teams carefully review the links we include in our resumes, such as LinkedIn, GitHub or GitLab and, if they deem it necessary, even do Google searches to find out more about us.

In today's digital age, obtaining data on anyone is easy, and many times they do it for recreational purposes without measuring the consequences that this can have on selection processes.

A personal perspective on social media

Personally, I haven't used social media for years due to the lack of actual use and because I consider it a constant source of exposure to cybercriminals and people with malicious intent.

The personal information we share online can fall into unknown hands and we don't know how it could be used against us.

The importance of building a strong Professional Personal Brand

Just as companies build and take care of their corporate brands, we, as professionals, must develop and strengthen our **Professional Personal Brand**. This is a strategic representation of who we are, what value we bring and how we want to be perceived in our sector, regardless of the industry in which we work.

The Professional Personal Brand does not arise by chance; It is the result of conscious and constant work that reflects our experience, skills, values and professionalism.

The current situation of many technical professionals

Many developers and tech professionals who already have a good technical level or are close to it, spend little or no time reviewing and optimizing their online presence, especially on key platforms such as LinkedIn and code repositories such as GitHub or GitLab.

This is a critical mistake, because the first impression a recruiter has of us often comes directly from these digital profiles.

An incomplete, outdated, or sloppy profile can lead to us being perceived as amateurs or professionals who don't pay attention to detail, something that can cost us valuable opportunities.

Our goal in this phase

We want that, when a recruiter or employer visits our professional profiles, they are clear that we are rigorous, committed and careful professionals with our image and work.

That they perceive in our profiles a real and faithful reflection of our technical level, our work ethic and our dedication to continuous learning.

GitHub or GitLab profile: your platform for your projects as a programmer or developer

GitHub and GitLab are the ideal platforms to host and showcase all your projects and learnings as a programmer or developer.

Through them, recruiters and teammates can see not only the code, but also the evolution process through commits and updates that reflect your progress and continuous improvements.

The importance of good README.md

It is common to find repositories with many commits, but the absence or low quality of the README.md file can significantly devalue a project. The README is not only an aesthetic detail, it is a clear sign of professionalism, because in the work environment documenting the project is a fundamental part of the work.

I recommend writing the README once the project is finished or in a stable phase, since important changes usually arise during development, such as the replacement of dependencies or the incorporation of new knowledge that you did not initially consider.

Strategies to maintain motivation and productivity

I confess that documentation is the part that makes me laziest, and sometimes procrastinous. However, I've learned that planning this task makes the process much easier.

For example, during development I take screenshots of each feature or section, saving them organized in specific folders. This saves me time when it comes time to write the README, as the images are ready to use.

My routine includes creating a draft where I explain the development process, the technological stack used and the phases of the project, accompanied by the screenshots. Then, I edit the content with Markdown, a tool that I didn't know at first but that is very useful to give professional formatting to the README.

The preview offered by these platforms allows me to review and correct before publishing, and this whole process is recorded as commits, showing my progress also in the documentation.

Visual consistency and professionalism in your profile

Your profile picture on LinkedIn and GitHub/GitLab should be the same to maintain visual consistency between your professional platforms. In addition, it should be a professional image, light background, preferably white, and formal attire, such as shirt and jacket.

Take advantage of the sections to include relevant links, such as your LinkedIn profile, blog, YouTube channel, published eBook's, or any other merit you want to highlight.

For example, in my case, I have linked my LinkedIn profile and plan to add links to my eBook's published on Amazon KDP soon.

Beyond Projects: The "About Me" Repository

In addition to showcasing your learning projects and your star projects with their respective well-crafted READMEs, it is advisable to create a specific and empty repository, dedicated to a special README that works as a professional personal presentation, like a LinkedIn profile, but adapted to the code platform.

<u>This README must include:</u>

- A summary of who you are and what your most important merits are.
- Links to your featured projects, accompanied by a brief description and the tech stack used in each.
- Links to your profiles on other platforms and a quick explanation of the purpose of each.

If you want, you can check out my GitHub profile that I have included in my ebook to get ideas on how to structure this type of repository. These are small details that make a big difference and elevate your online presence, showing a level of professionalism that recruiters' value highly.

How to optimize your profile on GitHub or GitLab to stand out as a developer?

GitHub is much more than a code repository. It's your **technical showcase**. Here, employers, recruiters, and other developers can assess your skill level, your ability to document, your way of working in a team, and your evolution as a professional. Optimizing your profile requires **organization, quality, perseverance, and strategic focus**.

Below are the most common mistakes and how to fix them to transform your profile into a true technical cover letter.

Common GitHub Errors and How to Fix Them

1. Empty or incomplete repositories

Problem: They give the impression that you are inactive or that you do not finish what you start.

Solutions:

- Delete repositories with no content, failed tests, or irrelevant projects.
- Keep each repository with a clear purpose, complete files, and consistent structure.

2. Lack of documentation

Problem: Without a **README.md**, even a great project loses value.

Solutions: Make sure each repository includes a well-structured README.md with:

- Purpose of the project
- Technologies and tools used

Installation and Usage Instructions:

- Key functionalities
- Screenshots or demos if possible

A repository without documentation is like a product without instructions: no one will know how to use it or what makes it special.

3. Generic or confusing names

Problem: Names like "Project1" or "TestReact" don't communicate value.

Solution: Use descriptive and concise names. Examples:

- API-REST-Java-SpringBoot
- Ecommerce-React-Django
- ToDoApp-Vue-Firebase

4. Non-descriptive commits

Problem: Vague messages such as "update" or "final" make it difficult to track changes.

Solutions: Use clear and specific commit messages, for example:

Fix: Fixed error in the validation of the login form.
Feat: Added multi-language support in FrontEnd.

Good practices in git reflect professionalism and attention to detail.

5. Ignore Issues

Problem: Not resolving or closing issues indicates disinterest or lack of maintenance.

Solutions:

- Address issues quickly.
- Documents the solutions applied when they are closed.
- Use tags (bug, enhancement, question) to better organize.

6. Do not use. Gitignore

Problem: Uploading unnecessary files such as local settings, node_modules folders, or temporary files makes the repository dirty.

Solution: Add a. gitignore file appropriate for the language or framework used. GitHub offers predefined templates.

7. Lack of branching and tags

Problem: Working only in main or master limits collaboration and complicates version control.

Solutions:

- Create branches (feature/feature-name, fix/bug-name) for new features or fixes.
- Use tags (v1.0, v2.0-beta) to mark stable versions of the software.

8. Outdated profile

Problem: Reflecting old or irrelevant projects generates disinterest.

Solutions:

- Update your projects regularly.
- Use the "**Pinned repositories**" option to highlight 4-6 key projects that showcase your current experience and best practices.

9. Lack of collaboration

Problem: Not participating in other people's projects limits your growth and visibility.

Solutions:

- Contribute to open-source projects.
- Report and resolve issues in other repositories.
- Participate in Hacktoberfest, GitHub Discussions, or developer forums.

Collaborating not only expands your professional network, it also demonstrates your ability to integrate into real teams.

Additional tips to make you stand out even more

- Add a clear bio to your profile: "Full Stack Developer | React – Node.js PostgreSQL | Automation and DevOps enthusiast."
- Add links to your web portfolio, LinkedIn, and certifications.
- Use GitHub Pages to host documentation or demos of your projects.
- If you use GitLab, apply the same principles to documentation, branches, commits, issues, etc.

GitHub as a reflection of your professional value

An optimized profile on GitHub is much more than a code repository, it's tangible proof of your technical skills and ability to work professionally.

Not only does it show that you know how to code, but you also know how to organize projects, document your work, collaborate with others, and think from a holistic perspective of software development.

However, your technical profile on GitHub should be part of a comprehensive professional online presence strategy. This is where LinkedIn comes in. While GitHub shows what you know how to do and how you do it, LinkedIn communicates who you are, how you work, and why you're valuable to a team or company.

The synergy between GitHub and LinkedIn

Both platforms must work together to boost your personal brand:

LinkedIn provides you with visibility, reputation, and professional connections with recruiters, technical leaders, and industry peers. It is the space where you build and show your professional history, your soft skills, your achievements and recommendations.

GitHub is your technical showcase, where you can see your level of coding, your work style, your ability to solve real problems, and your constant evolution through your projects and contributions.

The importance of consistency and constant updating

To create a solid, trustworthy, and attractive personal brand, both profiles must be up-to-date and consistent in terms of image, information, and messaging. This builds trust and makes it easier for those evaluating you to understand your professional profile clearly and completely.

In today's competitive and dynamic tech industry environment, it's not enough to have technical knowledge. Knowing how to present your skills and experiences through a professional and strategic online presence is what will really make a difference and open doors.

It is not enough to know, you must know how to show what you know

Your personal brand on LinkedIn:

Your Personal Brand on LinkedIn: The key to standing out in the tech job market

Having an excellent technical level is essential, but it is not enough to stand out in a competitive market. The difference is your **Professional Personal Brand**, and a simple and effective way to build it is by optimizing your profiles on **LinkedIn** and **GitHub**.

Professional vs. Social Media: Understand the Difference

It's common to confuse them, but **a professional network is not a social network**. This distinction is crucial to manage your digital identity and project a coherent and professional personal brand, especially if you are looking for a job or want to advance your career.

Social networks (Facebook, Instagram, TikTok) are geared towards entertainment, personal life, and informal interactions.

Professional networks (LinkedIn) are designed for work networking, showcasing your career, skills, achievements and fostering professional connections.

Digital Identity vs. Personal Branding

Digital identity: It is the set of all the information about you on the Internet (publications, photos, comments, projects, etc.), both personal and professional.

Professional Personal Branding: It is the strategic and deliberate part of your digital identity, the way you choose to present yourself professionally and the impression you leave on those who can offer you opportunities.

Confusing the two can damage your professional image. An outdated or informal LinkedIn profile can raise doubts about your seriousness, even if your CV is impeccable.

LinkedIn: A key tool in selection processes

Recruiters often check the LinkedIn profile to get to know the candidates better.

Therefore, it is essential to:

- Keep your profile up to date and well-written.
- Include a clear and engaging professional summary.
- Showcase projects, technical skills, achievements, and recommendations.
- Participate in the community, commenting and sharing relevant content.

The importance of "About Me" on LinkedIn

The "About Me" section is your professional cover letter:

1. **Professional first impression:** This is the first thing many recruiters read after the headline. In seconds, it should convey who you are, what you do, and what sets you apart.
2. **Reflect your Personal Brand:** Summarize your career, strengths, values, and goals. Humanize your profile beyond a job listing.
3. **Improve your rankings:** LinkedIn uses keywords in this section to show you in relevant searches. It includes key technologies, methodologies and roles.
4. **Demonstrate your evolution:** An updated "About Me" shows that you are active, learning, and growing professionally.
5. **Professional pitch in text:** It works as a mini presentation that you can use in interviews or networking.

How to write a good "About Me"?

- Use a professional yet approachable tone.
- Write in the first person.
- Be brief, clear, and avoid unnecessary jargon.
- Add a call to action, for example:
  "Currently open to new challenges in backend development with Node.js and SQL/NoSQL databases."

It includes key data such as:

- Years of experience.
- Technologies and tools mastered.
- Outstanding achievements or projects.
- Motivation and current goals.

When to update your LinkedIn profile?

- At the end of an important project.
- By learning a new featured skill.
- When changing roles or sectors.

Optimizing your LinkedIn and GitHub profile not only improves your visibility, but also projects the professional image you want to convey and opens doors in your tech career.

The Skills section on LinkedIn: Much more than an ornament

The **Skills** section on LinkedIn is not only a decorative element in your profile, but a fundamental part that directly influences how recruiters find you and the professional perception you generate.

In my case, I have approximately 75 skills out of a possible 100, and I keep adding new skills as I learn them, such as **Docker, Kubernetes, Kafka, RabbitMQ, Ubuntu**, among others.

Some I have validated through training, others through projects and many have been supported by professional contacts on LinkedIn. If you want, we can connect and validate each other's skills to strengthen our profiles.

In addition, recommendations are also an excellent resource for improving professional credibility.

Why is the Skills section on LinkedIn so important?

1. **Improve your search rankings LinkedIn** uses skills as keywords to display profiles when recruiters search for specific terms, such as "React," "Node.js," or "Scrum."
   If you don't have an added skill on your profile, you won't appear in those filters, even if you have real experience with it.

2. **Helps you match job posting requirements**
   Many job postings include a list of required skills. LinkedIn compares those skills to those in your profile and ranks you better if there are matches. In addition, a motivational message usually appears:
   "You are a good candidate for this position. You have 8 out of 10 required skills."

3. **Provide credibility through** validations When former bosses, colleagues, or collaborators validate your skills, LinkedIn shows how many people confirm your mastery in those areas.
   A skill with many validations builds confidence and gives a strong image of your professionalism.

How to validate your skills on LinkedIn?

1. **Request validations from people you have worked with** Be specific in your request. For example:
   "Could you validate my Python and Django skills in the project we did together?"

2. **Validates the skills of others in a genuine way** Reciprocity is common. If you validate the skills of your contacts, many will do the same for you.

3. **Participate in LinkedIn Skills** Assessments These are quick tests that measure technical knowledge, such as HTML, Git or JavaScript's you pass, LinkedIn displays a badge on your profile, for example: *"Top 30% in SQL Assessment",* which increases your visibility and attractiveness to recruiters.

Recommendations to make the most of this section

Add only real skills relevant to your industry or the role you're looking for.

Sort your skills by priority: The first three are the most visible, so put there the ones that best represent you for your professional objective.

Adapt your skills to the language of the labor market: Review the job offers that interest you and adjust your skills according to the most used terms. For example, if most ask for "Spring Boot" instead of just "Java," include it.

Update regularly**:** Add new skills after trainings, certifications, or major projects to reflect your growth.

The Skills section on LinkedIn is much more than just a list: it's the bridge between what you know how to do and what companies are looking for. Optimizing it positions you better, adds credibility, and helps recruiters identify you as the ideal candidate.

LinkedIn Projects Section for Programmers and Developers

LinkedIn provides a specific section for you to **highlight your technical projects**, directly linking the most relevant repositories hosted on platforms such as **GitHub** or **GitLab**. You can also include links to demo pages or relevant documentation.

This section appears in a highly visible place on your profile, ideal for capturing the attention of recruiters and professionals in the sector.

However, it's common to find this section empty, even though LinkedIn tends to remind you of it constantly. It's no coincidence: for technical profiles, **the Projects section is essential to demonstrate your practical experience** and give more value to your Professional Personal Brand.

How to create and optimize the Projects section on LinkedIn?

1. **Create a specific "Projects" section** in your profile and start linking the repositories or demos of your most prominent projects.
2. **Include a clear, results-oriented description for each project**, answering key questions:
    - **Purpose:** What need, or problem does the project solve?
    - **Technologies used:** Frameworks, languages, tools, methodologies.
    - **Impact or result:** Measurable results, improvements implemented, or significant learnings.
3. Example of a professional description for a project:

"I developed a microservices-based application using **Spring Boot** and **PostgreSQL**, improving the scalability of the system by 40%. I implemented secure authentication with **Spring Security** and designed a user interface with **Thymeleaf**."

4. **Link your GitHub or GitLab profile** in the contact section and in the description of your projects to facilitate direct access to your work.
5. **Organize your repositories well**: Make sure each project has a **clear and comprehensive README.md** that explains:
    - The overall functionality of the project.
    - The structure of the code and dependencies.
    - Technologies and versions used.
    - How to install and run the project.
    - Screenshots or demo, when possible.

Additional tips to maximize the impact of your projects on LinkedIn

Update this section regularly: Add new projects or improve descriptions to reflect your progress and new knowledge.

Highlight projects relevant to the role you are looking for: Focus on those that showcase the skills and technologies in demand in your sector.

Include metrics and measurable results whenever possible, because they help demonstrate the concrete value of your work.

Incorporate multimedia with screenshots, videos, or links to demos, to make the presentation more engaging and visual.

Share your projects in professional publications and groups to increase their visibility and generate interaction with your network.

The projects section on LinkedIn is a unique opportunity for recruiters to not only read about your skills, but **to see and evaluate your actual work**.

Supplementing your profile with links to well-documented repositories and professional descriptions turns your profile into a solid and reliable digital portfolio.

Do not underestimate this section, it is one of the key elements to differentiate yourself as a serious, committed professional with practical experience.

GitHub/GitLab: Essential technical add-ons for your Personal Brand

In addition to LinkedIn, if you work in technology or software development, **it is essential to include links to your profiles on GitHub or GitLab in your CV and professional profile**. These platforms allow recruiters to evaluate your hands-on work directly and in-depth.

What can recruiters review on your GitHub/GitLab profiles?

- **Your public repositories:** where you host your projects and exercises.
- **The README.md files:** which explain the objective, use, technologies used and operation of each project.
- **Your coding style and project structure:** code quality, organization, and best practices.
- **Frequency and quality of your commits:** they show your consistency, evolution and professionalism.
- **Ability to document:** The importance of communicating the project well goes beyond code.
- **Specific cases that demonstrate your technical and methodological skills**, complementing what you write in your CV.

Including these links provides transparency and trust, as well as demonstrating your commitment to quality and continuous learning.

How to avoid the most common mistakes on LinkedIn and boost your Personal Brand

LinkedIn is a powerful tool for building and strengthening your professional personal brand. On many occasions, it is the first point of contact between a candidate and recruiters, companies or potential collaborators. Therefore, your profile must project a complete, coherent, up-to-date, and professional image.

Here are some common mistakes and recommendations to optimize your LinkedIn presence:

1. Not having a professional profile picture

The profile picture is the first thing that catches the eye of those who visit your profile. An unprofessional image or the absence of a photo can generate distrust or seem unserious.

How to do it right:

- Use a clear, well-lit photo, centered on your face.
- Wear formal or office wear such as a shirt, blouse, and jacket.
- Choose a neutral background, such as white, light gray, or a minimalist setting.

Avoid:

- Photos in informal contexts (parties, holidays, selfies).
- Images with exaggerated filters or illustrated avatars.

A good photo conveys confidence, closeness and commitment to your professional image.

2. Inconsistent use of photographs and banners

Visual consistency between your professional platforms (LinkedIn, GitHub, online portfolio) is key to building a strong and recognizable personal brand.

Recommendations:

- Use the **same profile picture** across all your platforms to make it easier to recognize.
- Choose a custom banner on LinkedIn that complements your profile:
    - It can be related to your field of expertise (technology, web development, data analytics, design, etc.).
    - Include subtle graphic elements, such as your name, job title, or a short phrase that represents your professional vision.
    - Avoid generic, pixelated, or unrelated banners for your workspace.

3. Incomplete or outdated profile

An incomplete profile can give the impression of improvisation or lack of interest. An outdated profile, even worse, can make you miss important opportunities.

How to optimize your profile?

Professional title: Beyond your current position, include keywords related to your specialization.
Example: "Java Developer | Passionate about technical training and mentoring."

About me: Write a concise paragraph that describes who you are, what you do, what your professional interests are, and what sets you apart. Use a professional yet approachable tone.

Experience: Describe your roles, responsibilities, and accomplishments clearly. Use bullet points to make it easier to read.

Education and certifications: Keep all your relevant studies and courses up to date.

Projects and repositories: If you're tech-savvy, link your GitHub or GitLab profiles. Make sure README.md are well-written, clear, and reflect your skills and contributions.

Skills and validations: Add key competencies (e.g. Java, Git, teamwork) and ask colleagues to validate your skills.

Recommendations: One or two recommendations can greatly bolster your credibility. Don't hesitate to ask for short testimonials from people you've worked with.

A well-groomed profile on LinkedIn, complemented by solid technical profiles on GitHub or GitLab, **enhances your Professional Personal Brand** and increases your chances of standing out to recruiters and recruiters.

Remember that it is not only enough to have knowledge, but also to know how to show and communicate your experience and skills in a professional, coherent and strategic way.

Why is it important to have a personal brand on LinkedIn?

A well-worked personal brand not only improves your chances of being hired, but also positions you as a benchmark in your field.

Here ‘s why:

1. Increased Visibility

- You will stand out from other similar profiles.
- You will be more easily found by recruiters through keyword searches.

2. Professional Credibility

- A detailed and up-to-date profile conveys confidence.
- Sharing valuable content demonstrates knowledge and authority.

3. More Networking Opportunities

- You connect with people related to your industry.
- Collaborations, invitations to projects or events may arise.

4. Control Your Narrative

- You decide what story to tell and how to differentiate yourself.
- You reflect your values, interests, and career goals.

5. Continuous Professional Development

- You get up-to-date content about your industry.
- You receive feedback, recommendations, and validations from your network.

How to Build Your Personal Brand on LinkedIn Step by Step

1. Professional photo
2. Headline that highlights your skills
3. Clear, focused, and results-oriented summary
4. Well-explained experience with measurable achievements
5. Relevant projects with active links
6. Consistent and consistent engagement
7. Real recommendations and validations
8. Custom URL
9. Strategic use of keywords in your industry

Strategies:

Update skills regularly:

- Add relevant technical skills such as *Java, Spring Boot, Docker, Git, REST APIs*, etc.
- Prioritize key skills that match the roles you're looking for.
- Make sure at least three people validate each skill to increase your authority.

Add certifications:

- It includes relevant courses, accreditations or certifications from platforms.
- Some platforms allow you to automatically sync your achievements with LinkedIn to display visible badges or diplomas.

Actively participate in publications:

- Share progress on your projects, recent learnings, or reflections on trends in your industry.
- Comment and share posts from industry professionals, positioning you as informed and engaged.
- Publish valuable content on a regular basis, even if it is brief: Articles, tips, resources or good practices.

Join professional communities and groups:

- She participates in specialized groups such as *Java Developers, Women in Tech, Data Science Spain*.
- Commenting, answering questions, or starting discussions helps you expand your network and generate new opportunities.

How to improve on LinkedIn to get a job as a programmer or developer?

Offer selection phase and preparation of CV plus cover letter

1. Strategic selection of offers:

- It is not about sending the CV in bulk. Select offers that:
- They align with your current technical competencies.
- Are consistent with your career or professional interest
- Allow for a realistic evolution in your career.

2. Adaptation of the CV and cover letter:

No matter what, they must adapt to each offer. The recruiter notices if it is a generic document.

Useful strategy: Have a draft of the CV and letter that you can modify according to the offer.

On the CV: Change the order or emphasis of certain experiences or skills.

On the menu: Personalize according to the company, its culture, its products, and how you can add value.

3. Type of CV: functional if you are just starting out:

If you have little work experience in the tech sector:

Use a functional CV: Highlight your technical skills.

- Group experiences by projects or competencies, not chronologically.
- Ideal to show your professional transition if you come from another sector.

When you have relevant experience, you can move on to the chronological or combination CV.

4. Cover letter: yes or no?

Advantages of doing it: You can explain things that don't fit on the CV. Motivation, change of sector, interest in the company, attitude. You demonstrate proactivity and personalization.

Disadvantages of not doing it: You may miss the opportunity to differentiate yourself, especially in companies where it is still valued.

Recommendation: If you have something to say that is not on the CV, make it personalized.

5. Key CV data

Includes:

- Professional email.
- Link to updated LinkedIn profile.
- Link to your GitHub or GitLab with featured projects.
- City or time zone if you apply for remote work is useful; if not, you can skip it.

Photo yes or no?

USA/Canada: Do not put a photo, due to anti-bias laws.

Spain/Europe: Optional. You can humanize the profile, but it is not mandatory.

Strategy: Make two versions if you're applying in different regions.

6. Project portfolio: Quality and quantity

Don't put 25 links. Select between ***3 and 5 featured projects***, depending on the offer.

Criteria to choose:

- That they align with the stack that the company is looking for.
- That show progression and level of complexity from simpler to more complete.
- That they are well documented in the README.md explanation, captures, steps, improvements.

Make sure that:

- The code is clean.
- Commits reflect a real development process.
- Projects will be public if you want recruiters to see them. You can keep them private until they're ready.

REFLECTION-CONCLUSION

When submitting your CV, make sure to include direct links to your LinkedIn profile and technical accounts on GitHub, GitLab, or other relevant platforms according to your preferences. This not only makes the recruiter's job easier, but also demonstrates professionalism, transparency, and organization.

A well-cared for digital identity aligned with your personal brand can make the difference between being just another profile or an outstanding candidate.

LinkedIn is much more than a digital resume

It is your professional showcase to the world. Avoiding common mistakes and applying effective strategies will allow you to stand out, transmit confidence and differentiate yourself positively in an increasingly competitive environment.

Your personal brand is not only what you say about yourself, but what others perceive. Therefore, take care of every detail and use LinkedIn as a strategic tool to achieve your professional goals.

LinkedIn has become one of the most powerful platforms for:

- Develop your personal brand.
- Increase your professional visibility.
- Connect with job opportunities.

However, many users do not take advantage of their full potential due to mistakes that can damage their image in the eyes of recruiters and companies.

LinkedIn is not just a network of contacts

It's your public cover letter, your brand, and your professional story told in the first person. **Leveraging each section with intention and strategy will make the difference between going unnoticed or grabbing attention for the opportunities you're looking for.**

Key elements to strengthen your professional profile

Investing time in your LinkedIn profile is investing in your career. From a professional photograph and an attractive headline, to the publication of achievements and the link to your technical repositories, **every action adds up in building your presence as a competent, approachable, and forward-thinking professional**.

An optimized profile is not only a professional showcase, but a strategic tool that communicates:

- Who are you?
- What do you know how to do?
- Where you're headed.

The importance of adapting and personalising your application

A well-selected portfolio, a functional CV and an authentic cover letter position you as someone serious, technical and with a professional vision. The strategy is not only to get the interview, but to demonstrate from the first click that you understand how the sector works and that you are prepared to add value.

Tips for searching for job opportunities and connecting with recruiters

Posting content on LinkedIn improves your visibility and positions your profile as a reference in your industry. Sharing knowledge, reflections and achievements demonstrates leadership, commitment and desire to continue growing.

Having an **active presence goes beyond "being visible"**; it's about building a solid digital reputation that is consistent with your professional profile. By sharing authentic and strategic content, you become a respected voice in your industry.

A **custom URL** shows attention to detail and professionalism. Adjust your profile URL to be easy to remember (example: linkedin.com/in/tu-nombre).

To stand out as a programmer or developer, present yourself as a well-rounded professional, with solid technical skills, tangible achievements, and a proactive attitude.

A well-optimized profile will attract the attention of recruiters and position you as a highly competitive candidate prepared for market opportunities.

Build a Comprehensive Technical Brand

In today's technological world, technical talent alone is not enough. To conquer the job market, you need to build a coherent, visible and strategic personal brand.

LinkedIn positions you professionally with recruiters, leaders, and companies. It's your space to communicate who you are, what you do, and how you've made a real impact. A good presence conveys seriousness, clarity and commitment.

GitHub is the best proof of your technical proficiency. Demonstrate what you know how to build, how you structure solutions, and the quality of your code. Your repositories tell a story about your knowledge, passion, curiosity, and continuous learning.

<u>In short:</u>

- Your LinkedIn should tell your professional story clearly and strategically.
- Your GitHub should demonstrate your technical capability with concrete evidence.
- Together, they project a complete, solid, and reliable image.

Having both profiles aligned makes the difference between being "one more" and being "the ideal candidate". It is the combination of saying "I am capable" and showing "here is the proof".

Techniques for self-assessment, stress management, and sustainable career growth

The technology sector is dynamic, competitive and demanding. To develop professionally, it is not enough to master languages or technical tools; It is also essential to cultivate skills of self-management, emotional well-being and constant reflection.

Key Techniques for Sustainable Personal Development

Objective self-assessment: Identify your strengths, weaknesses, and areas for improvement.

Recommendations:

- Evaluate your technical and soft skills (e.g. Java, Git, testing, communication, teamwork), rating them from 1 to 5 according to your level.
- Review your portfolio and CV every 2 to 3 months. Do your projects reflect progress? Did you incorporate new technologies? Did you improve the documentation?

Professional journaling: Spend 10 minutes a week answering questions such as:

- What did I learn this week?
- What cost me the most?
- How can I improve?
- Am I aligned with my career goals?

Taking care of your personal brand, aligning your digital profiles, and maintaining an attitude of continuous learning and self-management not only improves your professional visibility, but also prepares you to face the challenges and opportunities of the technology sector with confidence and success.

Stress Management in Technology: Mental Health Matters Too

Stress is a constant in technical careers due to tight deadlines, complex problems, and the permanent need for updating. Learning how to manage it is essential to maintain both productivity and personal well-being.

Effective strategies for managing stress:

Pomodoro Technique (5/25): Work for 25 minutes without interruptions and take a 5-minute break. This methodology improves concentration and helps reduce mental exhaustion.

Mindfulness and mindful breathing: Incorporating 5 minutes a day of deep breathing or meditation can decrease anxiety and improve decision-making.

Realistic planning: Break down large tasks into small, manageable steps. Use tools like Trello, Notion, or Jira to organize work according to priorities.

Scheduled digital disconnection: Establish clear boundaries between work or study time and personal time. Disconnection is key to maintaining sustainable performance in the long term.

Professional Development: Planning, Habits, and Approach

A career in technology must be managed with a strategic vision. It is not about "knowing everything", but about building a clear, focused trajectory with tangible impact.

Concrete actions to advance your career:

Define your plan for 6 months, 1 year and 2 years:

- What technologies do I want to master?
- What kind of projects do I want to build?
- What roles do I envision myself in (backend, DevOps, full stack, etc.)?

Directed training: Do not take courses on impulse. Choose programs that bring you closer to your goals and respond to market demands.

Build an aligned portfolio: Focus your projects on highlighting what you want professionally. It integrates current technologies, testing, professional documentation and code quality.

Seek external feedback: Participate in tech communities, seek mentorship, or share your portfolio to receive constructive suggestions.

Update your professional presence: Keep your LinkedIn, GitHub, and CV up to date so that they reflect your actual profile, progress, and goals.

Soft skills development

In the technology sector, soft skills are just as relevant as technical skills. Knowing how to communicate, work in a team, adapt to change and learn autonomously is essential.

Key areas to strengthen:

Effective communication: Learn how to explain technical problems clearly. This is crucial in interviews, presentations, and team collaboration.

Time management: Prioritize important tasks, avoid constant multitasking, and apply methodologies such as GTD or Eisenhower Matrix.

Critical thinking and problem solving: Analyze calmly before looking for solutions. Document errors, investigate causes, and record learnings.

Resilience: Accept mistakes as part of the learning process. Every bug, block, or rejection is an opportunity to grow.

REFLECTION-CONCLUSION

Personal development is a fundamental pillar to achieve success and well-being in the tech sector. Cultivating self-observation, stress management, and strategic planning will make you not only a better professional, but a more resilient, focused, and balanced person.

Investing in yourself is the best project you can build

Success and motivation in the tech sector**:** Staying motivated, overcoming challenges and celebrating achievements are key elements for professional growth in technology.

How to stay motivated and overcome obstacles: The tech industry is dynamic and demanding. For tech professionals, developing a resilient and emotionally intelligent mindset is just as important as technical skills.

Motivation is cyclical: Learn to manage it

You won't always be 100%. Motivation fluctuates due to the context, the difficulty of the projects and personal factors. The key is not to avoid casualties, but to manage them strategically.

Practical techniques to stay motivated:

- Break your goals down into small steps to stay focused and avoid overwhelm.
- Use a technical progress journal to record your weekly learnings and reinforce your sense of progress.
- Apply the 1% rule: get a little better every day. Cumulative growth is more powerful than sporadic big jumps.

How to overcome the most common obstacles?

Frustration, mental fatigue and a feeling of stagnation are frequent. Overcoming these challenges does not depend only on knowledge, but on mental focus.

Promote discipline as a sustainable engine: Motivation comes and goes, discipline keeps you active even on difficult days.

Tips for developing consistency:

- Establish a daily or weekly routine, even if it's just for 45 minutes.
- Use techniques like Pomodoro to stay focused and get proper rest.
- Surround yourself with an enabling environment: networks, mentors, and technical communities.
- It celebrates even small breakthroughs, such as understanding a design pattern or solving a complex bug.

The importance of celebrating achievements

Success is not measured only in jobs or salaries. Every step forward is valuable and celebrating it reinforces motivation and commitment.

Achievements that deserve recognition:

- Completing a course or certification.
- Solve a complex bug.
- Upload an entire project to GitHub.
- Pass a technical interview, even if it has not resulted in an offer.
- Receive positive feedback or a pull request accepted in an open source project.

Create a personal record of accomplishments or a private space to continually recognize and motivate yourself.

Redefining success in technology

Success in tech is not a straight line or just working in large companies or earning six figures. It is moving forward with purpose, acquiring skills, creating useful solutions and maintaining professional values.

Various ways of success:

- Mentor new developers.
- Create an app that solves a real problem.
- Achieve balance between personal and professional life.
- Transition from another industry to technology with your own effort.

Surround yourself with inspiration and support

No one grows alone. The environment profoundly influences your motivation and resilience.

Build your support network:

- Participate in online communities (GitHub, Discord, Slack, Twitter Tech, etc.).
- Connect with people who are where you want to go.
- Follow tech references on social networks.
- Form accountability groups to maintain motivation and commitment.

The path in technology is challenging but rewarding. Success isn't just in the code you write, but in your mindset, your ability to resist and learn from mistakes, and how you keep your passion alive.

Cultivate consistency, learn to manage your motivation, celebrate every step, and never underestimate the power of moving forward with purpose. In technology, success is not only about arriving, but about continuing to grow.

Trainer, junior, semi-senior and senior:

Roles, salary expectations, responsibilities and realities of the professional path.

Introduction

This topic is **recurrent in my group and individual mentoring**, and I believe it is essential to address it clearly in this ebook. Many people approach the tech world **motivated by high salaries**, which is understandable, it is a dynamic, global sector with many opportunities.

However, I have repeatedly encountered a **disconnect between expectations and reality**. People without work experience waiting for senior salaries, without having gone through the path of learning and responsibilities that this implies.

The truth is that in the clear majority of companies, **annual evaluations are carried out**, where the objectives met are reviewed and, if the expected is exceeded, there may be progressive salary increases. Of course, these increases are not automatic, they depend on the **degree of autonomy, responsibility and fulfillment of tasks**.

Meeting deadlines, asking for help judiciously, delivering functional software, taking on new responsibilities... all that counts. So, if you have realistic expectations, this chapter can help you better understand your process. If you don't, I invite you to read this with an open mind and a critical spirit.

The Phases of Professional Development

Phase 1: Trainer

<u>Responsibilities:</u>

Fulfill assigned tasks within established deadlines.

Make sure deliveries are working properly.

He is assigned increasingly complex tasks as he progresses.

<u>Degree of autonomy:</u>

Very low. It needs **constant supervision** from superiors.

<u>It's normal to get stuck and ask for help:</u> It's part of the learning curve.

<u>Years of experience:</u>

From 6 months to 1 year, usually in internships, bootcamps or first personal projects.

Phase 2: Junior

Responsibilities:

Start taking on more complex tasks within projects.

You are expected to be more efficient in your deliveries.

Degree of autonomy:

Moderate. You still need technical support, but **you can make progress on your own** on many tasks.

Consult with superiors when it gets stuck.

Years of experience:

From 1 to 3 years in professional environments.

Phase 3: Semi-Senior

Responsibilities:

Participate in more technical decisions.

Start supervising juniors or trainers.

She combines her own work with mentoring tasks.

Degree of autonomy:

High. It only requires **occasional supervision** from a senior.

Years of experience:

From 3 to 5 years old in real projects and continuously.

Phase 4: Senior

Responsibilities:

Supervision and delivery of all phases of a project.

Meetings with clients, presentation of deliveries, coordination of teams.

Ensures quality in pre-production and production.

He leads and guides all profiles: trainers, juniors and semi-seniors.

Degree of autonomy:

Total. It does not require supervision, on the contrary, **it is the one who supervises.**

Years of experience:

Generally, from 8-10 years in the sector, including technical leadership roles.

Salary expectations vs. Reality

The desire to achieve high salaries is legitimate. However, **expecting senior salaries without having walked the path** reflects an immature and unrealistic attitude.

Bootcamps, intensive master's degrees and courses often fall into **aggressive marketing**, promising quick employability and high salaries with very little experience. This **has created frustrations and disappointments** in my members of my Telegram groups.

In the real world:

Companies **publish salary ranges by levels**.

If you have no work experience, you will be **offered the lowest bracket**, even if you have many courses.

Your salary growth will depend on your **autonomy, professional maturity and real contributions to projects.**

Therefore, it is important that you define your acceptable minimum wage and are clear about it before you start sending CVs or interviews. In a technical interview, if you pass the tests, you will be made an offer. And if you don't have experience, remember: simply receiving an offer is already an achievement.

REFLECTION-CONCLUSION

Not everyone rises at the same rate and comparing yourself makes no sense. There are those who move up quickly, and others who decide to stay in intermediate roles because they prefer less responsibility and a more stable environment.

There are also two main profiles in terms of permanence in companies:

Restless people: They change every few years to look for new challenges and better salaries.

Stable people: They grow within the same company and prefer continuity and comfort.

Both options are valid. The important thing is that you know your personal motivations and stay true to your needs and values.

Over time, your priorities may change: at first you may just want to learn and later on you may seek leadership or better conditions. That's all part of the process.

Salary is a consequence, not a goal in itself. First prove your worth, and then negotiate based on facts, not wishful thinking.

Part II: Technical Learning and Good Practices

Building the foundation, from the essentials to Java microservices

How to move from the fundamentals to more complex and modern architectures with Java

Introduction

Many members of the Telegram groups I mentor want to quickly get to developing microservices, learning Spring Boot, using Docker, and deploying on Kubernetes.

But what they don't always understand is that to get there **you have to master the fundamentals first**, especially in a language like Java, which is robust, object-oriented and widely used in companies with distributed architecture.

This chapter walks you step-by-step through the path **from basics to microservices** using the Java-centric tech stack.

Step 1: Java Fundamentals

Before you think about frameworks like Spring, you should have a good understanding of the Java language itself.

Key Concepts

- Basic syntax: Variables, data types, operators
- Flow Control: If, switch, for, while, etc.
- Methods and functions
- Classes and Objects
- Encapsulation, inheritance, polymorphism
- Exception handling
- Using collections (List, Set, Map, etc.)

Basic tools

- **IDE:** Eclipse, NetBeans, Spring Framework Tools Suite IDE Eclipse or VSC or IntelliJ IDEA Community.
- **JDK:** Java 21 or higher
- **CLI:** Know how to compile and run with javac and java from terminal
- **Git:** Versioned from the start

Learn Java by writing small console programs before jumping into web frameworks.

Step 2: Data Structures and Algorithms in Java

Developing efficient logic is crucial, especially in technical interviews and for writing robust code.

- ✓ Arrays and collections (ArrayList, HashMap, TreeSet, etc.)
- ✓ Recursion and search/sorting algorithms.
- ✓ Complexity analysis (Big O).
- ✓ Use of Java Streams for functional processing.
- ✓ Practice with examples, exercises, and activities.

Step 3: First projects: Monolithic applications with Java

Before you think about splitting services, you need to build **a solid, centralized application**.

Learn the web concepts

- ✓ HTTP protocol and REST fundamentals.
- ✓ Introduction to Servlets and optional JSP, to understand the source.
- ✓ Maven or Gradle as dependency managers.

Learn Spring Boot

Spring Boot is the de facto standard for web applications and microservices with Java. Start from the basics:

- REST Drivers (@RestController)
- Dependency injection (@Autowired)
- Route management (@RequestMapping, @GetMapping, etc.)
- Configuration with application. properties.
- Data persistence with Spring Data JPA and Hibernate
- Use of relational databases with PostgreSQL and MySQL.

<u>Objective:</u> Create a complete CRUD using Spring Boot and a relational database.

Step 4: Best Practices and Design Principles

Once you're comfortable developing Spring Boot applications, start **improving the quality of your code.**

Approaches and principles

Layered architecture: Controller, service, and repository.

- Separation of responsibilities.
- SOLID principles applied to Java.
- Code cleanliness and readability.
- Unit tests with **JUnit** and mocks with **Mockito**
- Correct use of custom exceptions
- A good practice from the start is to document your APIs with **Swagger (OpenAPI).**

Step 5: Communication between RESTful services and APIs

Before you break your application down into microservices, you need to thoroughly understand how modern apps communicate:

- In-depth RESTful design.
- HTTP states and endpoint conventions.
- JSON serialization with **Jackson.**
- Consumption of external APIs using **RestTemplate** or **WebClient.**
- Learn how to test your endpoints with **Postman** and write API contracts.

Step 6: Introduction to Microservices with Java

This is where many wants to start, but **it should be the last step** of a well-worked foundation.

What are microservices?

An architecture where the application is divided into small, independent and separately deployable services. Each one has a clear responsibility and its own database.

Typical Java stack for microservices

Spring Boot: Microservice Foundation.

Spring Cloud: Microservices infrastructure.

Eureka: Service Discovery.

Feign Client: Communication between services.

Spring Gateway: API Gateway.

Hystrix / Resilience4j: Fault tolerance.

RabbitMQ/Kafka: Asynchronous messaging.

Docker: Containers.

Kubernetes: Container orchestration.

Prometheus + Grafana: Monitoring.

ELK Stack (Elasticsearch, Logstash, Kibana: Distributed logs.

Real challenges of microservices

- Operational complexity.
- Asynchronous communication.
- Distributed error management.
- Security between services.
- More complex end-to-end testing.

Microservices are not required for all projects. Sometimes a well-made, monolithic architecture is enough and more efficient.

Learning path with Java

1. Java fundamentals and object-oriented programming.

2. Data structures and algorithms.

3. Spring Boot with monolithic architecture.

4. Good practices + testing.

5. RESTful APIs.

6. Introduction to microservices with Spring Cloud and DevOps (Docker, Kubernetes)

Practical example of evolution

1. **Mini Project 1:** Pure Java console application.

2. **Mini Project 2:** Web CRUD with Spring Boot and Thymeleaf.

3. **Project 3:** REST API with PostgreSQL database.

4. **Project 4:** Layered architecture + testing.

5. **Project 5:** Distributed system with 2 microservices (users and products) + communication with Feign.

6. **Project 6:** Microservices deployed with Docker Compose and monitored with Prometheus.

REFLECTION-CONCLUSION

Don't rush. The Java stack is broad, robust, and highly valued, but its learning curve is progressive

Java + Spring Boot are powerful and prestigious technologies, widely used in the business world, which makes there are many expectations. That's good because there's demand; But it also means that you can feel that the requirements are high. That is why it is advisable to move forward calmly, cementing what you learn, not jumping from topic to topic without consolidating it.

The first thing you need to make sure you master

To build something solid with Java and Spring Boot, here are the basics you should be clear about before jumping into the complex:

- **Master Basic Java and Spring Boot** This includes syntax, data types, exceptions, collections, object-oriented programming. In Spring Boot, know well the automatic configuration, project structure, dependencies, dependency injection, Beans, life cycles, how the application starts, how to handle properties, profiles (dev, prod...), etc.
- **Understand how to build a coherent application** Not only that works, but that is structured in a way that makes sense: separation of layers (controller, service, repository, model), modularity, clarity of responsibilities, good handling of dependencies, that is readable by others, maintainable.
- **Apply good design practices Principles such as** *Single Responsibility*, *Open/Closed*, *DRY,* well-used dependency injection, testing (unitary, integration), proper error handling (exceptions), input validations, logging, minimum documentation of what is important. Also rules of style, naming, conventions that make the code not only functional, but of quality.
- **Know what problems a microservices architecture** solves Because it's not enough to know what it is: you need to understand when it makes sense to use microservices and when it doesn't. Advantages such as independent scalability of parts of the system, resilience against failures, independent deployments, possibility for different teams to work on differentiated services. But also its challenges: complexity of interservice communication, API versioning, network latency, distributed testing, observability, deployment, etc**. (**Atlassian)

What happens when you jump without these foundations?

If you try to move too fast, without having the above firm, the most likely:

- That you copy tutorials without understanding the reason for each technical decision, which generates fragility: when something changes, you won't know how to adapt.
- That you face errors that seem "mysterious" (mishandling of dependencies, incorrect configurations, compatibility problems, security errors) that frustrate you a lot because you won't have the basis to solve them without constant help.
- That your progress is slow because you will be correcting basic errors instead of continuing to progress.
- That you get discouraged when you see that you do not meet all the "technical requirements" that appear in job offers, thinking that you are not ready, when what is often missing is to deepen what you already know, rather than accumulate more superficial things.

How to build solid, scalable, professional projects?

Once you have those foundations, you can start building projects that not only work, but demonstrate professionalism:

- Real projects with meaning: Something that combines several pieces: simple interface or REST endpoints, database persistence, testing, local or cloud deployment, error handling, logging, profiles. For example: a management application, a microservice that communicates with another, that has retry/fallback, etc.
- Do not copy without understanding: When using tutorials, examples or code snippets, always ask "why did you do it this way?", "what alternatives were there?", "what happens if I change this?", "what impact does this decision have on maintenance, performance, scalability?"
- Iterate and refactor: A first version can work, but you will improve a lot if you review your own code, refactor it to make it cleaner, more readable, more decoupled, if you identify "bad practices" that you committed at the beginning and correct them.
- Spring Boot and microservices design best practices: Secure configuration, environment management, API documentation, testing, RESTful endpoint design, validation, version control, monitoring, logging, perhaps use of tools such as Docker, CI/CD, observability tools.

- Patience + constant practice: Dedicate time every day, even if it is little, but with perseverance. Solving exercises, reading documentation, doing tests, facing mistakes, exploring, making mistakes. Learn from mistakes. Have a steady pace rather than a sprint.

Concrete example of how it could be applied

To make it clear, this could be a project plan:

1. Create a simple API with Spring Boot.
2. Connect to a database, perform CRUD operations.
3. Implement validations, error handling, logging, unit testing.
4. Refactor to have well-structured services, repositories, controllers.
5. Add another microservice that consumes this API or communicates via REST or messages.
6. Dockerize each service.
7. Deploy in a lightweight cloud/test environment.
8. Add centralized monitoring or logs.
9. Do load testing or basic performance, see bottlenecks.

Each step introduces something new, but it doesn't take too many risks, and you grow with control.

The challenge of microservices and the reality check:

Common challenges when majoring in Java and how to deal with them.

Introduction

Going from writing code in pure Java, in environments like **Eclipse or NetBeans**, to building **professional microservices using Spring Boot** was one of the biggest challenges of my career.

At first, I thought it would simply be a matter of changing environments or adding libraries, but I soon found myself with a **very steep learning curve**. This change involved **breaking away from many previous ways of working**, adopting new best practices, and confronting tools, concepts, and processes that I was unfamiliar with.

This chapter is a **realistic x-ray** of that transition, with practical reflections on common challenges when specializing and how to overcome them, especially if you come from the academic or self-taught world.

From Pure Java to Spring Boot: The Paradigm Shift

Working with pure Java without frameworks gives you a solid foundation of the language, but developing modern, scalable applications requires adopting more powerful tools like **Spring Boot**.

This is where I discovered the **real complexity of the enterprise Java ecosystem**.

Main changes in the transition:

<u>From simple classes to complex annotations:</u>
Understand what each annotation does (@RestController, @Service, @Autowired, etc.) and how they fit into the Spring architecture.

<u>From main () to organized layers:</u>
Adopt a clean architecture (controller, service, repository), with separation of responsibilities.

From System.out.println() to logs, automated testing, and professional error handling.

Intensive study of tools and dependencies

One of the first challenges was learning how to work with the **Eclipse-based Spring Tool Suite (STS) IDE**, but ready for Spring Boot projects. Added to this was the management of multiple dependencies necessary for a real development environment:

Key dependencies I learned to manage:

- ✓ spring-boot-starter-web: To create REST services.
- ✓ spring-boot-DevTools: Automatic reloading and support in development.
- ✓ spring-boot-starter-data-jpa: Integration with JPA and Hibernate.
- ✓ spring-boot-starter-security: Security and authentication.
- ✓ JDBC drivers for **MySQL** and **PostgreSQL.**
- ✓ **Thymeleaf** to render HTML views.
- ✓ **H2 Database**: Embedded database ideal for testing.
- ✓ **Lombok**: To reduce the boilerplate with annotations.

Each dependency forced me to read documentation, try examples, and above all... **fail many times until they understood how they fit together.**

Technical challenges and how I faced them

During this stage, I faced **technical and organizational challenges** that forced me to professionalize my workflow:

Professional Portfolio & Release Management

I learned how to use **Git and GitHub autonomously**: Create branches, organized commits, merges, and conflict resolution.

I posted entire projects on my profile, **with professional**, structured README.md and with clear screenshots and descriptions.

I started documenting everything in **Markdown**, using lists, titles, tables, and blocks of code.

It automated screenshots of the execution, both backend with Postman and FrontEnd browser/client.

What used to be "just finishing the code" now included: documentation, version control, testing, and professional presentation of the project.

Validating Functionality with Postman

I learned how to use Postman consistently to:

- ✓ Test GET, POST, PUT, DELETE endpoints
- ✓ Verify HTTP Codes and Responses
- ✓ Automate basic tests in collections
- ✓ Simulate input errors and handle expected responses

Test automation and static analysis

Professionalizing the code also involved **testing it automatically**:

JUnit 5 for Unit Testing: Test Drivers, Services, and Logic.

Mockito to simulate dependencies in tests.

Sonar Lint (STS plugin) for static code analysis: detect smells, common errors and readability problems.

Each failed test was an opportunity to better understand the internal logic of the project.

Collaborative tools

To prepare for enterprise environments:

I learned how to use **Jira** as a tool for task tracking, agile management, and documentation of bugs or improvements.

I used **GitHub Projects** as an alternative to manage Kanban boards when I was working alone.

Lessons learned

- ✓ **Don't underestimate the paradigm shift** when moving from pure Java to Spring Boot + microservices.
- ✓ **Learn by doing:** Theory is important but **applying it to a real project is what consolidates knowledge.**
- ✓ **Making mistakes is necessary** but documenting them and reflecting on them is what makes you grow.
- ✓ Your GitHub **shouldn't just have code**, it should reflect the way you work: order, cleanliness, explanations, professionalism.

Are you ready for microservices?

Many dreams of working with **microservices**, but before that, it's critical:

- ✓ Master Spring Boot and its ecosystem well.
- ✓ Learn to think about independent, reusable components.
- ✓ Adopt modern tools: Git, Postman, Docker, automated testing.
- ✓ Accept that **it is a process of months, not weeks.**

The "reality check" was understanding that it is not enough for the code to work: **it must be clean, tested, documented, deployable and collaborative.**

Essential tools for everyday life

Editors, terminal, version control, containers, CI/CD, and more.

Introduction

Knowing how to code is important, but in the professional environment, what really makes the difference is **how you work**. Can you keep your code clean? Control versions in an orderly way? Automate deployments? Collaborate without depending on others?

These questions are answered not only with technical knowledge, but with the **mastery of tools** that allow you to write, test, deploy and maintain software in a professional way.

This chapter covers the fundamental tools you'll need **every day** as a modern Java developer, especially if you're working or planning to work with **Spring Boot and microservices**.

1. Publishers and IDEs

If you're working with Java, you need a solid IDE.

Spring Tool Suite (STS) is an Eclipse-based IDE, specifically prepared to work with Spring Boot. It's a great option if you're just starting out with microservices.

On the other hand, **IntelliJ IDEA** (Community or Ultimate) offers a modern and more agile experience, with advanced integration for Spring, testing, databases, and Docker.

It is also advisable to familiarize yourself with lightweight editors such as **Visual Studio Code**, ideal for editing configuration files, documentation, and working with complementary projects such as the FrontEnd or APIs.

Learning how to move comfortably between these environments gives you professional flexibility.

2. Terminal: Your best ally

The command line (CLI) is indispensable in the professional environment. It allows you to quickly move through projects, launch builds, run tests, lift containers, and more.

Mastering basic commands such as cd, ls, cat, grep, rm, mkdir, and nano or vim is essential. But you should also familiarize yourself with more advanced tools such as curl, docker, or git from terminal.

The more comfortable you feel in the terminal, the **more efficient and autonomous you will be**.

3. Version control: Git and GitHub

Git is the foundation of any collaborative workflow. It's the most widely used version control tool, and you need to master the basics: cloning repositories, creating branches, making organized commits, merging branches, and resolving conflicts.

GitHub, on the other hand, is more than code hosting: it's your professional storefront. Here you can build your portfolio, collaborate with others, create issues, document your projects, and automate processes with GitHub Actions.

Keeping your repositories tidy and well-documented is a professional practice that sets you apart.

4. Dependency Management and Construction

In Java projects, it is essential to know how to use construction tools such as **Maven** or **Gradle**. These allow you to declare and manage external libraries, build your application, launch tests, and package your project.

Maven is widely adopted in traditional Java environments. Learning how to work with the pom.xml file, running commands like mvn clean install or mvn spring-boot: run, and customizing configurations is part of the daily job.

Gradle is more modern and is frequently used in Android projects or when you are looking for greater flexibility.

5. Databases and management tools

Microservices often interact with databases. You must know how to use relational engines such as **MySQL** or **PostgreSQL**, as well as embedded tools such as **H2 Database**, which are very useful in development and testing environments.

To manage and consult your databases, visual tools such as **DBeaver**, **MySQL Workbench** or **PgAdmin** are your allies. These allow you to review tables, run SQL queries, review relationships, and diagnose problems.

Understanding how these foundations integrate with **Spring Data JPA** is essential to developing effective backend applications.

6. Containers: Docker

Docker has transformed the way applications are developed, tested, and deployed. In the context of microservices, it's practically mandatory.

With Docker, you can package your Java application and its environment into one image and run it on any machine that has Docker installed. This facilitates testing, deployments, and collaboration across teams.

Learning how to write a Dockerfile, build images, run containers, and use **Docker Compose** to build multiple services as an application + database is a fundamental step in your professional maturity.

7. Automation and CI/CD

CI/CD (Continuous Integration and Continuous Deployment) is a set of practices that allow you to automate the build, testing, and deployment of your software.

Tools like **GitHub Actions**, **GitLab CI,** Jenkins**,** or **Bitbucket Pipelines** allow you to define automatic flows that are triggered every time you push to the repository.

While it may seem complex at first, starting with basic flows like compiling, running tests, and uploading artifacts to GitHub is easier than it sounds. Over time, you'll be able to automate the entire project lifecycle.

8. Code quality and testing

An essential part of professional work is ensuring that the code **works properly** and is **maintainable**.

To do this, you need to master testing tools such as **JUnit** for unit testing, and **Mockito** for testing with simulated objects. It's also helpful to learn how to use assertion frameworks like AssertJ, which allow you to write more readable tests.

In addition, you can improve code quality by using static analysis with **SonarLint** (on-premises, within the IDE) or **SonarQube** (in team or enterprise). These tools detect bad practices, duplications, readability issues, and vulnerabilities.

9. Communication and task management

Beyond the code, in the professional environment it is key to organize and communicate clearly.

Tools like **Jira** help you manage tasks, bugs, and project plans. Lighter ones like **Trello** or **GitHub Projects** are also useful if you're working alone or in small teams.

For communication, tools such as **Slack**, **Discord**, **Microsoft Teams**, or **Google Meet** are standard in most companies. Knowing how to explain your progress, difficulties or ideas in these environments is as important as programming.

In addition, tools like **Loom** are very useful for recording presentations or walkthroughs of your projects and sharing them with your peers or mentors.

REFLECTION-CONCLUSION

Mastering those tools is no longer an "extra", it is essential if you want to grow as a professional developer. Here I will develop it in more detail, because understanding *why* they are so important helps you to internalize them and use them well.

Why are these tools essential?

1. **Consistency across environments** One of the biggest sources of errors comes from running code on "my machine" and something going wrong when moving it to another environment (test, staging, production). Docker, for example, allows you to package the application with all its dependencies, so that what works locally works the same in production**.** **(**DEV Community)

2. **Automation that saves time and reduces human error** Repetitive things like compiling, running tests, deploying, checking that nothing broke can be automated. CI/CD (Continuous Integration / Continuous Delivery / Continuous Deployment) are practices that allow each code change to go through a pipeline where builds, tests, validations, deployments are executed. That means large errors are caught early, not after days of work**.** **(**Medium)

3. **Version Control and Collaboration (Git, Dependency Control)**
   Working alone/with a team involves handling branches, merges, version conflicts, dependency on external libraries, internal libraries, etc. Git + Maven/Gradle (or other dependency tool) are the foundation for collaborating well, tracking changes, going back if something goes wrong, making integrations without chaos. Atlassian explains it well: Use Git + Maven for dependencies, clear versioning, integration between teams. (Atlassian)

4. **Facilitate repeatable deployments, reliable testing, and maintenance** When you use tools like Docker to isolate environments, CI/CD to automate testing/deployments, automated testing tools, you make your application not just something that "runs," but something maintainable. You can do repeated testing, integrations on small changes, and ensure that each new version doesn't break what already works. That gives confidence to the team and the client**.** **(**Security Compass)

5. **Scalability and adaptability** In a small project, little infrastructure, little automation may suffice; but in real projects, with multiple microservices, many environments, variable traffic, unexpected failures, you need your technical stack to allow you to scale — both in code and operations. Tools like Docker, CI/CD environments, proper dependency handling, terminal scripting, are what give you that ability to adapt when the project grows**.** **(**DEV Community)

How to incorporate these tools little by little so as not to overwhelm you?

To avoid feeling like you have to master everything at once, here are some strategies that can help you:

- Start with the essentials: Git + version control + dependency management (Maven / Gradle). Make every project of yours use Git correctly: clear commits, branches for features, pull/merge requests, even if it's personal.
- Master Spring Boot well, so you can assemble a project, configure profiles, properties, exceptions, unit tests. Have confidence that with Spring Boot you can go fast and clean.
- Then it adds Docker: First for on-premises environments. Learn how to Dockerize your main application, database, whatever you use, and make sure it works the same in container.
- Once Docker is mastered, integrate CI/CD: Use a tool (GitHub Actions, GitLab CI, Jenkins, etc.) to automate at least: build, test, deploy to test/staging. Let each commit go through automatic tests.
- Then move on to more advanced testing: Integration testing, contract testing, code review, coverage, maybe light load testing.
- Finally, it also works on the culture of the team: Good documentation practices, code standards, communication about versions, dependencies, deployments, errors.

What distinguishes someone "who knows how to code" from a true software professional

When it's no longer just about writing code that does "what the exercise says," but about building robust, maintainable, sustainable, scalable software, that's where you notice the difference. Some signs:

- In your profile or projects, you can show CI/CD pipelines, rollouts that can be reproduced, containers, automated tests, good code organization.
- That you can explain technical decisions: "Why did I use Docker?", "Why did I set up this pipeline?", "What tradeoffs did I make when choosing this dependency or design pattern?"
- That you can handle real production/dev errors: different environments, dependencies that fail, versions that break things, rollback, monitoring.
- That you deliver continuous value: not working alone until something is "perfect", but iterating, correcting, improving, responding to feedback, change, new requirements.

Good practices from the beginning

Clean Code, Basic Testing, Documentation, and SOLID Principles for Beginners with a Focus on Java

Good practices from the beginning

When you're starting out, your priority is usually "make the code work." But if you want to work in professional settings, that's **not enough**. Code that works, but is difficult to understand, maintain, test, or scale, is useless.

This chapter has a clear goal: to help you adopt **good practices from the beginning**, so that your code is clear, clean, and professional. If you apply them now, you will save yourself a lot of problems in the future and facilitate teamwork.

1. Clean Code: Write as if someone is going to read your code, because they will.

The concept of **Clean Code** was popularized by Robert C. Martin ("Uncle Bob"). Its essence: to write **clear, readable, understandable and maintainable** code.

Clean Code Key Principles:

Clear, descriptive names: Use names of variables, functions, and classes that express exactly what they do. Avoid confusing abbreviations like tmp, a, b, doStuff.

- Bad example: int x = 3;
- Good example: int maxRetries = 3;

Short, single-purpose functions: Each function must do **only one thing** and do it well.

Avoid duplication: If you repeat logic, turn it into a function.

Avoid unnecessary comments: Well-written code **explains itself**. Use comments only to clarify complex decisions.

Format your code consistently: Use indents, spaces, and line breaks to make it readable.

Activate a plugin like **CheckStyle** or **Prettier** and set your IDE to make code formatting automatic.

2. Basic Java Testing: Test Before You Trust

Testing is not an extra, it is a tool to know that your code **works today and will continue to work tomorrow**.

Why write tests?

- You avoid mistakes when modifying the code.
- You can refactor with confidence.
- It forces you to write modular and understandable code.
- You gain credibility as a developer.

Types of basic tests you should know:

Unit Tests

They are the simplest and most important. They evaluate individual functions or methods in isolation.

We use **JUnit** to write them.

Integration test later

They evaluate how different parts of the system interact with each other, for example, your API and database. You don't need them at first, but it's good to know that they exist.

Tools for testing in Java:

- **JUnit 5**: Current standard for unit tests.
- **Mockito**: To simulate dependencies (mock objects).
- **AssertJ**: To make the statements more readable.
- **Spring Boot Test**: Built-in testing framework for Spring applications.

Start with JUnit 5. You only need that for the first few months.

3. Documentation: README, Helpful Comments, and Swagger

README.md

Each project you upload to GitHub should have a clear README.md that includes:

- Project description
- How to run it (step by step)
- Technologies used
- Screenshots
- Useful links

If you're using Spring Boot, you can include how to start the project (mvn spring-boot: run) and what API paths are available.

Don't overdo comments, but use it when:

- There is a complex logic that is not obvious.
- You want to explain a technical decision.
- You leave an ALL to implement later (use // EVERYTHING:).

Use Swagger if you are working with APIs.

With Spring Boot, you can use **Springdoc OpenAPI** or **Swagger UI** to generate interactive documentation of your endpoints.

By installing the dependency on your pom.xml, you can view and test your in-browser API (/swagger-ui.html).

4. SOLID Principles Beginner's Version

The SOLID Principles are five best practices for writing maintainable, scalable, and flexible object-oriented code.

Here they are explained simply:

1. S – Single Responsibility Principle (SRP)

A class must have **only one responsibility**.
Example: Don't mix business logic with database access in the same class.

2. O – Open/Closed Principle (OCP)

The code must be **open to extension, closed to modification**.
Example: If you need to change the behavior of a class, it is better to extend it (inheritance or interfaces), rather than modify it directly.

3. L – Liskov Substitution Principle (LSP)

A subclass must be able to replace its parent class **without breaking code**.
Example: If you have a Vehicle class, a Car class that extends it shouldn't change its basic rules.

4. I – Interface Segregation Principle (ISP)

Don't force a class to implement methods it **doesn't need**.
Example: It is better to have several small interfaces than one very large one.

5. D – Dependency Investment Principle (DIP)

Classes should rely on **abstractions, not concrete implementations**. Example: Use interfaces to inject dependencies. Spring Boot does this automatically with @Autowired.

How do you address technical issues when you don't know if the bug is in the code, database, or server?

Gradual learning not only streamlines your process as a programmer, but also prepares you to face challenges with confidence. One of the most frustrating moments in software development happens when something goes wrong... and you don't know exactly where the problem is.

Is it a bug in your code? Is the database failing? Or maybe it's a server misconfiguration?

Here is a **systematic guide** to identify and resolve this type of situation in a professional, orderly and effective way.

1. Break the problem down into smaller parts

Isolate components: Start by identifying whether the problem is on the frontend, backend, database, or server. Questions such as *"Is the data not reaching the customer?"*, *is there a server connection error?* or *does the logic of the program break?* They'll help you narrow down the range.

Test specific units: Run individual tests on functions or modules to verify that each part works separately before analyzing them together.

2. Check for error messages

Server errors: Query server logs (Apache, Nginx, Node.js, etc.). These files contain key information about network errors, permissions, paths, memory, etc.

Code errors: Use the debugger to track code execution line by line. It identifies the exact location of the fault and analyzes what variables or conditions trigger it.

Database errors: Verify that SQL queries are well-structured, that credentials are correct, and that there are no access or permission issues.

3. Review the code step by step

Detailed inspection: Even the smallest errors can lead to serious failures. Carefully review conditions, data types, and expected values.

Use of strategic logs or print (): Prints key values in the program flow to see how far it runs successfully and what fails next.

4. Check the database

Successful query: Make sure that SQL statements run successfully and return the expected results.

Active Connection: Review connection status, host configuration, port, credentials, and permissions.

Data integrity: Validates that the stored or retrieved data is correct and in the proper format.

5. Check the server and infrastructure

Connectivity: Use tools like ping, telnet, or curl to verify that the server is accessible from your local machine.

Server resources: Make sure that the server has enough RAM, CPU, and disk space. Also check for runtime limits or crashed processes.

Key configurations: Errors in files, env, environment configurations, or firewalls can also cause invisible failures.

6. Check the input and output data

Controlled inputs: Be sure to validate data entering your app from forms, APIs, or other sources.

Server responses: Tools such as Postman, Insomnia or curl allow you to analyze headers, HTTP codes and content to understand how your system responds.

7. Use debugging and monitoring tools

Debuggers built into IDEs: Eclipse, IntelliJ, VS Code, NetBeans, and others offer visual tools for inspecting variables, function execution, and breakpoints.

Performance monitoring: Solutions such as New Relic, Datadog or Prometheus allow you to detect bottlenecks, performance errors and system failures in real time.

Database management tools: Use interfaces such as PgAdmin, MySQL Workbench, or DBeaver to explore, analyze, and optimize queries.

8. Check out the community and documentation

Active communities: Forums such as Stack Overflow, GitHub, Reddit, and groups on Telegram or Discord are useful for searching for common bugs and shared solutions.

Official documentation: Always refer to the documentation of the frameworks or tools you are using first. It is the most accurate and reliable source.

9. Take a strategic break

Mental distance: If you haven't solved a problem for hours, disconnecting for a moment can help you see it from another perspective.

Come back clearly: Often, by taking up the problem with rested eyes, the solutions become more apparent.

10. Ask for help when needed

Explain clearly: If you decide to ask for help, be specific. State the error, what you expect, what you did, and what tests you performed.

Tap into other points of view: Sometimes, a person with different experience can spot the problem faster or suggest an approach you hadn't considered.

When you don't know if the problem is with the code, the database, or the server, the most important thing is **to stay calm** and apply a neat approach. Isolating the problem, reviewing logs, validating data, and relying on tools and community will help you resolve errors effectively and professionally.

Instead of despairing, **think like a problem solver**, every mistake is an opportunity to learn, strengthen your skills, and become a more competent, autonomous, and resilient developer.

REFLECTION-CONCLUSION

Applying good practices from the beginning **will not slow you down**, on the contrary, **it will avoid errors, bugs and frustrations**. In addition, it will prepare you to work in collaborative environments, where code quality and clarity are key.

Anyone can write code that works. A professional writes code that others can understand, maintain, and scale. And above all, don't seek perfection, look for consistent progress.

Start with the basics:

- Use clear names.
- Write small functions.
- Take your first tests with JUnit.
- Document your projects with Markdown.
- Know and apply SOLID principles in your designs.

Learning by Building: From Concept to README

How to organize personal projects and boost your portfolio.

Why build your own projects?

Learning programming doesn't just mean understanding concepts but applying them. The best way to consolidate technical knowledge is by building. And when you build, you're not just learning: you're also creating real evidence of your skills.

A well-focused personal project helps you to:

- Consolidate what has been learned with a practical application.
- Show your ability to solve real problems.
- Practice tools and flows that are used in companies.
- Enhance your professional profile with a visible and well-documented portfolio.

1. How to choose a suitable personal project

You don't need an innovative idea. A good personal project meets three conditions:

- It is aligned with your current level.
- It allows you to apply the technologies you are learning.
- It makes practical sense or solves a simple problem.

It is recommended that the project has a minimum structure, e.g. CRUD, authentication, external API consumption, etc. and is focused on a domain that interests you: Task management, recipes, products, users, among others.

2. Minimal organization from the start

Organizing a project well from the beginning helps you keep it organized, facilitate teamwork, even if you are alone for now, and apply good practices. Define from the beginning:

- The structure of folders and packages.
- The separation of responsibilities (controllers, services, repositories, entities).
- The runtime environment and the required configurations (for example, configuration file for the database).
- A simple planning of functionalities and deliverables.

Creating a private or public repository on GitHub from scratch is a great practice. It allows you to version your work, practice Git, and have professional visibility.

3. Plan for the essentials

Before scheduling, clearly define:

- What minimum functionality the project must have to be considered complete.
- What technologies you are going to use.
- How you're going to organize deliverables: basic features first, improvements later.
- What quality criteria are you going to apply (testing, documentation, validations, etc.).

You don't need to have everything figured out from the beginning, but you do need a clear roadmap that allows you to move forward in an orderly manner.

4. Create, test and improve

Building means iterating. Start with a basic, functional version. Don't try to make it perfect at first.

Make sure to:

- Perform tests with tools such as Postman or from a web client.
- Validate that the core functionalities are complete and working correctly.
- Make frequent and descriptive commits.
- Document what you do as you move forward, not at the end.

Continuous improvement is also part of the process: You'll be able to come back to your project weeks later and add new features or apply best practices that you didn't know about when you started.

5. The value of a good README

A project without a README is like a product without a label. No one will know what it does, how it works, or why it exists.

Your README.md should include:

- A clear description of the project.
- The technologies used.
- Instructions for executing it.
- A short list of features.
- Relevant links (if any).
- A section of future improvements or pending features.

This file is often the first thing recruiters or technical reviewers see. Take care of it as much as you take care of your code.

6. How to Feature Your Projects on GitHub

To make a project shine in your portfolio:

- It has to be complete and functional.
- It should reflect good practices, clean structure, understandable code, basic testing, and clear documentation.
- If you can, add screenshots, a short video, or a unfolded demo.
- Show progress: If you've been committing for several weeks, that reflects consistency.

You don't need to have dozens of repositories. A handful of well-worked, varied and complete projects is enough to stand out.

7. Selecting projects for the portfolio

Not everything you do should be in your public portfolio. Choose those projects that:

- They clearly reflect your knowledge and work style.
- They use tools that are aligned with the type of job you're looking for.
- They are well explained and organized.
- They have a useful or interesting logic, even if it is simple.

Quality always counts more than quantity.

REFLECTION-CONCLUSION

Building is the best way to learn

There is no better way to consolidate knowledge than to put it into practice. Building not only improves your technique as a programmer, but also strengthens essential transversal skills. Your ability to **communicate ideas**, **plan tasks**, **solve problems**, and **present your work clearly and professionally**.

A personal project doesn't have to be perfect to be valuable. In fact, the most authentic projects often have bugs, iterations, and constant improvements. The important thing is that they are **well thought out**, **well executed** and **well documented**.

That's what really shows who you are as a developer: what you know how to do, how you approach challenges, and how you evolve with each line of code.

In a world saturated with courses, certificates, and degrees, your **portfolio** becomes tangible **proof** of your professional value. It's your most powerful letter of introduction. It not only demonstrates your technical skills, but also your initiative, your work style, and your ability to learn autonomously.

Every project you build:

- Actively reinforce your learning.
- It allows you to apply concepts in real contexts.
- It gives you material to share in interviews, professional networks and tech communities.
- It helps you discover your interests and strengths within development.

In addition, your portfolio can open doors that you can't even imagine. Sometimes, a well-documented project on GitHub, accompanied by a clear explanation on LinkedIn or on your blog, can capture the attention of recruiters, mentors, or collaborators. Don't underestimate the power of showcasing your work.

Take advantage of it.
Build, document, share. Every commit counts. Every bug fixed adds up. Each project is one more step in your professional path.

Part III: Personal and Professional Development

Programming is not just about writing code, soft skills:

Communication, feedback, leadership and teamwork.

Programming is not just writing code

When you think of a programmer, the typical image is that of someone in front of a screen, writing lines of code for hours. However, success in the world of software development does not depend solely on technical skill. Soft skills, those interpersonal and emotional skills, are just as important, and often determine whether a developer excels, collaborates effectively, or succeeds in leading teams.

Below, we explore some of the most relevant soft skills for the personal and professional growth of any developer.

1. Clear and effective communication

Communication is one of the most critical skills for any software professional. It's not just about explaining a function or presenting a bug, but about being able to convey complex ideas in a clear and understandable way to technical and non-technical audiences.

<u>Good practices:</u>

Active listening: Understand before responding.

Clear written communication: Document code, write tickets, and reports that others can understand.

Technical presentations: Know how to present progress, problems or design decisions in front of the team or stakeholders.

2. Feedback: Give and receive it

Feedback is an essential component of continuous learning. In software development, teams conduct code reviews, retrospectives, and evaluation meetings, where the exchange of opinions can make the difference between stagnation and improvement.

<u>Give effective feedback:</u>

- Be specific, not general.
- Focus on the behavior or action, not the person.
- Use a constructive approach: how can it be improved?

<u>Receive feedback with maturity:</u>

- Don't take it as a personal attack.
- Be thankful and reflect.

- Apply what you learn to improve your work.

3. Technical and human leadership

You don't need to have a "team leader" title to exercise leadership. Technical leadership involves making sound decisions, guiding others, and fostering a culture of quality and constant improvement.

Characteristics of a good technology leader:

Mentoring: Supporting developers with less experience.

Technical vision: Understand where the product or architecture should go.

Empathy and service: Leading from understanding and support, not from authority.

4. Teamwork

Modern software is built as a team. Whether you're a startup or a large corporation, knowing how to collaborate, negotiate ideas, and resolve conflicts is critical to the success of any project.

Keys to teamwork:

Shared responsibility: Achievements and mistakes are shared.

Goal alignment: Everyone works toward a common goal.

Respect and diversity: Valuing different perspectives strengthens the group.

5. Self-knowledge and emotional management

Being aware of your emotions and reactions to stress, mistakes or frustration will help you grow as a professional and as a person. Software development is a field that requires resilience and emotional balance.

Useful tools:

Mindfulness and active pause.

Time management techniques and priorities.

Periodic self-evaluation: What are you doing well? What could you improve?

Towards a well-rounded professional

I'd like you to imagine a programming path not only filled with logic and lines of code, but also with humanity, connection, and real growth.

Because you deserve to be someone who, in addition to writing clean code, inspires confidence, builds bridges with others, grows without losing your essence.

When I think of a well-rounded professional, I think of someone who knows how to solve bugs, yes, but also knows how to listen, communicate, share, adapt and lead with empathy. It's not something extra, it's part of what makes your work matter, that the team advances, that your journey makes sense.

Why Soft Skills Matter More Than Ever Today

In Spain, it is becoming clear that it is not enough to have good technical knowledge. Recent studies show that many companies in the technology sector highly value communication, adaptability, the ability to work in a team, and think analytically. People who can express themselves well, who understand what others want, who have initiative, make a difference. (AHK Talent)

This isn't just to "be nice," it has concrete consequences. Those who have good soft skills have more opportunities, better interviews, fewer misunderstandings, less stress, better relationships with colleagues and bosses, and many times greater stability.

Because when a project goes wrong, good code helps, but what saves the day is communication, empathy, creativity to look for solutions together.

What soft skills to cultivate, with love and realism

Here are some skills I've seen make a difference, and how you can start cultivating them without having to reinvent anything.

- **Authentic communication** That your ideas are understood without complications, that when you talk to someone who does not know about code, you do not explain yourself in gibberish but in simple examples.

  Know how to listen: Not only listen to what others say, but understand what they need, what they feel. Asking when you don't understand. Small gestures like making sure everyone feels included in a technical conversation, for example.

- **Empathy and emotional intelligence**
  Recognize that we all have good days and difficult days. That a teammate who is frustrated with a bug is not "incapable", he simply needs support. That a criticism is an opportunity to improve, not a sentence of incapacity. Learn to manage your emotions, not to get exhausted, to ask for help when you need it.

- **Adaptability** In this technological world, everything changes: frameworks, tools, work models and even methodologies. Adapting does not mean losing identity but continuing to learn, experiment, accepting that sometimes what you knew yesterday is no longer fully useful today. Be curious, try new things, take small risks.
- **Teamwork** Collaborate with respect, recognize the value of other profiles (design, product, marketing). Contribute when you can, ask for help when you need it. Share your questions. Share your achievements. Make the team's success as important as your lines of code.
- **Leadership from the everyday** You don't need a boss title to lead: you can lead by example (be punctual, well organized), you can lead by taking initiative (propose improvements, review code, help others), you can lead by recognizing your own mistakes, showing humility. That builds trust.
- **Time management and priorities** Having so many tasks and so many distractions that it's easy to get lost.

  Small habits: Making realistic lists, concentration blocks, prioritizing what brings the most value, learning to say "no" when something is not urgent. Make your pace sustainable.
- **Resilience** Because there will be difficult days: bugs that don't work, tight deadlines, hard feedback, own mistakes. It is not about ignoring tiredness or frustration, but about learning to get up, reflect, solve what you learned, move forward.

How to go step by step, without pressure

I want you to propose it with love, without stress, as if you were planting seeds that will grow little by little:

1. Choose one or two soft skills that you feel could give you more peace, more clarity, or more connection. It can be communicating better, managing your stress better, listening more. Start with that.
2. Observe your days: When something went well, what influenced. When something went wrong, what could have been different if you had spoken differently, asked for help, prioritized.
3. Look for small opportunities: In a meeting, in reviewing code, in explaining something to someone, in giving kind feedback. Every time you do it, you're training that skill.
4. Ask for feedback: Ask a colleague, your boss, whom you trust: "How do you think I communicate in projects?", "Do you think I adapt when circumstances change?", "Did my explanation seem clear to you?". Listening to the other is a powerful tool.

5. Celebrate what you advance, no matter how small. If you used to have a hard time speaking in meetings, and now you do, that's already a great achievement. If you managed a deadline better, if you resolved a conflict calmly, if you helped a colleague... Celebrate.

REFLECTION-CONCLUSION

I want you to always remember this, you are not alone on this path. There is no human being who was born with all the skills already polished; No one is born a perfect leader, or an infallible communicator, or with total control of their emotions.

All of us, absolutely everyone, start from somewhere of uncertainty, of error, of trial and error. And that's okay.

Growing up doesn't mean avoiding falls but learning what to do after every stumble.

Being a well-rounded professional is not about achieving perfection, but about awakening the awareness of who you are here and now, about committing to yourself, about putting in constant effort, about having good intentions, even making mistakes, but moving forward. It is giving yourself permission to feel, to doubt, to err, but also to get up with more clarity.

It is accepting that the road is long, sometimes dark, sometimes slow, but that every step count. That coherence between what you do and who you are is what ultimately remains, what elevates you, what makes you look back with gratitude instead of regret.

Your path as a programmer can be brilliant not only when your applications work without errors, not only when you get visible recognition, but, above all, when at dusk you look at what you have built and feel proud.

Because of your code, but also because of how you have written it, because of the decisions you made, because of what you put aside to maintain your values, because of what you learned in difficult times, because of how you chose to collaborate, respect, listen. Because true satisfaction is not only technical, it is human.

Also think about the relationships you've formed: that colleague you helped, even though you were also busy, that question you asked, even though you were embarrassed, that feedback you gave with empathy, that time you shared what you knew without expecting something in return. All that adds up. And over time, those small actions build reputation, confidence, that feeling that you are doing more than just programming, you are building who you are as a person, as a professional with values. And that, believe me, makes all the difference. *Not only in what you deliver, but in how you feel about delivering it. Not only in what others see, but in what you know you can achieve, even with mistakes, even on difficult days.*

Mental health in tech

Burnout, anxiety and strategies to maintain balance.

The tech industry offers exciting opportunities. Constant innovation, good salaries and the possibility of impacting millions of lives with a single line of code. But it also presents challenges that are rarely approached with the same seriousness: constant pressure, tight deadlines, last-minute changes, and a culture that often glorifies overexertion.

In this chapter, we explore the most common mental risks in the tech sector, such as burnout and anxiety. We share practical strategies for cultivating sustainable mental well-being.

1. What is Burnout?

Burnout, or professional burnout syndrome, is a state of physical, emotional, and mental fatigue caused by prolonged exposure to work-related stress. In the technological environment, this phenomenon is especially common due to factors such as:

- Long days without real disconnection.
- Constant changes in requirements or technologies.
- Excessive self-employment or poorly distributed work.
- High pressure for fast results.

<u>Symptoms of burnout:</u>

- Chronic tiredness, even after resting.
- Loss of motivation and cynicism towards work.
- Difficulty concentrating.
- Social and emotional isolation.
- Low productivity, even if effort increases.

2. Anxiety in the programmer's life

Anxiety in tech can be subtle and progressive. It can present itself as a constant fear of not being "up to the task", the feeling that "you are going to be discovered" as incompetent (imposter syndrome), or simply a daily tension when checking emails, attending meetings or delivering code.

<u>Common causes:</u>

- High exposure to code reviews.
- Unrealistic or ambiguous expectations.
- Lack of control over times and priorities.
- Isolation due to remote work or individualistic culture.

3. Strategies for maintaining mental balance

Preventing and managing anxiety or burnout doesn't always mean quitting your job or changing careers. It's about incorporating healthy practices and boundaries that protect your mental health.

A. Set clear boundaries

- Define your working hours and respect them.
- Learn to say "no" to tasks that exceed your capacity.
- Physically and mentally separate your workspace and rest.

B. Digital disconnection

- Take real breaks without devices.
- Avoid checking messages or emails after hours.
- Schedule "downtime" where you are not available.

C. Taking care of the body to take care of the mind

- Get 7 to 9 hours of sleep a night.
- Exercise, even if it's just 20 minutes a day.
- Eat a balanced diet and drink plenty of water.

D. Open Communication

- Talk to your team or leader if you're feeling overwhelmed.
- Share your boundaries and needs honestly.
- Promotes a culture of empathy and mutual support.

E. Professional support

- Don't be afraid to seek psychological help if you need it.
- Some companies offer employee assistance programs (EAPs).
- Therapy is not a sign of weakness, but a tool for growth.

4. Build a more human tech culture

Not everything depends on the individual. Organizations must also take responsibility for the mental health of their teams. A Health tech culture promotes:

- Realistic workload management.
- Leaders who prioritize the well-being of the team.
- Flexibility and autonomy.
- Recognition without exploitation.

Sustained productivity is born of balance, not wear and tear.

5. The Path to a Healthy Career

Succeed without getting lost in the process

Many people enter the tech world motivated by passion, curiosity, or a desire to solve complex problems. But somewhere along the way, that motivation can start to blur.

The pressure to perform, the need to always be up to date, the constant comparison in networks or communities, and a culture that often romanticizes overwork, can make you disconnect from yourself.

Success ≠ Burnout

A healthy career is not linear, nor perfect. It is not built on endless marathons of work, sleepless nights or endless coffees.

It is built with conscious decisions, prioritizing what is important, recognizing your limits and respecting them. Real success, the one that lasts, is the one you can sustain without breaking.

It's okay to slow down

The industry moves fast, but you're not a machine. Not all learning has to be immediate. Not all challenges have to be taken on. Sometimes the greatest act of professionalism is knowing when to stop, breathe, and rearrange your path.

Slowing down is not giving up. It's a way to regain perspective, to reconnect with what really matters to you. And many times, after a conscious pause, you come back clearer, more creative, and more present.

It's okay to ask for help

Many developers feel that they must solve everything on their own: the bug, the pressure, the stress, life. But asking for help is not a sign of weakness, but a sign of maturity. Talking to a colleague, expressing your boundaries to the team, going to therapy or a support group can make a profound difference.

No one builds a career alone. We all need a containment network. It's okay not to have all the answers.

It's okay not to be okay all the time

Tech culture usually rewards constant performance, as if we were immutable. But we are human, with emotions, good and bad days. Validating your own feelings, even if they are not "productive" is critical to authentic emotional health.

Allowing yourself to be bad sometimes is what allows you to heal, grow, and move on.

Write the code for your own limits and values

As a programmer, you're used to designing systems with logical rules, conditions, and structures. But have you designed the system of your life with the same principles?

Defines:

- **Your non-negotiables**: Hours of rest, personal time, work limits.
- **Your guiding values**: What brought you here? What motivates you beyond salary?
- **Your early warnings**: Signs of burnout, anxiety, or disconnection.

Being intentional about your personal development not only makes you a better professional, but it protects you from building a brilliant but empty career.

Taking care of yourself is also part of the job

"Taking care of myself is also part of the job."

Just as you document code, check bugs, or plan a sprint, you also need to schedule breaks, sleep well, surround yourself with healthy relationships, and nurture your life outside of the keyboard.

You're no less valuable if you decide to stop. You are no less committed if you take care of your health. In fact, you are more complete, wiser, more sustainable.

Your mental health is a non-renewable resource

You can rewrite code. You can restart a system. But you can't replace your mental health once it's fully compromised. Take care of it with the same rigor with which you test your most delicate applications.

Don't wait for the collapse to make changes. Don't normalize suffering as part of growing up. Your well-being is not the price you pay to be in tech.

Your well-being is the foundation that makes it possible for you to be in tech.

The best version of you isn't the fastest, it's the most present

Build a career you don't need to escape from. A career that doesn't depend on shutting down to move forward. A career that allows you to be you, without masks or rush.

Because at the end of the day, the most important achievement will not be an impeccable line of code, but a professional life lived with balance, dignity and meaning.

Final Thoughts: Autonomy and Self-Discipline

Why is autonomy essential in learning and professional development?

Autonomy and Self-Discipline

Why is autonomy essential in learning and professional development?

In tech training, as in many other disciplines, relying on others for every step won't get you far. Learning to program is, above all, an exercise in autonomy. You decide how much time to spend, how to organize your study, when to ask for help, and when to insist a little more.

Autonomy does not mean doing everything alone. It means taking control of your process. It means to stop being a spectator who only follows tutorials, and to start being a protagonist who makes decisions, makes mistakes, reflects and improves.

But autonomy without self-discipline is not enough. Because you will have days when you don't feel like it, times when you don't understand anything, streaks when you think you're not made for this. And that's where self-discipline comes in: the ability to keep going, to stay consistent, even when there's no motivation or immediate results.

Know the phases of the journey before you start

Before launching yourself fully into the world of programming, I want to share something that I have learned with experience, not only my own, but also watching many people start this path, technical learning is a progressive process, and skipping phases is usually the first big obstacle.

<u>Common beginner mistakes:</u>

- Starting to program without understanding what programming is or what it is for.
- Copying examples without understanding what they do.
- Jumping into complex exercises without mastering the basics.
- Getting frustrated because "it doesn't work for them", without knowing why.

This pattern repeats itself repeatedly, and many times it is not a lack of ability, but a lack of method.

Each phase has its time and purpose

Programming training is like learning a new language or musical instrument. You can't write poetry without knowing grammar. You can't improvise a melody without mastering the scales.

Likewise, the path in programming begins with:

1. **Understand the fundamentals and logic**: Before writing code, you need to know how a program thinks.
2. **Practice with small exercises**: Go from simple to complex, step by step.
3. **Apply what you have learned in real projects**: Without copying but creating with your own ideas.
4. **Face mistakes and blockages with maturity**: Not as failures, but as part of the process.
5. **Ask for help and learn how to look for solutions**: Google, Stack Overflow, documentation... Knowing how to use these tools is also a skill.

Learning is also knowing when to stop

A key part of learning is **getting a good night's rest**. Many times, blockages are not solved by programming more, but by doing less.

When you feel stuck:

- Walk and get some air, take a conscious pause.
- Do mechanical activities tidying up, cleaning, cooking?
- Try something creative like drawing, writing, or listening to music.

This is not a waste of time. It's a strategy to free your mind and let ideas fall into place on their own. Personally, it works a lot for me. After a real pause, I come back with more clarity and new ideas.

Learn to identify your own rhythm

There will be times when you advance quickly, and others when it seems that you are not advancing at all. Both are normal. Don't measure yourself against others, measure yourself against yourself yesterday.

And if you feel like something isn't moving forward, ask yourself:

- Am I skipping any phases?
- Do I need to go back and reinforce the basics?
- How long has it been since I really rested?

Your path is unique, learn to move forward without running over yourself

In the world of learning and especially in technology, it's easy to fall into the trap of comparing yourself to others. You see someone who is already working in a company, another who already makes applications, or someone who learned in three months... and you start thinking, **"I'm late"** or **"I'm not good enough."**

But that is not true.

Each person has their own rhythm, their own challenges, their own context. And the important thing is not to get there quickly, but to move forward with meaning, without running over, without burning out, without losing enthusiasm along the way.

This is not a sprint

Learning to code is not like a 100-meter race where the fastest runner wins. It is more like a long hike, sometimes uphill, sometimes with beautiful landscapes, and sometimes with a lot of fatigue.

What you need is not running. It's endurance, patience and clarity about where you're going.

<u>Your key tools:</u>

Autonomy: Being able to make decisions about your learning. Know where to start, what you need to improve, and when to change strategy.

Self-discipline: Having the perseverance to study, practice and advance even when you don't feel like it or feel lost.

Self-knowledge: Understand how you learn best, when you need to rest, which topics are more difficult for you and which ones motivate you.

Self-care: Don't forget about yourself in the process. Sleep well, rest, eat, do things you enjoy, talk to people. Your mental health is also part of the journey.

The most important thing is not the code, it's you

<u>Yes, you will learn how to write code. But the most valuable thing you'll take away is this:</u>

- How to solve problems calmly.
- How to be consistent even when something doesn't go well.
- How not to give up at the first mistake?
- How to learn from your own stumbles.

And that will serve you not only in programming, but in everything you do.

Moving forward consciously > Moving forward fast

Don't worry if you're going slower than others. Move at your own pace but do it with awareness. Don't copy for the sake of copying. Don't run out of anxiety. Learn with intention.

The best version of you will not be the one who knows everything in a short time, but the one who understands their process, really learns, and takes care of themselves along the way.

If you understand this, you're already making better progress than many.

My path to professional development has been marked by a mixture of enthusiasm, uncertainty, falls and learning. It has not been a linear or predictable journey, but rather a process full of adjustments, iterations, unexpected mistakes and small triumphs that, accumulated, have taught me to trust my evolution.

Every technical obstacle, every moment of frustration in the face of a senseless mistake, and every day I questioned whether it was really worth it, have ended up being key pieces in my growth. Learning to code not only taught me how to write code, but also how to think better, tolerate frustration, and most of all, persevere with intention.

Most valuable lessons

1. Patience and tolerance to frustration.

It is normal to want to move quickly, to feel an urgency to learn everything. But I soon discovered that learning in technology is cumulative and takes time.

Frustration is not a sign of inadequacy, but a sign that you are challenging your limits. And that, if you know how to sustain it, is pure growth.

2. Progressive and structured learning

Skipping stages out of anxiety or comparing yourself to others only led to confusion. I understood that building a solid foundation, before jumping into complex frameworks or projects, is what really accelerates progress in the long run.

3. Organization and mental structure

Setting clear goals, planning what I was going to study, keeping my tools organized, and documenting what I learned, was key to sustaining my progress. Intellectual disorder is one of the quietest enemies when learning to program.

Autonomy as a pillar of development

One of the most powerful lessons has been understanding that autonomy is not optional. It is an essential skill in the tech world, where change is constant, challenges are varied, and where no one has all the answers.

Being able to research, experiment, and solve on my own has allowed me to move forward with independence and confidence.

1. Self-Management Skills

Learn to research well: Knowing how to search reliable sources, use precise keywords, and distinguish a good technical answer from an improvised solution is as important as writing good code.

Confidence in your abilities: Solving a mistake on your own, no matter how small, builds technical self-esteem. Every victory counts.

2. Critical thinking

Evaluate sources: Not everything that appears on the internet is useful. I learned to prioritize official documentation and well-explained answers.

Understand the "why": I'm no longer satisfied with something working. I want to understand it so I can adapt and scale it as needed.

3. Problem solving

Normalize mistakes: Every mistake is a clue, not a condemnation. I learned how to read logs, analyze the flow of code, and debug with logic.

Avoid quick fixes without understanding: Going straight to copying code without understanding it is a trap. The real solution is the one you know how to explain.

4. Time optimization

Search with intention: I learned how to use Google as a professional tool, operators, quotes, specific sites, etc.

Reduce dependencies: Not asking before researching for myself helped me to be more decisive and efficient.

5. Research habits

Personal documentation: Take notes, record commands, save useful links. It all adds up.

Building knowledge repositories: GitHub, Notion, or Markdown have become my learning and consultation allies.

6. Community Participation

Sharing is learning twice: Explaining a topic to someone else forces me to master it better.

Receive feedback: Participating in Telegram communities specialized in development and programming, having mentors helped me see mistakes that I didn't see and gave me new perspectives.

7. Encourage Independence

Solve before asking: Try, fail, try something new... and only then, ask for help if necessary. This has been key to my personal and technical development.

REFLECTION-CONCLUSION

Autonomy, critical thinking and resilience are not soft skills, they are hard and necessary skills for any developer in the 21st century.

This stage of training has confirmed to me that it is not about knowing everything, but about knowing how to learn, how to organize knowledge, how to ask with intention, and how to get up every time something does not go as expected.

Today I am not the same person who started this path. I have developed a more strategic, more conscious, and more patient approach. I care about both the quality of the code and the process I follow to get there.

My goal is not to become an expert who knows everything. My goal is to become a professional who moves forward with intention, who builds with meaning, and who does not stop learning, improving and sharing.

To you, who are just starting out:

Don't compare yourself. Don't rush. Don't beat yourself up. Move at your own pace. Ask humbly. Document your path. And when you feel like you're lost, go back to basics. There you will always find clarity.

This process is not a race. It is a construction. And you're building much more than software: You're building a new version of yourself.

How to build a successful career in the world of programming?

Starting a career in programming is not only about learning languages or carrying out flashy projects. It's about **respecting the process**, moving forward step by step, consolidating the fundamentals, and developing technical and personal skills that will sustain you over time.

A successful career in programming **isn't built in weeks or by skipping key stages**. It requires time, perseverance, curiosity, and a strategic approach.

Starting from the simple, moving forward step by step and accepting mistakes as part of growth will allow you to become not only someone who knows how to program, but also a professional.

Here's why a **progressive and structured** approach is essential to become a competent, confident and competitive professional.

1. Consolidation of the fundamentals

Learn before applying: A deep understanding of variables, control structures, functions, lists, maps, and data structures prepares you to tackle any technology or language with robustness.

Intentional exercises**:** Practicing with basic challenges allows you to automate mental processes and gain technical confidence without overloading yourself.

2. Avoid frustration and mental overload

Learn progressively: Starting with simple examples avoids the confusion of facing complex projects without preparation.

Small achievements that motivate: Each solved exercise fuels your confidence and drives you to move forward with enthusiasm.

3. Development of problem-solving skills

Algorithmic thinking: Solving small problems helps you think like a programmer: decompose, test, debug, and refine solutions.

Structured strategy**:** This skill is key in technical interviews, teamwork, and bug solving in real environments.

4. Improved use of tools and technologies

Gradual familiarization: IDEs, Git, terminals, frameworks or testing tools become more understandable when they are introduced progressively.

Confidence in automation: Understanding how and when to use tools makes you more productive and less dependent on external solutions.

5. Learn from mistakes and continuously improve

Bugs as allies: In small activities, bugs are less costly and easier to debug. They allow you to develop patience, precision and continuous improvement.

Iterate to move forward**:** Conscious, targeted repetition is one of the keys to technical mastery.

6. Preparation for real projects

Apply what you have learned with purpose: Small projects allow you to put concepts together and solve problems with practical sense.

Scale with logic: This practice gives you the tools to tackle larger projects without losing control or motivation.

7. Building a CV with Solid Projects

Well-designed projects: Having a few well-finished, documented, and explained projects communicates professionalism more than dozens of empty or broken repositories.

Reflection of your capabilities**:** Your portfolio speaks for you. The clearer and more polished it is, the more doors it will open for you.

8. Increased self-confidence

Results-based security: Seeing your progress from basic exercises to functional projects gives you real validation, not just theoretical.

Constant motivation: This approach makes learning an enjoyable process, not a source of frustration or anxiety.

Conclusion: Lessons Learned and Survival Guide for Beginner Programmers

Reflections for those who are starting out: From chaos to clarity.

This book was born out of a need, both personal and professional.

I wanted to put my learnings in order, give value to my evolution and share those lessons with those who, like me, are taking their first steps in the tech world. Because I know what it's like to feel lost among tutorials, doubts and lines of code that don't work. I also know what it means to feel small in the face of job offers that seem to demand too much.

Over time, I learned something that professional development is not a straight line. It is a road full of curves, unexpected stops, setbacks and constant corrections.

Sometimes it seems that you take three steps forward and go back one; other times, you take two steps forward and two steps back. But every one of those steps counts. Every line of code written, every project uploaded to GitHub, every concept that at first seemed Chinese and now starts to make sense... everything adds up.

Every mistake, every bug, every time I had to redo something, have been the invisible bricks that today support my profile as a developer.

Beyond the languages, frameworks and technical tools, this journey has also been a journey of self-knowledge. I have learned to recognize my limits, which still exist, but also to identify strengths that I did not see clearly before.

That feedback has come both from those who are starting out in the tech sector and debate in my Telegram groups, and from my mentors, who have guided me in the last phases of my learning.

They have helped me not only technically, but also emotionally, accompanying me in my doubts and insecurities, and giving me that push I needed in the most difficult moments.

I have understood that making mistakes is not synonymous with failure, but part of the workshop where character and technical skill are forged.

This eBook is just the beginning. It is the base, a starting guide and accompaniment for those who are still defining their specialization, who feel that they are in professional transition, or who believe that their portfolio does not yet reflect all that they can contribute.

Every line of code you write, every improvement you apply to your CV, every message you send nervously, every doubt you manage to solve... all of that is already making you grow. Even if you don't see it right away, you're weaving your experience and building your professional voice.

A chapter of my life that gave me confidence

In August something happened that made me see that all the work I had been doing was beginning to have an echo. I was contacted by two companies through LinkedIn. Having an optimized profile, with more than 3,700 followers, is not essential, but it does help to gain visibility.

It positions you as a tech professional when you share quality content, add value to the community and actively participate in debates with a critical eye, in the midst of an environment where superficial goodwill sometimes predominates.

Standing out is not a matter of appearing, but of making a difference with arguments, sharing lessons learned and acknowledging mistakes. Because we all make mistakes, and the important thing is to learn so as not to repeat the same mistakes over and over again.

You don't need to be an influencer. What really matters is that your professional profile on LinkedIn works as a coherent extension of your personal brand, in tune with your GitHub or GitLab. Every *commit*, every project, every technical improvement is one more step in your journey. Everything counts, more than you imagine.

It was no coincidence. One company wrote to me by email, another by private message, both with job offers. The messages came after I updated my profile, built my own projects, uploaded repositories, and spent time improving my CV presentation.

And that's when the nerves appeared

The first interviews with Human Resources by video call, those conversations in which you try to take a deep breath, explain what you know, show what you do. I learned from each one, what questions they asked me, how I felt, what I could improve for next time.

That's when I saw my LinkedIn start to bear fruit. My profile stood out enough for companies to seek me out. I realized that it wasn't just about what I knew, but how I displayed it:

- The CV format
- Clarity in presentation
- The projects I chose to share
- The way I expressed my goals

I forwarded my CV with a more professional and straightforward design. The first occupied two pages full of disordered information. The second, cleaner, clearer, also included the unofficial training he had acquired in these years, with and without a degree. In total, four well-structured pages.

Then came the waiting, calls, emails resolving doubts about conditions, pre-contracts, technical tests, whether I would work for internal or external clients. I answered every message, I tracked every step of the process: date, company, status. I wrote down what they answered me, what I changed, what I learned. That record gave me confidence.

Until that moment, he had never done formal technical tests. He knew that there were still phases to overcome. The technical tests, the temporary contract in the probationary period ... and dreaming of the indefinite contract.

What's next: Part Two

Survival Guide for Beginner Programmers Part 2: From Search to First Job

While writing this eBook, my mentor suggested I create a second part. At the time, I was developing the three-star projects for my portfolio, fine-tuning the CV and writing cover letters tailored to each offer. All that work deserved its own space.

My plan was to continue with the phases that still needed to be addressed, the ones I mentioned earlier, but to do so in a new book. Therefore, this first volume focuses on phases 1 to 5, and the next volume will cover phases 6 to 11.

Looking ahead to 2026

My goal is to unify both books in a single work that reflects the entire journey. From initial training to professional consolidation as an active part of a tech team. And as a final touch, I will include a gift bonus with resources, learnings and tools that have accompanied me on this journey.

But everything has its time.

Today, with this first book, I close a stage. A stage of searching, of construction, of many questions and some answers. And at the same time, I prepare the ground for what is to come: interviews, technical tests, first contracts, first real challenges as a team.

Thanks to you, reader

Thank you for joining me on this journey. Thank you for reading, for reflecting, for allowing you to grow together with these pages. I hope this book has been as useful to you as it was to me in writing it.

That you find here a map to orient yourself, a mirror to recognize yourself and a little company so you don't feel alone on the road. Because this is not the end.

It's just a new starting point.

Annex

Free resources for learning with and without certification Recommended platforms, tools, and links.

Recommended platforms, tools and links to train you in programming and technology from scratch.

Below, you'll find a selection of platforms, training programs, and learning channels that offer free content, some officially certified, in areas such as web development, programming, artificial intelligence, networking, cybersecurity, and more. They are organized by global reach and by specific resources for Spain and Latin America.

Global Platforms

FreeCodeCamp

https://www.freecodecamp.org
One of the most comprehensive platforms for learning web development, JavaScript, Python, data structures, databases, and more. It offers hands-on projects and highly-rated free certifications.

Roadmap.sh

https://roadmap.sh
Interactive visual guides that guide you on what to learn according to your goal: FrontEnd, backend, DevOps, artificial intelligence, etc. Ideal for structuring your training process.

Cisco Networking Academy

https://www.netacad.com
Free courses on networking, Python, cybersecurity, Linux, IoT, and more. It includes internationally recognized certifications. Access through training centres or individually.

Microsoft Learn

https://learn.microsoft.com
Free official training on Microsoft technologies such as Azure, Power BI, .NET, GitHub, and web development. Many learning paths offer free certification.

Google Career Certificates

https://grow.google/intl/es_es/certificates
Scholarships for professional training in IT support, data analysis, cybersecurity and more. In Spain, they are free through Fundae. Estimated duration: 6 months.

Train for Employment (Carlos Slim Foundation)

https://capacitateparaelempleo.org
Free training platform with certification. It offers courses in programming, networking, technical support, digital skills, and more. Available throughout Latin America.

Specific resources for Spain

Free courses from SEPE and FUNDAE

https://www.sepe.es

Courses aimed at unemployed and employed people. Many are focused on emerging technologies and are taught in collaboration with training centers and technology companies.

Certificates of Professionalism in Computer Science (TodoFP)

https://www.todofp.es

Official and free technical training in development, networks, support, etc. It requires registration for specific calls, but offers direct job recognition.

Professional specialization courses (Ministry of Education)

Free training in strategic areas such as AI, full-stack development, cybersecurity or big data. Taught in Vocational Training centres in Spain. Limited places.

Telegram groups and channels to learn programming

Telegram is a key space to access free resources, courses, books, guides, and technical discussions. Here are some recommended groups and channels in Spanish and English:

In Spanish

Programming Group for Beginners Community to share resources, solve doubts and accompany those who are starting out. Courses, guidance and support are offered.

I Want to Learn Java Space dedicated to collaborative learning of the Java language. Ideal for answering questions, sharing books and learning in community.

Books Programming – Computer Science now renamed Programming and Computer Science Books Open access group (requires manual search on Telegram) with a shared library of technical books in PDF and ePub. Very varied content.

Dev 101 English A channel that constantly publishes free courses, bootcamps, and resources on web development, programming, and popular frameworks.

In English

IT Cloud Automation Channel focused on technologies such as cloud computing, DevOps, QA testing, continuous integration and automation.

Dev 101 English International version of the previous channel. It offers comprehensive courses, up-to-date bootcamps, and training organized by topics and technologies.

Recommended use

To get the most out of Telegram groups:

- Use the desktop version to better organize downloads and view large content.
- Interact in chats to solve doubts or provide knowledge.
- Create folders or bookmarks by technologies/languages to structure what you learn.

Final recommendation

Regardless of your level, these resources can help you structure your learning, strengthen your technical skills, and prepare you to make the leap into the job market. The key is consistency, curiosity and commitment to your own process.